



Douzone Cloud DX 6th.	
주제	냉장고 속 재료 기반 레시피 추천 및 식단, 식비 관리 서비스 플랫폼
팀원	김봉수, 이건우, 임정현, 현정은, 윤태현
강사	이현용
프로젝트 기간	2026.02.25 - 2026.08.26
github / site	https://app.mealbong.cloud · https://aws.mealbong.cloud

목차

목차를 갱신하려면 이 영역을 클릭하고 F9 를 누르세요.

1. 프로젝트 개요

1.1 개발 배경

2. 프로젝트 설계

2.1 기술 설계

2.2 기능 설계

2.3 서비스 설계

2.4 ERD 설계

2.5 디자인 설계

2.6 협업 방식

3. 핵심 기술

3.1 AI 재료 인식과 레시피 추천

3.2 데이터 수집 파이프라인

4. 시스템 구성

4.0 시스템 구성 개요, 3 단계 전환

4.1 AWS 구성도

4.2 AWS EKS 클러스터 구성도

4.3 Kubernetes 사용 기술

4.3.1 클러스터 구성

4.3.2 Cilium, CNI 및 kube-proxy 대체

4.3.3 Istio, 서비스 메시

4.3.4 진입 구성, Gateway API · ALB · ACM · WAF

4.3.5 리소스 관리 · 우선순위

4.3.6 오토스케일, HPA · KEDA · Karpenter

4.3.7 스토리지

4.3.8 데이터 티어

4.3.9 배포, ArgoCD GitOps · Blue-Green

4.4 GitLab CI/CD

4.5 Observability

4.5.1 Docker 기반 모니터링 구축

4.5.2 Kubernetes 기반 모니터링 구축

4.5.3 AWS EKS 기반 Observability 이전

4.5.4 이상징후 Dashboard 구축

4.5.5 장애 근거 데이터 기반 Bedrock 원인 분석 및 운영 자동화

4.6 Serverless 구성도

4.7 Serverless 사용 기술

4.8 Backup

4.9 Security

4.9.1 IAM · IRSA, 신원과 권한

4.9.2 Security Group

4.9.3 NetworkPolicy · mTLS

4.9.4 Secret Manager · External Secrets

4.9.5 WAF · GuardDuty · CloudTrail

4.10 Terraform 을 활용한 IaC

5. 프로젝트 수행 결과

5.1 개발 환경

5.2 구현 내용

1. 프로젝트 개요

1.1 개발 배경

(1) 문제 정의

식비는 가게 지출에서 큰 비중을 차지하지만, 다른 지출과 달리 매일 여러 번에 걸쳐 조금씩 발생한다. 그래서 한 달이 끝나고 나서야 얼마를 썼는지 알게 되는 경우가 많다. 특히 자취를 하거나 처음 살림을 시작한 사람에게는 한 끼에 얼마가 드는지조차 감이 잡히지 않는다.

장을 볼 때의 어려움은 무엇을 먹을지 고르는 데 있지 않다. 이번 달에 얼마까지 쓸 수 있는지, 이 요리를 하면 예산이 얼마나 남는지를 판단하기 어렵다는 데 있다. 레시피 사이트는 요리하는 방법을 알려주지만 그 요리에 드는 돈은 알려주지 않고, 가계부 앱은 이미 쓴 돈을 기록하지만 앞으로 무엇을 먹을지는 돕지 않는다. 두 축이 서로 끊겨 있다.

여기에 냉장고 문제가 겹친다. 이미 가지고 있는 재료를 잊어버리고 다시 사거나, 사 둔 재료를 쓰지 못하고 버리는 일이 반복된다. 같은 재료라도 마켓마다 가격이 다르지만, 이를 매번 비교하는 것은 번거롭고 시간이 걸린다.

이러한 문제를 해결하려면 예산과 레시피와 장바구니와 가계부가 하나의 흐름으로 이어져야 한다. 우리 프로젝트는 이 요구를 충족시키기 위해 냉장고 속 재료를 기준으로 레시피를 추천하고, 그 재료비를 실제 판매 가격으로 계산하며, 소비 내역이 자동으로 기록되는 서비스를 개발하고자 한다.

(2) 요구사항 정의

앞서 언급한 문제를 해결하기 위해 다음 항목을 만족하는 서비스를 개발하고자 했다.

첫째, 월 식비 예산을 설정하고 남은 예산이 실시간으로 갱신되도록 한다.

둘째, 레시피를 고르면 그 요리에 실제로 쓰는 양을 기준으로 재료비가 계산되도록 한다.

셋째, 재료마다 여러 마켓의 현재 판매가를 비교하여 선택할 수 있도록 한다.

넷째, 냉장고에 이미 있는 재료와 상비양념은 장바구니에서 자동으로 제외되도록 한다.

다섯째, 영수증 사진 한 장으로 냉장고 재고와 식비가 함께 기록되도록 한다.

이를 만족하는 월 식비 예산 기반 밀플래닝 서비스인

밥풀이(Mealbong)를 개발하였다. 밥풀이는 냉장고 속 재료를 기준으로 만들 수 있는 요리를 추천하고, 그 재료비를 마켓 현재가로 계산해 예산과 함께 보여주는 서비스이며 다음과 같은 기능을 요구한다.

예산 관리 서비스

월 식비 예산을 정하고 소비에 따라 잔액을 추적한다.

레시피 추천 서비스

냉장고 보유 재료로 만들 수 있는 요리를 추천한다.

재료비 산출 서비스

레시피에 쓰이는 양을 기준으로 비용을 계산한다.

마켓 비교 서비스

품목마다 여러 마켓의 현재가를 비교해 선택할 수 있다.

냉장고 관리 서비스

보유 재료와 소비기한을 관리하고 만료 임박 재료를 알린다.

AI 인식 서비스

영수증 사진과 요리 영상에서 재료 정보를 자동으로 추출한다.

(3) 기대 효과

냉장고 속 재료를 기준으로 레시피를 추천하고 그 재료비를 마켓 현재가로 계산해주는 서비스이다. 이 서비스는 사용자들이 식비 예산 안에서 식단을 계획하고, 가지고 있는 재료를 낭비 없이 사용할 수 있도록 돕는 것을 목표로 한다.

이 서비스는 예산 제약이 뚜렷한 자취생과 1인 가구에게 특히 도움이 되며, 요리를 고르는 행위가 곧 예산 판단이 되도록 만든다. 또한 사용자는 여러 쇼핑물을 오가며 가격을 비교할 필요 없이 한 화면에서 모든 정보를 확인할 수 있어 장보기에 드는 시간과 노력을 크게 줄일 수 있다. 서비스 관리자는 축적된 소비 데이터와 레시피 선택 기록을 통해 사용자 경향을 분석하고, 개인화된 추천 품질을 지속적으로 개선할 수 있다. 이로 인해 식자재 낭비를 줄이고 합리적인 소비 습관을 형성하는데 기여할 수 있을 것으로 기대한다.

2. 프로젝트 설계

2.1 기술 설계



- AWS EKS 를 통해 기본적인 시스템을 구축
- MSA 를 구축하기 위해 Service Mesh 도구인 Istio 를 사용
- CNI 는 Cilium 을 사용해 eBPF 기반으로 kube-proxy 를 대체
- Observability 는 kube-prometheus-stack 과 Loki, Tempo, Alloy 를 사용

- Terraform 으로 클라우드 자원을, Ansible 로 노드 베이스라인을 구성해 코드형 인프라(IaC) 구축
- AI 추론은 전량 CPU 에서 수행하며 CRF, XGBoost, LightGBM 을 사용
- 생성형 AI 는 Amazon Bedrock 과 Google Gemini, Vertex AI 를 API 로 호출하여 사용
- AI 기능은 AWS Lambda 로 별도 구성하여 서버리스로도 동작하도록 구현
- 우리 서비스의 Database 로 PostgreSQL 을 선정하고 CloudNativePG 로 자체 운영
- 레시피와 상품 검색을 위해 Elasticsearch 를 nori 형태소 분석기와 함께 사용
- 캐시와 세션 저장소로 ElastiCache for Valkey 를 사용
- 크롤링 데이터 전달에 Kafka 를 사용하고 KEDA 로 소비자를 자동 확장
- GitLab 은 EC2 에서 따로 구축하고 CD 는 ArgoCD 로 구성
- Backend 와 AI, 데이터 파이프라인 모두 Python(FastAPI), Frontend 는 React 와 Vite(PWA)를 사용
- 지라로 일정 관리, 노션으로 문서 관리, Slack 으로 연락을 함

2.2 기능 설계

기능 명세서

구분	기능	설명
계정	소셜 로그인	카카오, 구글 OAuth 로그인과 이메일 가입을 지원

예산	월 예산 설정	월 식비 예산을 정하고 소비에 따라 잔액을 계산
예산	식비 캘린더	일자별 식비를 달력에 표시하고 예산 대비 소비를 추적
레시피	검색	Elasticsearch nori 형태소 분석 기반 한국어 검색
레시피	추천	냉장고 보유 재료 기준으로 만들 수 있는 요리를 추천
레시피	재료비 산출	요리에 쓰는 양 기준으로 비용 계산, 상비양념은 제외
레시피	레시피북	직접 작성한 레시피를 저장하고 공유
장바구니	마켓 비교 담기	품목별로 마켓컬리와 오아시스마켓 가격을 비교해 선택
냉장고	재고 관리	보유 재료와 수량을 등록하고 관리
냉장고	소비기한 관리	만료 임박 재료를 홈 화면에서 안내
AI	영수증 인식	영수증 사진에서 품목과 금액을 추출해 재고와 식비에 반영
AI	영상 레시피 추출	유튜브 URL 에서 재료와 조리 단계를 구조화
AI	챗 어시스턴트	자연어 질의로 레시피와 가격을 조회
AI	최저가 알림	관심 품목의 가격이 급락하면 알림을 발송

2.3 서비스 설계

- 앱 실행 하면 랜딩페이지가 나옴.
- 로그인이 되어있다면 바로 홈 화면으로, 되어있지 않다면 소셜 로그인을 통해 인증을 받고 로그인 후 홈 화면으로 넘어감.
- 홈 화면에서 이번 달 예산 잔액과 냉장고의 만료 임박 재료를 함께 확인 가능

- 레시피 추천은 냉장고에 있는 재료를 기준으로 후보를 뽑고, 개인화 모델이 순서를 다시 정함
- 레시피 상세에서 재료비가 요리에 쓰는 양 기준으로 표시되고, 총 재료비를 예산과 비교 가능
- 담기 모달에서 품목마다 마켓을 고를 수 있고, 냉장고에 있는 재료와 상비양념은 기본으로 선택이 해제되어 있음
- 선택을 마치면 장바구니에 담기고, 결제 금액이 식비 캘린더에 자동으로 기록됨
- 영수증은 촬영하거나 갤러리에서 고른 뒤 인식 결과를 사용자가 확인하고 저장
- 챗 어시스턴트에서 자연어로 물어보면 레시피 카드와 가격을 답으로 받음
- 마이페이지에서는 예산 설정 변경과 로그아웃이 가능

2.4 ERD 설계

초기 PostgreSQL 설계

요구 사항 수집 분석

사용자 요구 사항

- **회원가입 및 로그인 기능**
 - 사용자는 이메일과 비밀번호를 이용하여 회원가입 및 로그인할 수 있어야 한다.
 - 사용자는 카카오 또는 구글 계정을 이용하여 소셜 로그인할 수 있어야 한다.

- 소셜 로그인 시 이메일, 닉네임, 소셜 제공자 식별자 등의 사용자 정보를 가져와야 한다.
- 로그인 성공 시 사용자 식별정보가 포함된 JWT 를 발급해야 한다.
- 월 식비 예산 관리 기능
 - 사용자는 월별 식비 예산을 등록하고 수정할 수 있어야 한다.
 - 사용자는 현재까지 사용한 식비와 남은 예산을 확인할 수 있어야 한다.
 - 사용자는 이전 달의 예산과 지출 내역을 확인할 수 있어야 한다.
- 레시피 검색 기능
 - 사용자는 이름, 카테고리, 조리시간 등을 기준으로 레시피를 검색할 수 있어야 한다.
 - 사용자는 레시피의 재료, 조리순서, 인분, 난이도 및 영양정보를 확인할 수 있어야 한다.
 - 레시피 재료는 표준 품목과 연결되어야 한다.
- 레시피 추천 기능
 - 사용자는 냉장고에 보유한 재료를 기반으로 레시피를 추천받을 수 있어야 한다.
 - 사용자의 월 예산과 현재 식비 지출을 고려하여 레시피를 추천해야 한다.
 - 사용자가 등록한 제외 재료가 포함된 레시피는 추천 대상에서 제외해야 한다.
 - 사용자의 조회 및 장바구니 추가 이력을 활용하여 개인화된 추천을 제공할 수 있어야 한다.
- 개인 레시피북 기능
 - 사용자는 원하는 공용 레시피를 북마크할 수 있어야 한다.

- 사용자는 직접 작성한 레시피를 저장할 수 있어야 한다.
- 사용자는 YouTube URL 을 입력하여 영상에서 추출된 레시피를 저장할 수 있어야 한다.
- 사용자는 자신의 레시피를 수정하거나 삭제할 수 있어야 한다.
- 사용자는 자신이 작성한 레시피를 공유 링크 또는 공개 레시피로 발행할 수 있어야 한다.
- **냉장고 재고 관리 기능**
 - 사용자는 보유한 식재료를 냉장고 재고에 등록할 수 있어야 한다.
 - 사용자는 식재료의 이름, 수량, 보관 위치 및 소비기한을 입력할 수 있어야 한다.
 - 사용자는 등록한 재고를 수정하거나 삭제할 수 있어야 한다.
 - 사용자는 재고 상태를 보유 중, 소비 완료, 폐기로 변경할 수 있어야 한다.
 - 사용자는 소비기한이 임박한 식재료를 확인할 수 있어야 한다.
- **영수증 OCR 기능**
 - 사용자는 종이 영수증 또는 온라인 구매내역 이미지를 업로드할 수 있어야 한다.
 - 시스템은 영수증에서 매장명, 구매일자, 총금액 및 구매 품목을 추출해야 한다.
 - 사용자는 OCR 로 인식된 품목명, 수량 및 가격을 확인하고 수정할 수 있어야 한다.
 - 사용자가确定的 식품 항목은 냉장고 재고에 등록할 수 있어야 한다.
 - 영수증의 총금액은 식비 지출 내역에 반영할 수 있어야 한다.
- **식재료 가격 비교 기능**

- 사용자는 마켓컬리와 오아시스에서 판매되는 식재료의 가격을 비교할 수 있어야 한다.
- 서로 다른 용량의 상품을 비교할 수 있도록 100g, 100ml 또는 개당 가격을 제공해야 한다.
- 사용자는 상품의 현재 가격, 정상 가격, 할인율 및 핫딜 여부를 확인할 수 있어야 한다.
- 상품 가격은 수집 시점별로 저장하여 가격 변화를 확인할 수 있어야 한다.

● 장바구니 관리 기능

- 사용자는 레시피에 필요한 재료를 장바구니에 추가할 수 있어야 한다.
- 사용자는 냉장고에 보유한 재료를 확인한 뒤 구매할 품목을 선택할 수 있어야 한다.
- 동일한 표준 품목이 여러 번 추가되면 하나의 항목으로 합치고 수량을 증가시켜야 한다.
- 사용자는 장바구니 항목을 수정하거나 삭제할 수 있어야 한다.
- 사용자는 장바구니의 예상 구매 금액과 남은 예산을 확인할 수 있어야 한다.

● 식비 지출 관리 기능

- 사용자는 장보기, 외식, 배달, 기타 항목으로 지출을 등록할 수 있어야 한다.
- 지출은 수동 입력, 영수증 OCR 또는 장바구니 구매를 통해 생성할 수 있어야 한다.
- 사용자는 일자별 지출 내역을 캘린더에서 확인할 수 있어야 한다.

- 장바구니 구매 완료 시 장바구니 항목 삭제와 지출 생성이 함께 처리되어야 한다.

● 관심 가격 등록 기능

- 사용자는 가격 변화를 확인하고 싶은 식재료를 관심 품목으로 등록할 수 있어야 한다.
- 사용자는 관심 품목을 여러 개 등록할 수 있어야 한다.
- 사용자는 등록한 관심 품목을 삭제할 수 있어야 한다.
- 동일한 품목을 중복 등록할 수 없어야 한다.

● 알림 기능

- 사용자는 최저가, 소비기한 임박, 핫딜 및 예산 관련 알림을 받을 수 있어야 한다.
- 사용자는 알림 목록을 최신순으로 확인할 수 있어야 한다.
- 사용자는 알림을 읽음 상태로 변경할 수 있어야 한다.
- 사용자는 알림 유형별 수신 여부를 설정할 수 있어야 한다.

● 챗봇 기능

- 사용자는 냉장고 재고, 식비 예산, 레시피 및 가격에 관한 질문을 할 수 있어야 한다.
- 챗봇은 하나의 세션 안에서 이전 대화 내용을 활용할 수 있어야 한다.
- 사용자 동의를 받은 경우에만 챗봇 대화 내용을 저장해야 한다.
- 저장된 대화 내용은 사용자 선호도 분석과 추천 기능에 활용할 수 있어야 한다.

● 사용자 행동 데이터 수집 기능

- 시스템은 사용자의 레시피 조회, 장바구니 추가 및 알림 클릭 행동을 기록할 수 있어야 한다.

- 시스템은 사용자에게 노출한 추천 레시피와 노출 순위를 기록할 수 있어야 한다.
 - 사용자 동의를 받은 경우에만 행동 데이터를 저장해야 한다.
 - 수집된 행동 데이터는 개인화 레시피 추천 모델에 활용할 수 있어야 한다.
-

개념적 설계

주요 엔티티 도출

APP_USER (사용자 정보)

- user_id (PK)
- email
- password_hash
- nickname
- provider
- provider_uid
- activity_consent
- consented_at
- created_at
- updated_at

USER_BUDGET (월별 식비 예산)

- budget_id (PK)
- user_id (FK)
- month
- amount

- created_at

USER_EXCLUDED_ITEM (사용자 제외 재료)

- excluded_item_id (PK)
- user_id (FK)
- item_id (FK)
- name
- created_at

ITEM_MASTER (표준 품목 정보)

- item_id (PK)
- canonical_name
- category
- note

ITEM_ALIAS (표준 품목 별칭)

- alias (PK)
- item_id (FK)
- source

FOOD_NUTRITION (식품 영양정보)

- food_cd (PK)
- item_id (FK)
- food_name
- serving_g
- kcal
- carb_g
- protein_g

- fat_g
- sugar_g
- sodium_mg
- fetched_at

SHELF_LIFE_REF (소비기한 참조정보)

- shelf_life_id (PK)
- item_id (FK)
- source
- food_category
- item_name
- storage
- days_min
- days_max
- note
- fetched_at

RECIPE (공용 레시피 정보)

- recipe_id (PK)
- source
- src_recipe_id
- name
- category
- cook_method
- cooking_time
- level_nm

- kcal
- carb_g
- protein_g
- fat_g
- sodium_mg
- serving
- image_url
- fetched_at

RECIPE_INGREDIENT (레시피 재료)

- recipe_ingredient_id (PK)
- recipe_id (FK)
- item_id (FK)
- seq
- ingredient_name
- quantity
- ingredient_raw
- ner_status
- is_non_ingredient

RECIPE_STEP (레시피 조리순서)

- recipe_step_id (PK)
- recipe_id (FK)
- step_no
- description
- image_url

BOOKMARK (공용 레시피 북마크)

- bookmark_id (PK)
- user_id (FK)
- recipe_id (FK)
- created_at

USER_RECIPE (사용자 작성 레시피)

- user_recipe_id (PK)
- user_id (FK)
- origin
- title
- source_url
- image_url
- ingredients
- steps
- cooking_time
- serving
- level_nm
- is_public
- share_token
- created_at

SHARED_RECIPE (사용자 공개 레시피)

- shared_recipe_id (PK)
- user_recipe_id (FK)
- user_id (FK)

- title
- image_url
- ingredients
- steps
- source_url
- origin
- share_token
- published_at

RECIPE_EXTRACT_JOB (영상 레시피 추출 작업)

- extract_job_id (PK)
- user_id (FK)
- user_recipe_id (FK)
- url
- status
- result
- created_at

PANTRY_ITEM (냉장고 재고)

- pantry_item_id (PK)
- user_id (FK)
- item_id (FK)
- name
- quantity
- storage
- expire_at

- source
- status
- created_at
- closed_at

OCR_RECEIPT (OCR 영수증)

- receipt_id (PK)
- user_id (FK)
- status
- store
- purchased_at
- total_amount
- created_at

OCR_RECEIPT_ITEM (OCR 영수증 품목)

- receipt_item_id (PK)
- receipt_id (FK)
- item_id (FK)
- raw_text
- name
- quantity
- price
- is_food
- confirmed

RETAIL_PRODUCT (판매상품 정보)

- retail_product_id (PK)

- item_id (FK)
- source
- product_id
- name
- name_norm
- weight_g
- volume_ml
- category
- url
- image_url
- storage
- origin
- expiry_text
- first_seen
- last_seen

RETAIL_PRICE (판매상품 가격 이력)

- retail_product_id (PK, FK)
- crawled_at (PK)
- price
- original_price
- discount_rate
- deal_type
- timeddeal_end
- unit_price

- unit_basis
- is_sold_out

CART_ITEM (장바구니 항목)

- cart_item_id (PK)
- user_id (FK)
- retail_product_id (FK)
- recipe_id (FK)
- item_id (FK)
- name
- qty
- quantity
- added_at

EXPENSE (식비 지출)

- expense_id (PK)
- user_id (FK)
- receipt_id (FK)
- amount
- category
- spent_on
- memo
- source
- created_at

PRICE_WATCH (관심 가격 품목)

- price_watch_id (PK)

- user_id (FK)
- item_id (FK)
- created_at

NOTIFICATION (사용자 알림)

- notification_id (PK)
- user_id (FK)
- type
- title
- body
- payload
- is_read
- created_at

NOTIFICATION_SETTING (알림 수신 설정)

- user_id (PK, FK)
- low_price
- expiry
- hotdeal
- budget
- updated_at

CHAT_MESSAGE (챗봇 대화)

- chat_message_id (PK)
- user_id (FK)
- session_id
- role

- text
- intent
- item_ids
- unanswered
- basis
- created_at

USER_EVENT (사용자 행동 이벤트)

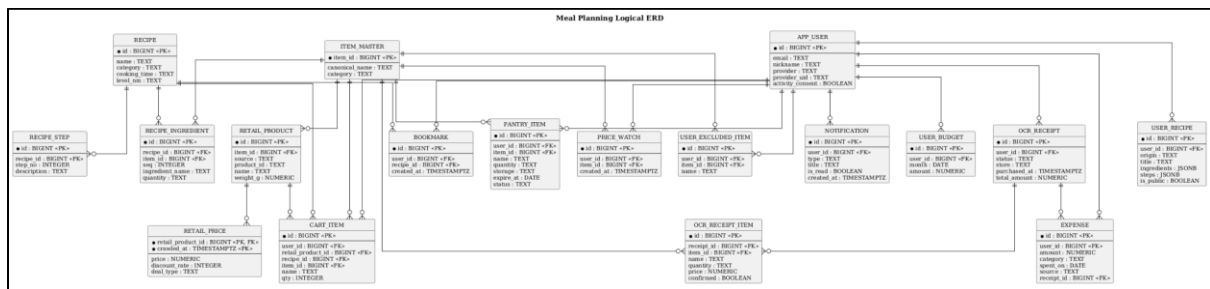
- user_event_id (PK)
- event_id (UK)
- user_id (FK)
- session_id
- event_type
- recipe_id (FK)
- item_id (FK)
- occurred_at
- context
- created_at

RECIPE_IMPRESSION (추천 레시피 노출 이력)

- recipe_impression_id (PK)
- impression_id (UK)
- user_id (FK)
- session_id
- recipe_id (FK)
- rank

- rule_score
- score_stock
- score_expiry
- score_cost
- request_ctx
- shown_at

논리적 설계



물리적 설계

표준 품목 정보

```
CREATE TABLE `item_master` (  
  `item_id` BIGSERIAL NOT NULL,  
  `canonical_name` TEXT NOT NULL,  
  `category` TEXT NULL,  
  `note` TEXT NULL  
);
```

표준 품목 별칭 정보

```
CREATE TABLE `item_alias` (  
  `alias` TEXT NOT NULL,  
  `item_id` BIGINT NOT NULL,  
  `source` TEXT NULL  
);
```

식품 영양정보

```
CREATE TABLE `food_nutrition` (  
  `food_cd` TEXT NOT NULL,  
  `food_name` TEXT NOT NULL,  
  `serving_g` NUMERICNULL,  
  `kcal` NUMERICNULL,  
  `carb_g` NUMERICNULL,  
  `protein_g` NUMERICNULL,  
  `fat_g` NUMERICNULL,  
  `sugar_g` NUMERICNULL,  
  `sodium_mg` NUMERICNULL,  
  `item_id` BIGINT NULL,  
  `fetched_at` TIMESTAMPTZ NOT NULL  
);
```

소비기한 참조정보

```
CREATE TABLE `shelf_life_ref` (  
  `id` BIGSERIAL NOT NULL,  
  `source` TEXT NOT NULL,  
  `food_category` TEXT NULL,  
  `item_name` TEXT NOT NULL,  
  `storage` TEXT NOT NULL,  
  `days_min` INTEGER NULL,  
  `days_max` INTEGER NULL,  
  `note` TEXT NULL,  
  `item_id` BIGINT NULL,  
  `fetched_at` TIMESTAMPTZ NOT NULL  
);
```

공용 레시피 정보

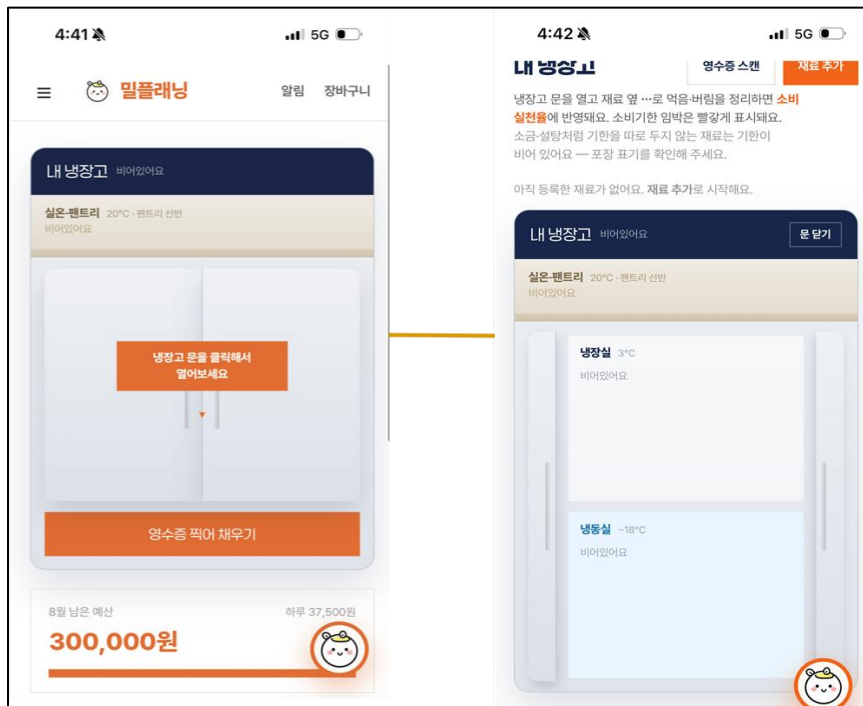
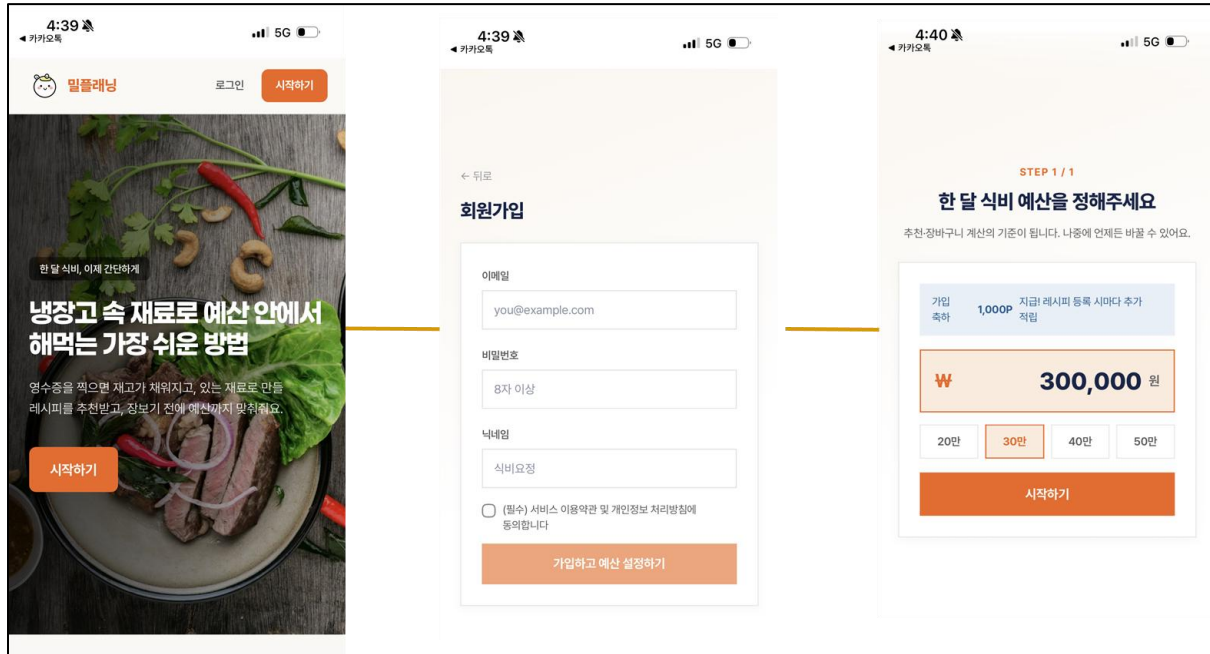
```
CREATE TABLE `recipe` (  
  `id` BIGSERIAL NOT NULL,  
  `source` TEXT NOT NULL,  
  `src_recipe_id` TEXT NOT NULL,  
  `name` TEXT NOT NULL,  
  `category` TEXT NULL,  
  `cook_method` TEXT NULL,  
  `cooking_time` TEXT NULL,  
  `level_nm` TEXT NULL,  
  `kcal` NUMERICNULL,  
  `carb_g` NUMERICNULL,  
  `protein_g` NUMERICNULL,  
  `fat_g` NUMERICNULL,  
  `sodium_mg` NUMERICNULL,  
  `serving` TEXT NULL,  
  `image_url` TEXT NULL,  
  `fetched_at` TIMESTAMPTZ NOT NULL  
);
```

레시피 재료 정보

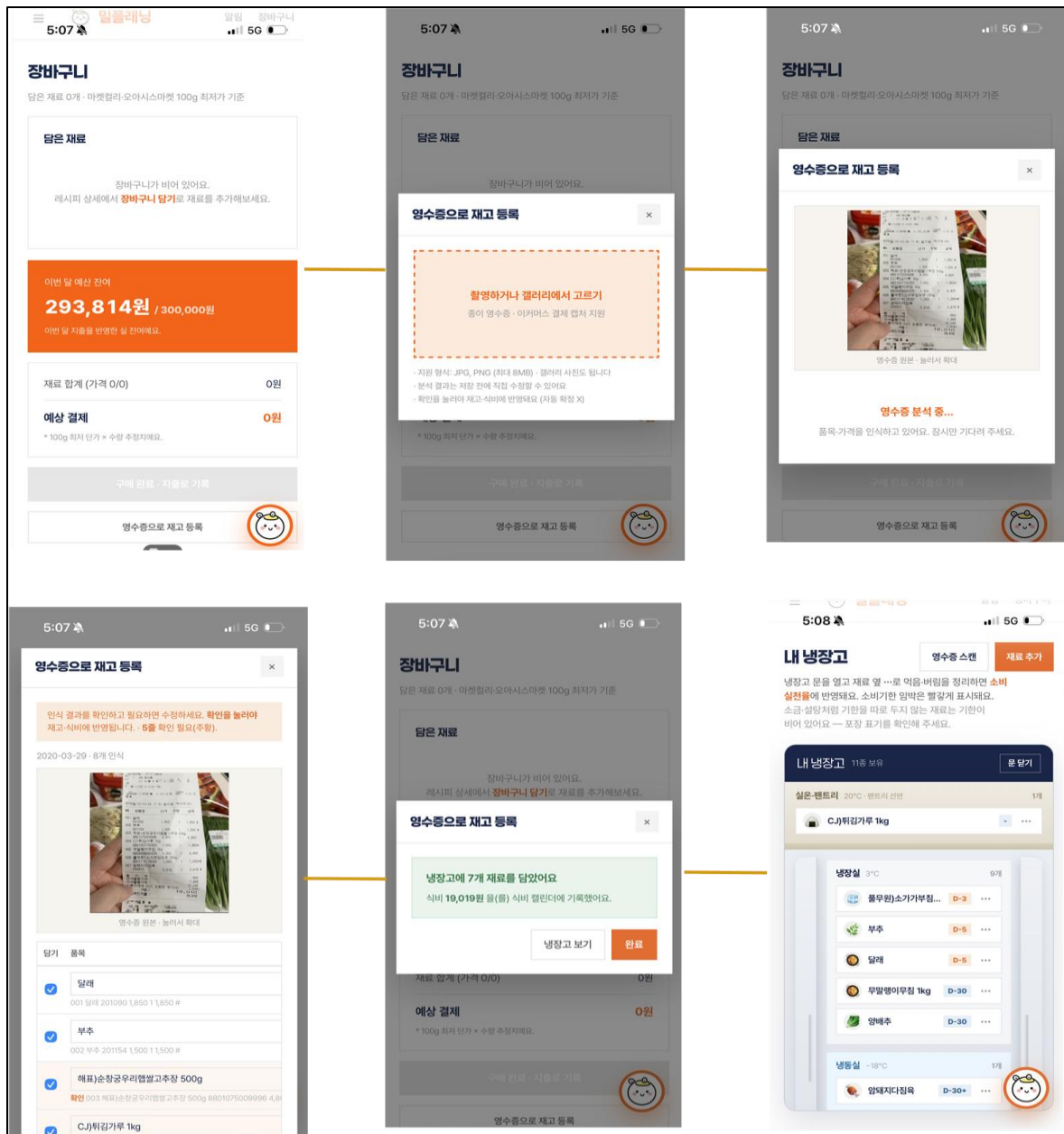
```
CREATE TABLE `recipe_ingredient` (  
  `id` BIGSERIAL NOT NULL,  
  `recipe_id` BIGINT NOT NULL,  
  `seq` INTEGER NULL,
```

2.5 디자인 설계

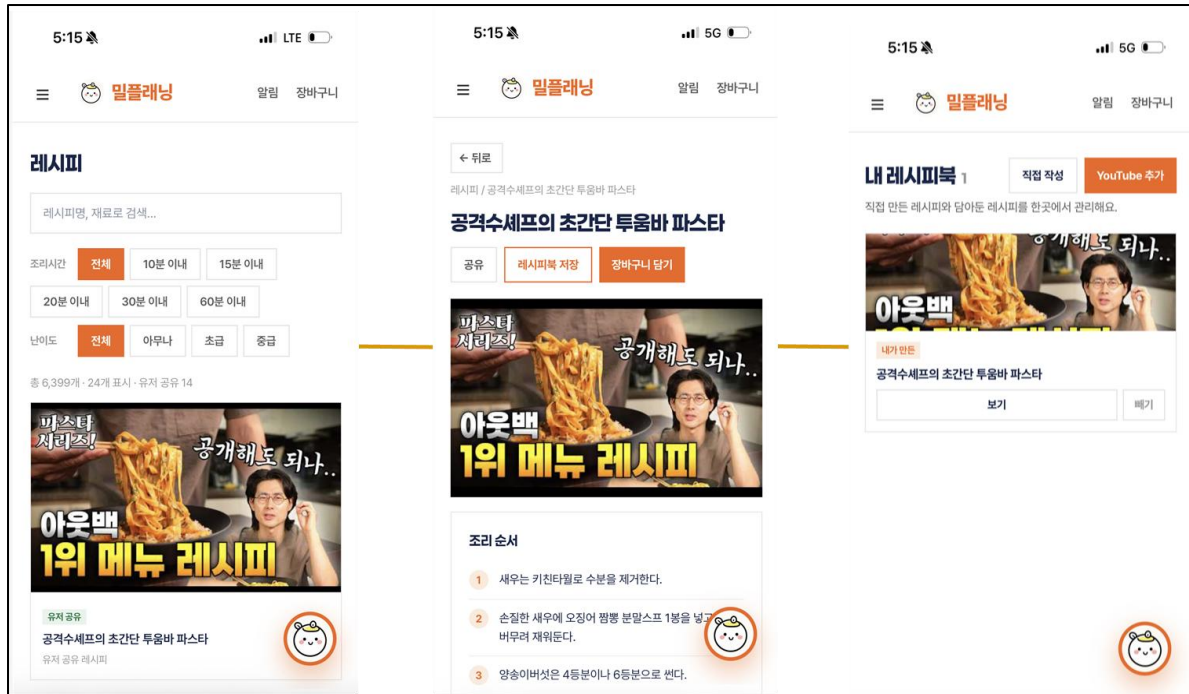
(로그인 및 회원가입)



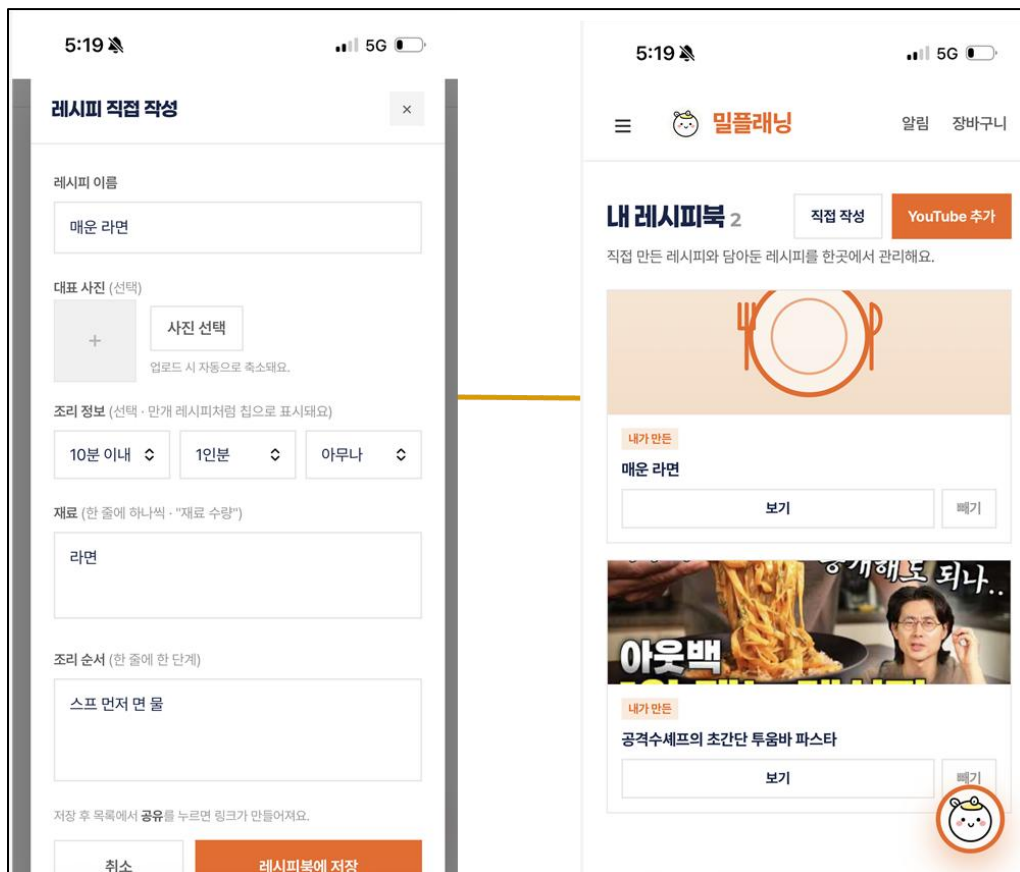
(영수증 등록)



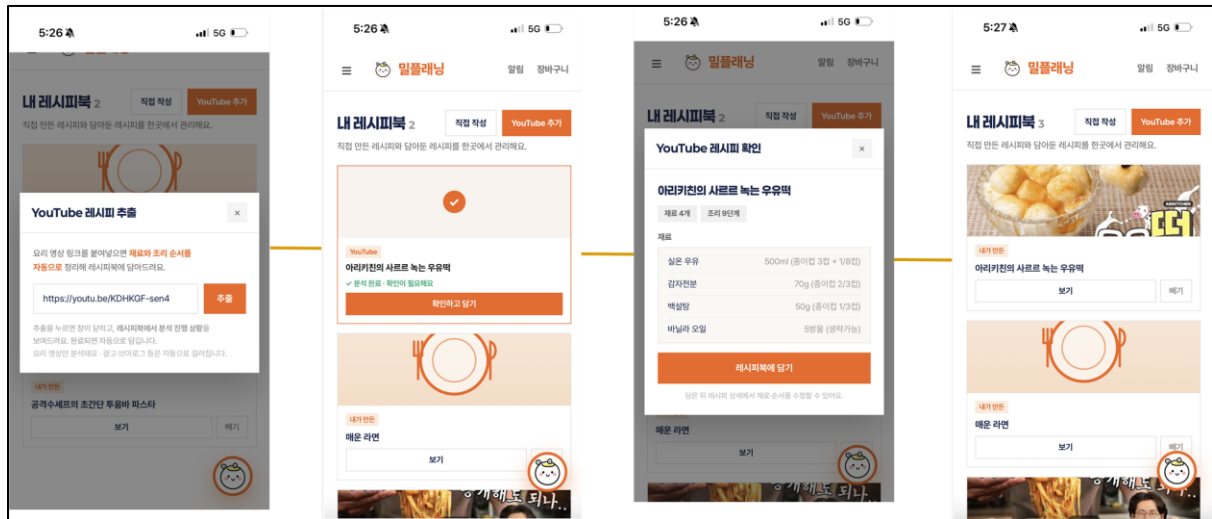
(레시피북 저장)



(레시피 직접 작성)



(유튜브 추출)



2.6 협업 방식

Slack 으로 팀 소통

final_project_bravo

플랜 업그레이드

대화 찾기...

스레드

허들

초안 및 전송됨

디렉터리

즐거찾기

중요한 사항은 여기에 끌어 놓으세요

채널

업무리포트

업무회의

일정관리

자유토론

주요링크

팀장회의

cost

monitoring

monitoring-alerts

mp-security

ops-ai-helpdesk

ops-daily-report

업무리포트

메시지

캔버스 추가

파일 및 링크

이상징후 시연영상

2026-08-04 20:46:57_내보내기.mp4

서비스 운영 개요

3:01

권우 오후 12:35

시시연

260805_기능시연 (90초).mp4

냉장고 속 재료로 예산 안에서 해결하는 가장 쉬운 방법

1:30

Notion 으로 주간 계획 공유

🔔 최종 일별 회의 목록

2026년 8월

캘린더에서 관리하기 < 오늘 >

일	월	화	수	목	금	토
9	10	11	12	13	14	15
클라우... 이전 계획 ...	서비스스 ...	데이터 파... 보안 ai 서버리스	모니터링 ... 세부적인 ...	모니터링 ... 비용관리 ...	세부 로직 ...	서비스 정...
16	17	18	19	20	21	22
최종 이전 ...	서비스 안...	서비스 이...	워크북 정리	ppt 제작	ppt 만들기 리하설	ppt 만들기 발표 연습 워크북 정리
23	24	25	26	27	28	29
ppt 만들기 발표 연습						
30	31	9월 1일	2	3	4	5

Jira 로 일정관리

도커 프로젝트 4 주제 선정 및 기획, 각종 설계도 작성 기한 2026년 7월 10일 △ ☒ BRAVO-19 = 👤 docker 애플리케이션 구축 기한 2026년 7월 15일 △ ☒ BRAVO-20 = 👤 발표준비 기한 2026년 7월 21일 △ ☒ BRAVO-21 = 👤 docker 애플리케이션 발표 기한 2026년 7월 22일 △ ☒ BRAVO-22 = 👤 + 만들기	김영수 4 기본 서비스 개발 완료 기한 2026년 7월 14일 △ ☒ BRAVO-47 = 🔴 UI/UX 리뷰 및 참여 / AI 실습 기한 2026년 7월 15일 △ ☒ BRAVO-32 = 🔴 UI/UX 리뷰 및 추가 서비스 개발 기한 2026년 7월 16일 △ ☒ BRAVO-33 = 🔴 전체 서비스 리뷰 및 각자 파트 마지막 점검(서비스 완성) / 발표 준비 시작 기한 2026년 7월 17일 △ ☒ BRAVO-34 = 🔴	이건우 4 AI 챗봇 구현 기한 2026년 7월 14일 △ ☒ BRAVO-23 = 🟢 AI 실습 기한 2026년 7월 15일 △ ☒ BRAVO-24 = 🟢 AI 챗봇 구현 기한 2026년 7월 16일 △ ☒ BRAVO-25 = 🟢 전체 서비스 리뷰 및 각자 파트 마지막 점검(서비스 완성) / 발표 준비 시작 기한 2026년 7월 17일 △ ☒ BRAVO-26 = 🟢	한정은 4 데이터 구조 공부 및 재료 이미지 데이터베이스 찾아보기/ 데이터 명세서 작성 기한 2026년 7월 14일 △ ☒ BRAVO-35 = 🟢 UI/UX 리뷰 및 DB 구조 파악/ 데이터 명세서 기한 2026년 7월 15일 △ ☒ BRAVO-36 = 🟢 구현된 서비스 기반으로 후후에 AWS 아전시 Fin쪽 어떻게 최적화 할 지 구성 기한 2026년 7월 16일 △ ☒ BRAVO-37 = 🟢 전체 서비스 리뷰 및 각자 파트 마지막 점검(서비스 완성) / 발표 준비 시작	임정현 4 보안 요소 문서화 배포(도커파일단 보안명세서) 애플리케이션단 보안구성 / 모니터링 대시보드구성 기한 2026년 7월 14일 ☒ BRAVO-39 ✓ = 🟢 서비스에 모니터링 QA 시작 기한 2026년 7월 15일 ☒ BRAVO-40 ✓ = 🟢 모니터링 QA 및 서비스 보안 구성 확인 기한 2026년 7월 16일 ☒ BRAVO-41 ✓ = 🟢 전체 서비스 리뷰 및 각자 파트 마지막 점검(서비스 완성) / 발표 준비 시작 기한 2026년 7월 17일
--	--	--	--	---

3. 핵심 기술

3.1 AI 재료 인식과 레시피 추천

우리 서비스의 핵심 AI 는 재료 텍스트를 구조화하는 재료 인식(NER)과, 사용자에게 맞는 순서로 레시피를 다시 정렬하는 개인화 랭킹이다. GPU 를 사용하지 않는다는 제약이 있어 두 모델 모두 CPU 에서 학습하고 추론하도록 설계했다.

Api flow



[그림 3.1-1] AI 재료 인식과 추천 처리 흐름

한식 재료 인식 (CRF)

- 레시피 재료는 돼지고기 600g, 다진 마늘 1 숟가락, 후추 약간처럼 자유 텍스트로 적혀 있음
- 여기서 재료명과 수량, 단위를 뽑아야 가격과 영양 정보를 연결할 수 있음
- CRF(Conditional Random Field)로 스펠을 추출하며, 문맥 의존이 강한 한국어 표기에 적합
- 학습과 추론이 모두 CPU 에서 가능해 GPU 없이 운영
- gold 데이터 50 건 기준 F1 0.9238 달성

피처 설계에서 한 가지를 일부러 제외했다. 어떤 낱말이 사전에 있는지 알려주는 피처를 주면 학습이 쉬워 보이지만, 학습 라벨 자체가 사전 매칭에서 나왔기 때문에 모델이 사전을 그대로 외워 버려 사전에 없는 재료를 잡지 못하게 된다. 문자와 문맥 패턴만 학습해야 미등재 재료로 일반화할 수 있고, 그것이 순수 사전 매칭 대신 CRF 를 두는 이유다.

```

mp@mealbong:~/food-budget-app/ml/ingredient-ner$ python score_gold.py
gold 레코드: 50건 (사람 검수)
약지도(사전)      vs gold - P 0.934 · R 0.928 · F1 0.931
CRF               vs gold - P 0.930 · R 0.918 · F1 0.924

--- CRF 오류 표본(최대 5건) ---
새우두부계란찜
  농침: ['새우', '두부', '계란찜']
  오람: ['새우두부계란찜']
김 10g, 마늘 8g, 생강 5g, 통깨 1.3g [소스소개]양념간장:물 30g
  농침: ['소스', '양념간장']
닭가슴살(40g), 노랑 파프리카(20g), 양파(15g), 오이(15g), 당근
  농침: ['잣가루', '간장']
  오람: ['겨잣가루', '올리고당', '저염 간장']
돼지고기(불고기용) 120g, 파인애플 35g, 양배추 15g, 양파 10g, 당
  오람: ['불고기용']
닭고기(가슴살, 30g), 쌀(50g), 양파(10g),
  오람: ['가슴살']
mp@mealbong:~/food-budget-app/ml/ingredient-ner$

```

서빙 번들 최적화

서버리스로 배포할 때 의존성 크기는 곧 첫 호출 지연이 된다. 추론에 실제로 쓰이지 않는 라이브러리를 걷어내자 다음과 같이 줄었다.

항목	제외 전	제외 후
번들 크기	224MB	19MB
import 시간	18,986ms	243ms

sklearn-crfsuite 는 추론에 한 줄도 쓰이지 않는데 scikit-learn 과 scipy, numpy 를 함께 끌고 들어온다. python-crfsuite 만 남기고 학습 모델을 .crfsuite 네이티브 포맷으로 저장해 래퍼 라이브러리 없이 로드하도록 바꾸었다.

train_crf.py (CRF 학습과 피쳐 설계)

```

from __future__ import annotations
import json
import pickle
import shutil
import re
from pathlib import Path
import sklearn_crfsuite

_DIR = Path(__file__).parent / "data"
_MODEL = _DIR / "model"
_HANGUL = re.compile(r"[ㄱ-ㅎ]")
_PUNCT = set("(),[]{}·●: : /.-~¼½¾")
def _ctype(c: str) -> str:
    if c.isdigit():
        return "d"
    if _HANGUL.match(c):
        return "h"
    if c.isspace():
        return "s"
    if c in _PUNCT:
        return "p"
    return "o"
def char_features(text: str, i: int) -> dict:
    c = text[i]
    f = {"bias": 1.0, "c": c, "ct": _ctype(c)}
    if i > 0:
        f["-1c"], f["-1ct"] = text[i - 1], _ctype(text[i - 1])
        f["-1:0"] = text[i - 1] + c
    else:
        f["BOS"] = True
    if i < len(text) - 1:
        f["+1c"], f["+1ct"] = text[i + 1], _ctype(text[i + 1])
        f["0:+1"] = c + text[i + 1]
    else:
        f["EOS"] = True
    if i > 1:
        f["-2c"] = text[i - 2]
    if i < len(text) - 2:
        f["+2c"] = text[i + 2]
    return f
def to_xy(rows: list[dict]):
    X, Y = [], []
    for r in rows:
        text, labels = r["text"], r["labels"]
        if not text:
            continue
        X.append([char_features(text, i) for i in range(len(text))])
        Y.append(labels)
    return X, Y
def bio_to_spans(labels: list[str]) -> set[tuple[int, int]]:
    spans, start = set(), None
    for i, l in enumerate(labels):
        if l == "B-ING":
            if start is not None:
                spans.add((start, i))
            start = i
        elif l == "I-ING":
            if start is None:
                start = i
        else:
            if start is not None:
                spans.add((start, i))
            start = None
    if start is not None:
        spans.add((start, len(labels)))

```

레시피 개인화 랭킹 (LambdaMART)

- 1 차로 규칙 기반 정렬을 수행. 냉장고 보유 재료 커버리지, 소비기한 임박도, 예산 적합도를 사용
- 2 차로 LightGBM LambdaMART 가 사용자별 선호를 반영해 순서를 다시 정함
- 학습 데이터가 부족한 초기 구간에서는 규칙 순서를 그대로 사용하고 모델은 개입하지 않음

학습 라벨은 어떤 화면에서 무엇을 골랐는지로 만든다. 그러려면 노출 기록과 행동 기록을 이어야 하는데, 조인 조건이 네 개다.

노출과 행동을 잇는 조인 조건

```
e.user_id = i.user_id
e.session_id = i.session_id
e.recipe_id = i.recipe_id
e.occurred_at BETWEEN i.shown_at AND i.shown_at + interval '30 minutes'
```

초기에는 프론트엔드가 session_id 를 보내지 않아 이 조인이 통째로 실패했다. 노출 기록은 서버가 uuid 를 발급해 행을 남기고 행동 기록은 값이 비어 있는 채로 지나갔기 때문에, 양쪽 테이블 모두 행이 쌓이고 있으니 정상으로 보였다. 그러나 교집합이 없어 학습할 것이 없었다. 이 경험 이후 적재가 실패하는 지점에 카운터와 경고 로그를 붙였다.

표준품목 사전

재료명과 실제 판매 상품을 잇는 사전이다. 이 매핑이 있어야 재료비를 계산할 수 있다.

측정 항목	값
레시피 재료 매칭률	93.0% (92,301 행 중 85,882 행)
표준품목 / 별칭	461 종 / 1,079 개

남은 공백을 사람이 손으로 채우기 어려워, 생성형 AI로 후보를 만들고 검수하는 파이프라인을 별도로 구성했다. 다만 생성형 AI는 서비스 실행 중에는 호출하지 않고 사전에 만들 때만 사용한다. 검수를 통과한 후보라도 실제 판매 상품 수를 코드가 직접 세어 기준에 못 미치면 차단한다. 검수자 세 명이 모두 통과시킨 항목 중 8건이 이 단계에서 걸렸는데, 확인해 보니 판매 상품이 없거나 한 건뿐이었다.

propose_item_master.py — 검수 통과 후보를 실제 상품 수로 거르는 코드 게이트

```
MIN_PRODUCTS = int(os.environ.get("PROPOSE_MIN_PRODUCTS", "3"))

def gate(cur, proposals: list[dict]) -> list[dict]:
    """`new` 제안에 대해 그 이름의 상품이 실제로 몇 건인지 센다."""
    for p in proposals:
        if p.get("action") != "new":
            p["gate"] = {"checked": False}
            continue
        name = p.get("name") or ""
        cur.execute("""
            select count(*) n, min(won_per_100g) lo, max(won_per_100g) hi
            from retail_unit_price
            where name like %s and won_per_100g is not null
        """, ("% " + name + "%",))
        n, lo, hi = cur.fetchone()
        # 상품이 적으면 신설하지 않는다 — 가격이 안 붙어 «어제까지 있던 값이 사라진다».
        ok = n >= MIN_PRODUCTS
        p["gate"] = {"checked": True, "products": n,
                    "price_range": [int(lo) if lo else None, int(hi) if hi else None],
                    "pass": ok}
        if not ok:
            p["gate_reason"] = f"상품 {n}건 < 최소 {MIN_PRODUCTS}건 — 신설 시 가격 공백"
    return proposals
```

propose_item_master.py (검수 통과 후보를 실제 상품 수로 다시 거르는 코드)

3.2 데이터 수집 파이프라인

가격 비교와 레시피 추천을 하려면 마켓의 현재 판매가와 레시피 원문이 계속 들어와야 한다. 수집은 정해진 시각에 도는 배치와, 사용자가 요청할 때만 도는 온디맨드 두 갈래로 구성했다.

```
mp@mealbong:~/food-budget-app$ kubectl get kafkatopics -n data
```

NAME	CLUSTER	PARTITIONS	REPLICATION FACTOR	READY
events.user.activity	kafka	3	3	True
events.user.activity.dlq	kafka	3	3	True
price.anomaly.detected	kafka	3	3	True
price.anomaly.detected.dlq	kafka	3	3	True
recipe.crawl.raw	kafka	3	3	True
recipe.crawl.raw.dlq	kafka	3	3	True
retail.crawl.raw	kafka	3	3	True
retail.crawl.raw.dlq	kafka	3	3	True
retail.deal.raw	kafka	2	3	True
retail.deal.raw.dlq	kafka	3	3	True

```
mp@mealbong:~/food-budget-app$
```

수집 대상과 주기

대상	수집 내용	주기
마켓컬리	신선식품과 가공식품 현재가	일 1~2 회 정기 폴링
오아시스마켓	판매가와 딜 정보	일 1~2 회, 딜은 15 시와 17 시
만개의레시피	레시피 원문과 재료 목록	주 1 회
유튜브	사용자가 요청한 영상의 레시피	온디맨드
영수증	사용자가 올린 영수증 이미지	온디맨드

배치 수집 구성

- Kubernetes CronJob 으로 정해진 시각에 실행하며 시간대는 KST 로 지정
- 수집한 원문은 Kafka 로 보내고, 소비자가 정규화해 데이터베이스에 적재
- KEDA 를 사용해 큐에 메시지가 없을 때는 소비자를 0 개까지 줄임
- 실패한 메시지는 DLQ 로 보내 유실되지 않도록 함

```
mp@mealbong:~/food-budget-app$ kubectl get cronjobs -n pipeline
```

NAME	SCHEDULE	TIMEZONE	SUSPEND	ACTIVE	LAST SCHEDULE	AGE
mp-chat-insights	0 6 * * *	Asia/Seoul	False	0	6h13m	25d
mp-crawl-s3-relay	*/30 * * * *	Asia/Seoul	False	0	13m	7d21h
mp-data-invariants	0 6 * * 1	Asia/Seoul	False	0	6h13m	23d
mp-deal-pruner	*/10 * * * *	Asia/Seoul	False	0	3m32s	25d
mp-pantry-expire-recompute	30 5 * * 0	Asia/Seoul	False	0	30h	23d
mp-poller-deal-closesale	5 17 * * *	Asia/Seoul	False	0	19h	25d
mp-poller-deal-timesale	5 15 * * *	Asia/Seoul	False	0	21h	25d
mp-poller-es-recipes	30 6 * * 0,3	Asia/Seoul	False	0	29h	25d
mp-poller-kurly	30 3 * * *	Asia/Seoul	False	0	8h	25d
mp-poller-oasis-dawn	10 4 * * *	Asia/Seoul	False	0	8h	25d
mp-poller-oasis-noon	10 13 * * *	Asia/Seoul	False	0	23h	25d

```
mp@mealbong:~/food-budget-app$
```

온디맨드 수집 구성

영상과 영수증은 사용자가 요청한 순간에만 처리한다. 처리에 수십 초가 걸려 요청을 받은 자리에서 끝낼 수 없으므로, 접수와 처리를 분리했다.

접수와 처리 분리 구조

```
클라이언트 — 접수 — 큐 — 워커 — 결과 캐시
— 202 작업번호 |
— 폴링으로 결과 조회 —————
```

- 접수는 요청을 큐에 넣고 즉시 작업 번호와 함께 202 를 반환하며 모델을 호출하지 않음
- 워커가 큐에서 꺼내 생성형 AI 를 호출하고 결과를 캐시에 저장
- 같은 영상을 다른 사용자가 요청하면 캐시에서 바로 응답하여 모델을 다시 부르지 않음
- 단일비행 락을 두어 동시에 들어온 같은 요청이 두 번 처리되지 않도록 함

여기에는 지켜야 하는 제약이 있다. 워커의 실행 시간 제한이 단일비행 락의 유효 시간보다 길면, 락은 이미 풀렸는데 워커는 아직 도는 구간이 생겨 같은 영상이 두 번 분석되고 유료 API 비용이 두 배가 된다. 반대로 워커의 제한이

내부 처리 제한보다 짧으면 실패를 기록하는 코드가 실행되기 전에 잘려서 작업이 대기 상태에 영영 남는다.

영수증 이미지 전달

영수증 사진은 보통 2MB 에서 5MB 사이인데, 로드밸런서에서 함수로 전달되는 요청 본문의 상한은 1MB 로 고정되어 있어 올릴 수 없다. 두 단계로 해결했다.

- 클라이언트에서 긴 변을 1600px 로 줄여 전송. 서버도 어차피 같은 크기로 줄이므로 인식 품질에 손실이 없고, 줄이기에 실패하면 원본을 그대로 보냄
- 그래도 한계를 넘으면 접수가 413 과 함께 S3 에 직접 올릴 수 있는 주소를 안내하고, 클라이언트는 그때만 2 단계 업로드로 전환

영상 접수 함수 (큐에 넣고 즉시 반환)

```

from __future__ import annotations
import os
import sys
import uuid
from pathlib import Path

# URL 정규화는 `ml/video-recipe` 의 `pipeline` 에 있다(컨테이너에선 `VIDEO_LIB_PATH`).
# `services/video/app` 이 아니다 — 거기엔 FastAPI 껍데기만 있다.
_HERE = Path(__file__).resolve()
for _p in (_HERE.parents[1], _HERE.parents[2] / "ml" / "video-recipe"):
    if _p.exists() and str(_p) not in sys.path:
        sys.path.insert(0, str(_p))

from common import alb, jobs          # noqa: E402
from common.runtime import logger, log_start    # noqa: E402
FUNCTION = "mp-ai-video-api"
log = logger(FUNCTION)
QUEUE_URL = os.environ.get("VIDEO_JOBS_QUEUE_URL", "")
def _normalize(url: str) -> str | None:
    """URL 정규화는 파이프라인의 것을 그대로 쓴다 — 캐시 키가 같리면 캐시가 통째로 무의미해진다.
    접수와 워커가 반드시 같은 함수를 써야 한다. 여기서 정규화한 키로 락을 잡고 워커가
    다른 키로 캐시를 쓰면, 락은 걸렸는데 캐시는 안 맞는 상태가 된다(중복 분석이 영구화).
    """
    from pipeline import normalize_url          # noqa: PLC0415
    return normalize_url(url)
def _backend_ready() -> bool:
    """현재 라우트와 같은 규약 — vertex 는 프로젝트 ID, api_key 는 키."""
    backend = os.environ.get("VIDEO_GENAI_BACKEND", "api_key")
    if backend == "vertex":
        return bool(os.environ.get("GCP_PROJECT_ID"))
    return bool(os.environ.get("VIDEO_GEMINI_API_KEY"))
def _submit(event: dict) -> dict:
    if not _backend_ready():
        return alb.error(503, "영상 분석이 아직 준비되지 않았어요.")
    url = (alb.body(event).get("url") or "").strip()
    if not url:
        return alb.error(400, "유튜브 영상 URL 이 아니에요.")
    norm = _normalize(url)
    if not norm:
        return alb.error(400, "유튜브 영상 URL 이 아니에요.")
    job_id = uuid.uuid4().hex
    cached = jobs.get_cached(norm)
    if cached is not None:          # 이미 분석된 영상 → 워커도 Gemini 도 안 부른다
        jobs.put_job(job_id, {"status": "DONE", "stage": "cached", "from_cache": True, **cached})
        log.info("■ %s 캐시 히트 · job=%s", FUNCTION, job_id)
        return alb.reply(202, {"job_id": job_id, "status": "DONE", "from_cache": True})
    if not jobs.acquire(norm):
        return alb.error(409, "같은 영상을 분석 중이에요. 잠시 후 다시 시도해 주세요.")
    jobs.put_job(job_id, {"status": "PENDING"})
    try:
        jobs.enqueue(QUEUE_URL, {"job_id": job_id, "url": url, "norm": norm})
    except Exception:              # noqa: BLE001
        # 큐 전송이 실패했는데 락과 PENDING 을 남기면 아무도 처리하지 않는 잡이 되고
        # 유저는 폴링만 계속한다. 되돌리고 정직하게 실패로 답한다.
        jobs.release(norm)
        jobs.put_job(job_id, {"status": "FAILED", "error": "queue_unavailable"})
        log.exception("%s 큐 전송 실패 · job=%s", FUNCTION, job_id)
        return alb.error(503, "잠시 후 다시 시도해 주세요.")
    log.info("■ %s 접수 · job=%s", FUNCTION, job_id)
    return alb.reply(202, {"job_id": job_id, "status": "PENDING", "from_cache": False})
def _poll(job_id: str) -> dict:
    payload = jobs.get_job(job_id)
    if payload is None:
        return alb.error(404, "결과를 찾을 수 없어요(만료되었거나 잘못된 요청).")
    return alb.reply(200, {"job_id": job_id, **payload})
def handler(event, context):
    """ALB 가 부른다. 경로 2 개를 메서드로 가른다."""
    method, path = alb.method(event), alb.path(event)
    log_start(log, FUNCTION, {"method": method, "path": path}, context)
    if method == "POST":
        return _submit(event)

```

영상 처리 워커 (큐를 소비하고 생성형 AI 호출)

```

from __future__ import annotations
import asyncio
import os
import sys
from pathlib import Path

# 추출 라이브러리는 `services/video` 가 아니라 `ml/video-recipe` 에 있다.
# 컨테이너에서는 `/opt/video-recipe`(`VIDEO_LIB_PATH`)로 붙는다 — 그래서 `from pipeline import ...`
# 가 성립한다. 번들에서는 build.sh 가 평평하게 넣으므로 번들 루트가 곧 그 자리다.
_HERE = Path(__file__).resolve()
for _p in (_HERE.parents[1], _HERE.parents[2] / "ml" / "video-recipe"):
    if _p.exists() and str(_p) not in sys.path:
        sys.path.insert(0, str(_p))
from common import jobs # noqa: E402
from common.assets import gcp_credentials # noqa: E402
from common.runtime import logger, log_start # noqa: E402
from common.secrets import inject # noqa: E402
FUNCTION = "mp-ai-video-worker"
log = logger(FUNCTION)

# 이 함수는 Gemini 를 실제로 부른다(영상 → 레시피 추출). 키가 없으면 접수는 202 로
# 통과하는데 처리만 조용히 실패한다 — `ai_ocr_worker` 와 같은 이유로 여기 있어야 한다.
inject()
# EKS 는 API 키가 아니라 Vertex 다 — `VIDEO_GENAI_BACKEND=vertex` +
# `GOOGLE_APPLICATION_CREDENTIALS=/etc/gcp/gcp-sa.json`(볼륨 마운트) 실측 2026-08-18.
# ⇒ Lambda 도 같은 경로를 타게 파일을 만든다. 안 그러면 «EKS 는 vertex, Lambda 는
# api_key» 로 갈라서 같은 입력에 다른 결과가 나올 수 있다.
gcp_credentials()
EXTRACT_TIMEOUT_S = float(os.environ.get("VIDEO_TIMEOUT_S", "120"))

class _CacheAdapter:
    """파이프라인이 기대하는 `get/set` 모양. 동기 redis 라 원본의 루프 브리지가 필요 없다."""
    def __init__(self, norm: str) -> None:
        self._norm = norm
    def get(self, key):
        raw = jobs.get_cached(key)
        if not raw:
            return None
        from models import RecipeExtraction # noqa: PLC0415
        return RecipeExtraction(**raw)
    def set(self, key, recipe):
        jobs.set_cached(key, recipe.model_dump())

def _done_payload(result) -> dict:
    """현재 `_run_job` 의 DONE 본문과 키까지 같게 만든다 — 프론트가 이미 이 모양을 읽는다."""
    r = result.recipe
    return {
        "status": "DONE", "stage": result.stage, "from_cache": result.from_cache,
        "title": r.title, "is_recipe": r.is_recipe,
        "servings": r.servings, "servings_known": bool(r.servings),
        "source_url": r.source_url, "source_creator": r.source_creator,
        "video_seconds": r.video_seconds,
        "ingredients": [i.model_dump() for i in r.ingredients],
        "steps": [s.model_dump() for s in r.steps],
        "soft_flags": list(getattr(result, "soft_flags", []) or []),
        # 재료비는 여기서 빼 뒀다. 원본은 `_estimate_cost` 로 PG 를 치는데, 그건 VPC-물.
        # 커넥션이 붙는 별개 관심사다. 원본도 "실패해도 추출 결과는 그대로 준다"고 적어 둔
        # 부가 정보라, 접수·워커 분할이 검증되기 전에 같이 옮기면 실패 원인이 섞인다.
        # 이관 완료 조건이 아니다 — 별건으로 붙인다(아래 modules.txt 주석과 짝).
        "cost": None,
    }

def _process(body: dict) -> dict:
    job_id, url, norm = body.get("job_id"), body.get("url"), body.get("norm")
    if not job_id or not url:
        # 우리가 만든 메시지가 아니다 — 재시도해도 같으므로 조용히 버린다(DLQ 로 보낼 이유 없음).
        log.warning("%s 잘못된 메시지: %s", FUNCTION, list(body))

```

4. 시스템 구성

4.0 시스템 구성 개요, 3 단계 전환

본 서비스의 인프라는 한 번에 만들어지지 않았다. Docker 기반 VM 구성에서 시작해 온프레미스 Kubernetes 로 옮기고, 다시 AWS EKS 로 이전하는 3 단계를 거쳤다. 각 단계는 앞 단계에서 실제로 겪은 한계를 근거로 넘어갔으며, 이전 단계를 폐기하지 않고 역할을 바꿔 재활용했다.

- (1) Docker (VM) 단계, Proxmox 위 VM 4 대에 Docker Compose 로 앱·데이터·관측·CI 를 배치했다. 구성은 단순했으나 전 컴포넌트가 단일 인스턴스였고 백업이 존재하지 않았다.
- (2) 온프레미스 Kubernetes 단계, kubeadm 5 노드 클러스터를 구축하고 데이터 티어를 오퍼레이터로 인클러스터화했다. HA·GitOps·서비스 메시·관측을 확보했으나 물리 호스트 2 대가 랙·전원·회선을 공유하는 구조적 한계가 남았다.
- (3) AWS EKS 단계, 동일한 Kubernetes 버전(1.34)과 동일한 CNI(Cilium)·메시(Istio)를 유지한 채 AZ 2 개로 이전했다. 온프레미스는 DR 대기 사이트 겸 크롤 상시 프로덕션으로 재활용한다.

단계별 주요 형상 변화는 다음과 같다.

계층	(1) Docker (VM)	(2) 온프레미스 K8s	(3) AWS EKS
컴퓨트	Proxmox VM 4 대 16 vCPU / 26 GB	kubeadm 노드 5 대 물리 2 대 위	m7g.xlarge × 2~4 Graviton / arm64
오케스트레이션	Docker Compose	Kubernetes 1.34.10	Amazon EKS 1.34
CNI	Docker bridge	Cilium 1.19.6	Cilium 1.19.6 (ENI)

		(VXLAN)	
외부 진입	호스트 nginx	MetalLB L2 Istio GW	ALB + WAF Istio GW
데이터 티어	Compose 단일 4 종	CNPG · ECK · Strimzi · Redis	CNPG · ECK · ElastiCache
스토리지	VM 로컬 디스크	OpenEBS LVM LocalPV	EBS gp3 CSI
CI / CD	GH Actions / 수동	Jenkins ArgoCD	GitLab CI ArgoCD
배포 전략	컨테이너 재기동	롤링 + 카나리	Blue-Green
백업	없음	barman S3	barman S3 + 크로스사이트
가용성	단일 VM = SPOF	노드 5 대	AZ 2 개

설계 원칙, 매니지드는 운영 부담이 실제로 줄 때만 채택한다

AWS 로 이전하면서도 RDS · OpenSearch Service · MSK 는 모두 채택하지 않았다.

매니지드에서 사는 것은 백업과 페일오버인데, 그건 CloudNativePG 와 ECK 가 이미 제공한다.

반대로 Redis 는 ElastiCache 로 옮겼다, 판단 측은 비용이 아니라 Sentinel 운영 부담이었다.

즉 '전부 매니지드' 도 '전부 자체운영' 도 아니고, 항목마다 얻는 것과 잃는 것을 따로 계산했다.

4.1 AWS 구성도

AWS 클라우드 구성도 설명

AWS 의 격리된 가상 네트워크인 VPC 를 서울 리전(ap-northeast-2)에 2 개 생성해 서비스 VPC(mp-vpc-service · 10.10.0.0/16)와 CI VPC(mp-vpc-ci · 10.11.0.0/16)로 분리

두 VPC 사이에 피어링을 두지 않아, 빌드 서버가 프로덕션 대역을
구조적으로 볼 수 없게 구성

서비스 VPC 는 가용 영역 2 개(ap-northeast-2a · 2b)에 공개 · 노드 · 데이터
3 티어 서브넷을 생성

노드 서브넷(10.10.64.0/20 · 10.10.80.0/20)에 EKS 관리형 노드그룹을
배치하고, Cilium ENI 모드로 파드가 노드 서브넷의 VPC IP 를 직접
받도록 구성

데이터 서브넷(10.10.32.0/24 · 10.10.33.0/24)에 ElastiCache 를 배치하고
라우팅 테이블에 기본 경로를 넣지 않아 인터넷 경로 자체를 제거

공개 서브넷에 ALB 를 배치하고 ACM 인증서로 TLS 를 종단, AWS WAF 를
부착해 웹 공격을 차단

외부 통신을 위해 NAT Gateway 1 대를 공개 서브넷(AZ-a)에 두고 두 AZ 가
공유

S3 Gateway Endpoint 와 Interface Endpoint 3 종(SQS · Secrets Manager ·
STS)을 구성해 핵심 호출이 NAT 를 거치지 않도록 구성

CI VPC 의 공개 서브넷에 EC2 를 생성해 GitLab · GitLab Runner ·
SonarQube 를 구성하고 CI 를 관리

도메인은 Cloudflare 에 등록하고 ALB 를 연결, Amazon Route 53 은
채택하지 않음

GuardDuty 와 CloudTrail 을 활성화하고 EventBridge Lambda Slack 으로
보안 알림을 전달



[그림 4.1-1] AWS 전체 구성도

VPC · 서브넷 구성

티어	Name	CIDR	입주자	라우팅
공개 (AZ-a)	mp-subnet-public-ap-northeast-2a	10.10.0.0/24	ALB ENI · NAT GW · 대시보드 EC2	mp-rt-public IGW
공개 (AZ-b)	mp-subnet-public-ap-northeast-2b	10.10.1.0/24	ALB ENI	mp-rt-public IGW
노드 (AZ-a)	mp-subnet-node-ap-northeast-2a	10.10.64.0/20	EKS 노드 + 파드 + VPC Endpoint	mp-rt-node NAT
노드 (AZ-b)	mp-subnet-node-ap-northeast-2b	10.10.80.0/20	EKS 노드 + 파드 + VPC Endpoint	mp-rt-node NAT(AZ-a)
데이터 (AZ-a)	mp-subnet-data-ap-northeast-2a	10.10.32.0/24	ElastiCache (mp-cache)	mp-rt-data, 기본 경로 없음
데이터 (AZ-b)	mp-subnet-	10.10.33.0/24	ElastiCache	mp-rt-data,

	data-ap-northeast-2b		복제본	기본 경로 없음
CI (AZ-a)	mp-subnet-ci-public	10.11.0.0/24	CI 서버 EC2 (GitLab · Runner · SonarQube)	mp-rt-ci-public mp-igw-ci

설계 근거 (1), 데이터 서브넷에 기본 경로를 넣지 않은 이유

라우팅 테이블 mp-rt-data 에는 10.10.0.0/16 local 항목만 있고 0.0.0.0/0 이 없다.

이것이 이 라우팅 테이블의 목적이다, 보안 그룹은 사람이 실수로 열 수 있지만, 경로가 없으면 나갈 수 없다.

ElastiCache 는 VPC 내부에서만 접근하면 되므로 인터넷 경로가 애초에 필요 없다.

설계 근거 (2), VPC Endpoint 3 종의 근거는 비용이 아니라 가용성이다

NAT Gateway 를 1 대만 두었으므로(비용) AZ-a 가 끊기면 두 AZ 모두 아웃바운드를 잃는다.

그런데 SQS · Secrets Manager · STS 세 호출은 그 상황에서도 살아 있어야 한다

각각 크롤 컨슈머 · 비밀 주입 · IRSA 토큰 교환의 경로이기 때문이다.

그래서 이 세 개만 Interface Endpoint(월 \$56.94)로 NAT 를 우회시켰다. 나머지는 NAT 를 쓴다.

함정 : private DNS 를 켜 Interface Endpoint 는 NAT 로 폴백하지 않는다. 보안 그룹에서 막히면

요청이 응답 없이 드롭되고, 증상이 자격증명 오류처럼 보인다(실측 · MR !88).

설계 근거 (3), Route 53 미채택

도메인이 이미 Cloudflare 에 있고, DR 전환을 Cloudflare 레코드 하나로 수행한다.

권한을 두 곳으로 쪼개면 장애 국면에서 '어디를 고쳐야 하나' 가 하나 더 늘어난다.

AWS 측 진입 엔드포인트는 ALB DNS 이름이며, Cloudflare 가 CNAME 으로 가리킨다.

4.2 AWS EKS 클러스터 구성도

AWS EKS cluster 구성도 설명

고가용성을 위해 파드 층(HPA · KEDA)과 노드 층(Karpenter) 두 겹의
오토스케일을 구성

MSA 환경을 구성하기 위해 EKS 에 Istio 서비스 메시를 sidecar 모드로
구축하고 mTLS STRICT 적용

외부 ALB 에서 클러스터 내부로 들어오는 트래픽은 Gateway API 의
Gateway(mp-gw-public)로 수신

Gateway 는 진입 전용 네임스페이스 mp-ingress 에 두어 앱 네트워크
정책과 진입 정책이 서로를 넘지 않게 분리

Gateway 에서 HTTPRoute 를 통해 목적에 맞는 서비스의 파드로 라우팅
app 네임스페이스의 모든 파드는 Istio proxy 컨테이너를 함께 생성해
네트워크 트래픽을 관리

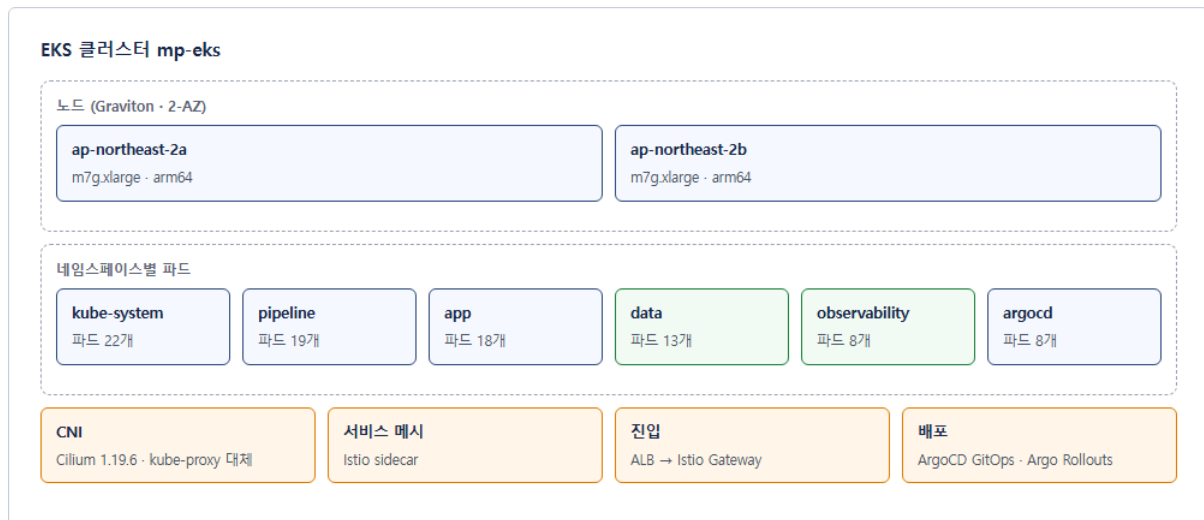
역할별로 네임스페이스를 분리, app(애플리케이션 12 종) · data(데이터
티어) · pipeline(수집) · observability(관측) · argocd(배포) · mp-
ingress(진입) · cost(FinOps)

데이터 티어는 오퍼레이터로 관리, CloudNativePG(PostgreSQL) ·
ECK(Elasticsearch)

Redis 는 클러스터 밖 Amazon ElastiCache for Valkey 를 데이터 서브넷에
배치

관측을 위해 Prometheus · Loki · Tempo 를 인클러스터로 구성하고 Alloy
를 DaemonSet 으로 수집

배포를 위해 ArgoCD 를 인클러스터에 두고, CI 는 클러스터 밖(GitLab
EC2)에 분리



[그림 4.2-1] AWS EKS 클러스터 구성도

네임스페이스 구성

네임스페이스	담는 것	메시	PodSecurity
app	애플리케이션 12 종 (account · recipe · price · chat · pantry · recipebook · mealplan · notify · ocr · video · ranking-serving · frontend)	ON	restricted
data	CloudNativePG(pg-1 · pg-2 · pgBouncer) · ECK(Elasticsearch 3) · PGSync	OFF	baseline
pipeline	SQS 컨슈머 3 종(KEDA) · CronJob · ranking-retrain	OFF	baseline
mp-ingress	Gateway(mp-gw-public) · cloudflared(DR 전용)	OFF	baseline

observability	Prometheus · Alertmanager · Grafana · Loki · Tempo	OFF	baseline
argocd	ArgoCD (Application 44 개)	OFF	baseline
cost	kubecost	OFF	baseline
kube-system	Cilium · CoreDNS · EBS CSI · Karpenter · KEDA · Alloy	OFF	privileged

data · pipeline 을 baseline 으로 둔 이유

오퍼레이터(CloudNativePG · ECK)가 생성하는 파드를 restricted 로 막으면 원인을 찾기 어려운 실패가 난다.

실제로 ECK 의 vm.max_map_count 초기화 initContainer 가 여기 걸려, 노드 sysctl 선반영으로 우회했다.

다만 baseline 도 특권 initContainer 는 거부하므로, 완전히 열어둔 것은 아니다.

4.3 Kubernetes 사용 기술

본 절은 EKS 위에 올린 플랫폼 기술을 계층 순서로 기술한다. 각 항목은 구성 구축 절차 검증 선택 근거 순으로 서술하며, 온프레미스에서 이미 검증한 뒤 AWS 로 옮긴 것과 AWS 에서 새로 도입한 것을 구분해 표시한다.

항목	기술	온프레미 검증	AWS 신규
4.3.1 클러스터	EKS 1.34 · Graviton 노드그룹	kubeadm 1.34.10	○
4.3.2 CNI	Cilium 1.19.6	○ (VXLAN)	ENI 모드로 변경

4.3.3 서비스 메시	Istio 1.30.3 sidecar · mTLS STRICT	○	
4.3.4 진입	Gateway API + ALB + ACM + WAF	○ (MetalLB)	ALB·WAF 신규
4.3.5 리소스·우선순위	ResourceQuota · LimitRange · PriorityClass	○ (3 티어)	5 티어로 확장
4.3.6 오토스케일	HPA · KEDA · Karpenter	○ (HPA·KEDA)	Karpenter 신규
4.3.7 스토리지	EBS CSI · StorageClass · PV 회수정책	○ (OpenEBS)	EBS 로 교체
4.3.8 데이터 티어	CloudNativePG · ECK · ElastiCache	○	ElastiCache 신규
4.3.9 배포	ArgoCD · Argo Rollouts Blue-Green	○ (카나리)	Blue-Green 신규

4.3.1 클러스터 구성

EKS 클러스터는 Terraform 으로 생성하며, 노드는 관리형 노드그룹을 고정 용량으로 두고 그 위에 Karpenter 를 버스트 전용으로 엮는 2 층 구조로 구성했다.

항목	구성 (실측)
Cluster name	mp-eks
Kubernetes version	1.34 (platform version eks.31)
Region / AZ	ap-northeast-2 / ap-northeast-2a · 2b
노드그룹	mp-mng-general (관리형 · ON_DEMAND)

인스턴스	m7g.xlarge (Graviton · arm64 · 4 vCPU / 16 GiB)
스케일링	min 2 · desired 2 · max 4 (Terraform 은 desired 를 ignore)
AMI / 루트 볼륨	AL2023_ARM_64_STANDARD / 60 GiB gp3 (암호화)
애드온	aws-ebs-csi-driver · coredns (2 종)
엔드포인트 접근	public + private
컨트롤플레인 로깅	audit CloudWatch Logs (/aws/eks/mp-eks/cluster)

구축 절차, 클러스터 생성

```
# 클러스터 생성 (Terraform)
cd infra/terraform/aws-platform
terraform init -backend-config=backend.conf
terraform plan -out=tfplan
terraform apply tfplan
# kubeconfig 갱신
aws eks update-kubeconfig --region ap-northeast-2 --name mp-eks --profile <프로필>
kubectl config current-context # ...:cluster/mp-eks 확인
```

eks_nodegroup.tf, 관리형 노드그룹

```
resource "aws_eks_node_group" "general" {
  cluster_name = aws_eks_cluster.mp.name
  node_group_name = "mp-mng-general"
  node_role_arn = aws_iam_role.node.arn
  subnet_ids = [for s in aws_subnet.node : s.id]
  ami_type = "AL2023_ARM_64_STANDARD"
  instance_types = ["m7g.xlarge"]
  capacity_type = "ON_DEMAND"
  disk_size = 60
  scaling_config {
    min_size = 2
    desired_size = 2
    max_size = 4
  }
}
# 정상 상태를 코드가 되돌리지 않게 desired 는 무시한다
lifecycle { ignore_changes = [scaling_config[0].desired_size] }
```

aws 서울 (ap-northeast-2)							
EKS > 클러스터 > mp-eks > 컴퓨팅							
노드 그룹 ACTIVE							
이름	상태	인스턴스 유형	용량 유형	최소	원하는 수	최대	AMI 유형
mp-mng-general	ACTIVE	m7g.xlarge	ON_DEMAND	2	2	4	AL2023_ARM_64_STANDARD

검증

```
$ kubectl get nodes -o wide
NAME STATUS ROLES AGE VERSION INSTANCE-TYPE
ip-10-10-64-xxx.ap-... Ready <none> ... v1.34.x m7g.xlarge
ip-10-10-80-xxx.ap-... Ready <none> ... v1.34.x m7g.xlarge
$ kubectl get nodes -L topology.kubernetes.io/zone
# AZ-a · AZ-b 에 각각 배치되었는지 확인
```

선택 근거, Graviton(arm64) 과 Spot 미채택

(1) Graviton, 동일 성능 대비 단가가 낮다. 우리 스택(Python · Java 없음 · Go 오퍼레이터)은 arm64 지원이 완전하다.

대가는 이미지를 arm64 로 다시 구워야 한다는 것인데, 이는 CI 를 GitLab 으로 새로 만들면서 함께 해결했다.

(2) Spot 미채택, 흔히 '배포할 때만 Spot' 을 권하지만 우리 구조에서는 성립하지 않는다.

Blue-Green 은 promote 후 green 이 프로덕션 그 자체가 되어 파드가 이사하지 않기 때문이다.

상시 Spot 도 실익이 작다, 실측 결과 CPU 는 요청 대비 실사용이 9.4 배 여유인 반면 메모리는 1.1 배였다.

즉 우리가 사는 것은 CPU 가 아니라 메모리이고, Spot 할인은 메모리 비용의 10% 수준에만 걸린다.

(3) AZ 3 개 미채택, AZ 3 개는 정족수가 필요한 컴포넌트가 있을 때 의미가 있는데, 우리 데이터 티어의 정족수는 클러스터 안에서 이미 만족된다.

4.3.2 Cilium, CNI 및 kube-proxy 대체

CNI 로 Cilium 을 사용한다. eBPF 기반으로 kube-proxy 를 대체하며, 온프레미스와 AWS 양쪽에 같은 버전(1.19.6)을 설치해 NetworkPolicy 자산을 그대로 이식할 수 있게 했다. IPAM 모드만 사이트에 따라 다르다.

항목	온프레미스	AWS EKS
----	-------	---------

버전	Cilium 1.19.6	Cilium 1.19.6
kube-proxy	미설치 (kubeProxyReplacement)	미설치 · vpc-cni 애드온도 미설치
라우팅 / IPAM	VXLAN 오버레이	ENI 모드, 파드가 VPC IP 를 직접 수령
노드 간 암호화	WireGuard	VPC 내부 · Security Group 으로 통제
파드 CIDR	클러스터 내부 대역	노드 서브넷에서 직접 할당

구축 절차, Cilium 설치 (ENI 모드)

```
# Helm 으로 설치 (Ansible roles/eks_cilium 이 수행)
helm repo add cilium https://helm.cilium.io/
helm upgrade --install cilium cilium/cilium \
--version 1.19.6 \
--namespace kube-system \
--set kubeProxyReplacement=true \
--set k8sServiceHost=<EKS API 엔드포인트> \
--set k8sServicePort=443 \
--set eni.enabled=true \
--set ipam.mode=eni \
--set routingMode=native \
--set egressMasqueradeInterfaces=en+ \
--set cni.exclusive=false # Istio CNI 체이닝 전제
```

검증

```
$ kubectl -n kube-system get pods -l k8s-app=cilium
$ kubectl -n kube-system exec ds/cilium -- cilium-dbg status --brief
$ kubectl -n kube-system exec ds/cilium -- cilium-dbg status | grep -i masquerad
# 파드에서 외부 통신 확인
$ kubectl -n app run t --rm -it --image=curlimages/curl --restart=Never -- \
curl -s -o /dev/null -w '%{http_code}\n' https://api.ecr.ap-northeast-2.amazonaws.com
```

```

mp@mealbong:~$ kubectl -n kube-system exec ds/cilium -- cilium-dbg status
KVStore: Disabled
Kubernetes: Ok 1.34 (v1.34.9-eks-bca9cf6) [linux/arm64]
Kubernetes APIs: ["cilium/v2::CiliumCIDRGroup", "cilium/v2::CiliumClusterwideNetworkPolicy", "cilium/v2::CiliumEndpoint", "cilium/v2::CiliumNetworkPolicy"]
KubeProxyReplacement: True [ens5 10.10.x.x fe80::3a:9bff:fe18:2681 (Direct Routing), ens6 10.10.x.x fe80::25:b8ff:fe69:4b39, ens7 10.10.x.x fe80::25:b8ff:fe69:4b39]
Host firewall: Disabled
SRv6: Disabled
CNI Chaining: none
CNI Config file: successfully wrote CNI configuration file to /host/etc/cni/net.d/05-cilium.conflist
Cilium: Ok 1.19.6 (v1.19.6-9a898243)
NodeMonitor: Listening for events on 4 CPUs with 64x4096 of shared memory
Cilium health daemon: Ok
IPAM: IPv4: 47/55 allocated,
IPv4 BIG TCP: Disabled
IPv6 BIG TCP: Disabled
BandwidthManager: Disabled
Routing: Network: Native Host: Legacy
Attach Mode: TCX
Device Mode: veth
Masquerading: IPTables [IPv4: Enabled, IPv6: Disabled]
Controller Status: 245/245 healthy
mp@mealbong:~$

```

[그림 4.3.2-1] cilium-dbg status 출력

실측 사고, egressMasqueradeInterfaces 인터페이스명 (#45~#49)

증상 : 파드가 VPC 밖(ECR · S3 · 외부 API)으로 나가지 못했다.

원인 : 온프레미스 값 eth0 를 그대로 옮겼는데, AWS Nitro 인스턴스의 인터페이스명은 ens5 다.

진단이 어려웠던 이유 : 1 차 ENI 에 붙은 파드만 우연히 정상 동작해 '일부만 안 된다' 로 보였다.

노드 단위나 서비스 단위 문제로 오인하기 쉬운 형태였다.

해결 : 값을 en+ 로 지정했다. 빈 값은 허용되지 않고 콤마로 여러 개를 나열할 수도 없어, 접두사 와일드카드가 유일한 해답이었다.

선택 근거, 왜 vpc-cni 가 아니라 Cilium 인가

(1) 이식성, 온프레미스에서 작성한 NetworkPolicy 26 종과 CiliumNetworkPolicy 29 종을 그대로 옮길 수 있다.

CNI 를 바꿨다면 정책 자산을 전부 다시 쓰고 다시 검증해야 했다.

(2) FQDN 기반 egress, 표준 NetworkPolicy 는 IP/포트만 다루는데, 외부 API(Gemini · S3)를 도메인 이름으로 제한하려면 CiliumNetworkPolicy 의 toFQDNs 가 필요하다.

(3) kube-proxy 제거, iptables 규칙이 서비스 수에 비례해 늘어나는 구조를 eBPF 로

대체했다.

(4) 관측, Hubble 로 파드 간 흐름을 그대로 볼 수 있어, 정책을 잠글 때 무엇이 끊기는지 먼저 확인할 수 있었다.

4.3.3 Istio, 서비스 메시

MSA 환경에서 서비스 간 통신을 통제하기 위해 Istio 를 sidecar 모드로 구축했다. app 네임스페이스의 워크로드에만 주입하며, data · pipeline 네임스페이스는 제외한다.

버전 Istio 1.30.3, base + istiod + istio-cni(체이닝) 3 개 차트로 구성
app 네임스페이스에 istio-injection=enabled 라벨을 부여해 사이드카 자동 주입

PeerAuthentication 을 STRICT 로 설정해 메시 내부 통신을 전부 mTLS 로 암호화

Gateway API 의 구현체로 Istio 를 사용, 별도의 Ingress Controller 를 두지 않는다

ambient 모드는 미채택, sidecar 가 이미 온프레미스에서 검증됐고 우리 규모에서 얻는 이득이 작다

구축 절차, Istio 설치

```
helm upgrade --install istio-base istio/base -n istio-system --create-namespace --version 1.30.3
helm upgrade --install istiod istio/istiod -n istio-system --version 1.30.3 \
--set pilot.cni.enabled=true # 필수, 누락하면 사이드카가 트래픽을 못 잡는다
helm upgrade --install istio-cni istio/cni -n istio-system --version 1.30.3
kubectl label namespace app istio-injection=enabled --overwrite
```

PeerAuthentication, mTLS STRICT

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: mp-default-strict
  namespace: app
spec:
  mTLS:
```

```
mode: STRICT
```

검증

```
# 사이드카 주입 확인 — spec.containers 가 아니라 initContainers 를 본다
$ kubectl -n app get pod <파드> -o jsonpath='{.spec.initContainers[*].name}'
istio-validation istio-proxy
# mTLS 적용 확인
$ istioctl x describe pod <파드>.app | grep -i mtl
```

```
mp@mealbong:~$ kubectl -n app get pods
mp-account-68cdcfc688-b6sq2      account      Running
mp-account-68cdcfc688-ssdsp      account      Running
mp-chat-7f747475c5-9stmj         chat         Running
mp-frontend-f747497f4-fjdwf      frontend     Running
mp-frontend-f747497f4-rv642      frontend     Running
mp-mealplan-6d8c7446bb-sj2d1     mealplan     Running
mp-notify-5744ccd567-c21s7       notify       Running
mp-ocr-config-canary-29781855-49jmr ocr-config-canary Succeeded
mp-ocr-config-canary-29791935-ftgdw ocr-config-canary Succeeded
mp-ocr-f7658c6-54dwj            ocr          Running
mp-pantry-dc59d95c6-fcrxc        pantry       Running
mp-price-5bfcbbc758-pzrj7        price        Running
mp-price-5bfcbbc758-z7d4b        price        Running
mp@mealbong:~$
```

[그림 4.3.3-1] app 네임스페이스 워크로드 (사이드카 주입 상태)

합정, Istio 사이드카는 initContainers 에 있다

Istio 1.30 은 네이티브 사이드카 방식을 쓴다. istio-proxy 가 spec.containers 가 아니라 restartPolicy: Always 를 가진 initContainers 로 주입된다.

그래서 kubectl get pod 의 READY 열이나 spec.containers 만 세면 사이드카가 0 개로 보인다.

실제로 리소스 산정 중 이 착시로 '사이드카가 주입되지 않았다' 고 오판한 적이 있다. 사이드카 19 개 × 100m CPU 요청이 컨테이너 집계에서 통째로 빠져 있었다.

4.3.4 진입 구성, Gateway API · ALB · ACM · WAF

외부 트래픽은 ALB 가 받아 TLS 를 종단하고, 클러스터 내부의 Istio Gateway 로 전달한다. Gateway 는 진입 전용 네임스페이스 mp-ingress 에 분리 배치했다.

Cloudflare(DNS) ALB (ACM TLS 종단 · AWS WAF 부착) Istio Gateway
HTTPRoute 서비스

ALB 는 공개 서브넷 2 개(AZ-a · b)에 ENI 를 두고, Cilium ENI 모드 덕분에
파드를 직접 타킷한다

Gateway API v1.6.1 standard 채널 · 구현체는 Istio

**NLB 패스스루에서 ALB 로 정정했다, WAF 를 부착하려면 L7
로드밸런서여야 한다**

Gateway, mp-ingress 네임스페이스

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: mp-gw-public
  namespace: mp-ingress
spec:
  gatewayClassName: istio
  listeners:
  - name: http
    port: 80
    protocol: HTTP
  allowedRoutes:
    namespaces:
      from: Selector # Same 이 아니다, app ns 의 HTTPRoute 를 받아야 한다
  selector:
    matchLabels:
      kubernetes.io/metadata.name: app
```

검증

```
$ kubectl -n mp-ingress get gateway mp-gw-public
NAME      CLASS ADDRESS      PROGRAMMED AGE
mp-gw-public istio <ALB DNS>    True      ...
$ curl -s -o /dev/null -w '%{http_code}\n' https://aws.mealbong.cloud
200
```



이름	프로토콜	포트	대상 유형	상태 검사 경로
mp-tg-gw-public	HTTP	443	ip	/healthz/ready

[그림 4.3.4-1] ALB 대상 그룹 · 상태 검사

설계 근거, 진입점을 별도 네임스페이스로 분리한 이유

app 네임스페이스에 양방향 default-deny NetworkPolicy 를 적용하면서, 진입 게이트웨이가

같은

네임스페이스에 있으면 '외부에서 들어오는 트래픽' 과 '서비스 간 트래픽' 을 정책으로 구분할 수 없었다.

게이트웨이를 mp-ingress 로 분리하니 진입 정책과 앱 정책이 서로를 넘지 않는다.

분리 시 함정 : NetworkPolicy 의 podSelector 는 같은 네임스페이스만 본다.

cross-namespace 허용은 namespaceSelector 로 따로 써야 하고,

Gateway 의 allowedRoutes 도 Same 에서 Selector 로 바뀌야 한다.

4.3.5 리소스 관리 · 우선순위

네임스페이스별 자원 상한(ResourceQuota)과 컨테이너 기본값(LimitRange)을 두고, 노드가 찾을 때 무엇을 먼저 살릴지 PriorityClass 5 티어로 못박았다.

PriorityClass	값	적용 대상	의미
data-critical	1000000	PostgreSQL · Elasticsearch · PGSync	데이터를 잃지 않는 것이 최우선
platform-critical	900000	Cilium · CoreDNS · ArgoCD · ESO	복구 수단 자체, 이게 죽으면 손을 쓸 수 없다
observability-high	800000	Prometheus · Loki · Alertmanager	진단 능력, 장애 국면에 필요하다
app-normal	100000	애플리케이션 12 종	서비스 본체
pipeline-low	1000	수집 컨슈머 · CronJob	지연돼도 되는 것 · preemptionPolicy Never
mp-unclassified	0	globalDefault, 지정하지 않은 전부	분류 누락을 드러내기 위한 기본값

PriorityClass 정의 예

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: pipeline-low
  value: 1000
# 다른 파드를 밀어내지 않는다, 수집은 지연돼도 되지만 서비스를 죽여선 안 된다
preemptionPolicy: Never
globalDefault: false
description: "수집 파이프라인, 최하위"
```

검증, 미분류 파드가 남아 있지 않은지

```
$ kubectl get pods -A -o custom-columns=NS:.metadata.namespace,\
NAME:.metadata.name,PRIO:.spec.priority | awk '$3==0' | wc -l
0 # priority 0 파드 34 개 0 개
```

실측 사고 (1), ResourceQuota 가 오토스케일을 막고 있었다

app 네임스페이스의 ResourceQuota(mp-app-quota)가 메모리 6Gi 였는데 평시 사용률이 84% 였다.

그래서 HPA 가 replica 를 늘려도 파드가 생성되지 않았다.

진단이 어려웠던 이유 : 쿼터 초과는 Pending 이 아니라 FailedCreate(admission 거부)로 나온다.

Pending 파드가 없으므로 Karpenter 가 반응할 대상 자체가 존재하지 않았다.

즉 '노드를 늘려도 해결되지 않는' 형태의 병목이었다.

교훈 : 쿼터는 노드 용량보다 느슨해야 한다. 쿼터의 역할은 용량 계획이 아니라 폭주 차단이다.

실측 사고 (2), preemptionPolicy 는 불변 필드다

PriorityClass 를 만든 뒤 preemptionPolicy 를 바꾸려 하면 거부된다.

kubectl replace --force 로 삭제 후 재생성해야 하며, 그 사이(실측 791ms) 해당 클래스를 참조하는 새 파드는 스케줄되지 않는다. 운영 중 변경할 때는 이 창을 감안해야 한다.

4.3.6 오토스케일, HPA · KEDA · Karpenter

확장은 세 층으로 나뉜다. CPU 기반 파드 확장(HPA), 이벤트 기반 파드 확장(KEDA), 그리고 노드 확장(Karpenter)이다. 온프레미스에서는 앞의 두 층만 있었고, 노드 층은 AWS 에서 처음 생겼다.

층	기술	대상	설정
파드 (CPU)	HPA	mp-account · mp-recipe	min 2 / max 4 · CPU 기준
파드 (이벤트)	KEDA	수집 컨슈머 3 종	SQS 큐 깊이 · min 0 / max 1
노드	Karpenter	NodePool mp-burst	arm64 · on-demand · 버스트 전용

KEDA ScaledObject, SQS 큐 깊이 기반

```

apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: mp-refiner-retail
  namespace: pipeline
spec:
  scaleTargetRef: { name: mp-refiner-retail }
  minReplicaCount: 0      # 평시 0 파드
  maxReplicaCount: 1      # 1 을 넘기지 않는다 — 아래 근거 참조
  triggers:
    - type: aws-sqs-queue
      authenticationRef: { name: mp-keda-aws }
      metadata:
        queueURL: https://sqs.ap-northeast-2.amazonaws.com/<ACCOUNT_ID>/mp-crawl-retail
        queueLength: "5"
        awsRegion: ap-northeast-2
        identityOwner: operator # roleArn 을 여기 적으면 AccessDenied 가 난다

```

Karpenter NodePool, 버스트 전용

```

apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: mp-burst
spec:
  template:
    spec:
      requirements:
        - key: kubernetes.io/arch

```

```

operator: In
values: ["arm64"]
- key: karpenter.sh/capacity-type
operator: In
values: ["on-demand"]
nodeClassRef: { name: mp-default, group: karpenter.k8s.aws, kind: EC2NodeClass }
# 위치 주의 — EC2NodeClass 가 아니라 NodePool 이다.
# 잘못 넣으면 patched (no change) 로 조용히 무시된다.
startupTaints:
- key: node.cilium.io/agent-not-ready
  effect: NoExecute
  value: "true"
disruption:
consolidationPolicy: WhenEmptyOrUnderutilized

```

검증

```

$ kubectl get scaledobject -n pipeline
$ kubectl get hpa -n app
$ kubectl get nodeclaim # Karpenter 가 만든 노드
$ kubectl get nodes -l karpenter.sh/nodepool=mp-burst

```

```

mp@mealbong:~$ kubectl get scaledobjects -A

```

NAMESPACE	NAME	SCALETARGETKIND	SCALETARGETNAME	MIN	MAX	READY	ACTIVE	TRIGGER
pipeline	mp-deal-notifier	apps/v1.Deployment	mp-deal-notifier	0	1	True	False	aws-sqs-queue
pipeline	mp-recipe-refiner	apps/v1.Deployment	mp-recipe-refiner	0	1	True	False	aws-sqs-queue
pipeline	mp-retail-refiner	apps/v1.Deployment	mp-retail-refiner	0	1	True	False	aws-sqs-queue

```

mp@mealbong:~$

```

[그림 4.3.6-2] KEDA scale-to-zero 동작

실측 (1), Karpenter 과발주와 startupTaints

증상 : 파드 1 개를 스케줄하는 데 노드를 3 대 띄웠다.

원인 : 노드 지명(nomination)은 20 초 만에 끝나는데 노드가 실제로 Ready 가 되는 데 45 초가 걸린다.

그 사이 Karpenter 는 '아직 배치할 곳이 없다' 고 판단해 노드를 더 발주한다.

해결 : startupTaints 를 선언해 CNI 준비 전에는 배치 대상으로 세지 않게 했다 1 대로 수렴.

실측 : 노드 요청부터 Running 까지 51 초.

실측 (2), KEDA 의 IRSA 배선 위치

TriggerAuthentication 에 roleArn 을 적으면 AccessDenied 가 난다.

올바른 배선은 keda-operator 서비스 어카운트에 IRSA 를 붙이고,

트리거에는 identityOwner: operator 만 지정하는 것이다.

설계 판단, maxReplicaCount 를 1 로 묶은 이유

온프레미스에서는 Kafka 파티션이 product_id 로 나뉘어 있어 컨슈머를 늘려도 같은 상품을 두 워커가

동시에 건드리지 않았다. SQS 에는 그 파티션 친화성이 없다.

실제로 컨슈머를 2 개로 올렸을 때 PostgreSQL 데드락이 발생했다.

확장성을 포기한 것이 아니라, 순서 보장을 큐가 해주지 않으므로 애플리케이션이 감당할 때까지 묶어둔 것이다.

HPA 를 걸지 않은 서비스도 있다

부하 시험 결과 mealplan · chat 은 replica 를 늘려도 응답이 빨라지지 않았다.

두 서비스 모두 다운스트림(PostgreSQL · 외부 API) 대기가 지배적이기 때문이다.

늘려서 이득이 없는 것을 늘리지 않는 것도 설계 판단이며, 자원을 다른 서비스에 남긴다.

4.3.7 스토리지

온프레미스에서는 OpenEBS LVM LocalPV 를 사용했고, AWS 로 이전하면서 EBS gp3 CSI 로 교체했다. 가장 큰 차이는 볼륨이 노드에 결박되는지 여부다.

항목	온프레미스	AWS EKS
드라이버	OpenEBS LVM LocalPV 1.9.1	aws-ebs-csi-driver (애드온)
접근 모드	RWO (RWX 금지)	RWO
바인딩	WaitForFirstConsumer	WaitForFirstConsumer
StorageClass	openefs-lvm 2 종	gp3 (Delete) · gp3-retain (Retain)
기본 SC	Delete	gp3-retain (Retain),

		온프레미스와 반대
이동성	노드 결박, 노드를 벗어날 수 없다	AZ 내에서 재부착 가능
총량	PVC 125 GiB	EBS 520 GiB (노드루트 120 + PVC 230 + CI 140 + 대시보드 30)

StorageClass, gp3-retain (기본)

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3-retain
  annotations:
    storageclass.kubernetes.io/is-default-class: "true" # 기본을 Retain 쪽에 둔다
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  encrypted: "true"
  reclaimPolicy: Retain
  allowVolumeExpansion: true
  volumeBindingMode: WaitForFirstConsumer
```

검증

```
$ kubectl get sc
NAME PROVISIONER RECLAIMPOLICY DEFAULT
gp3 ebs.csi.aws.com Delete false
gp3-retain (default) ebs.csi.aws.com Retain true
$ kubectl get pv -o custom-columns=NAME:.metadata.name,POLICY:.spec.persistentVolumeReclaimPolicy
# 전 PV 가 Retain 인지 확인
```

설계 근거, 기본 StorageClass 를 Retain 으로 둔 이유

온프레미스에서는 라이브 PV 21 개를 Retain 으로 일괄 전환했지만(파드 재시작 0 으로 무중단 적용),

기본 StorageClass 는 여전히 Delete 였다. 즉 그 뒤에 새로 만들어지는 PVC 는 보호 밖이었다.

AWS 에서는 이 구멍을 처음부터 막았다, 기본값을 Retain 으로 두면,

실수로 PVC 를 지워도 EBS 볼륨이 남아 데이터를 되찾을 수 있다.

대가는 쓰지 않는 볼륨이 쌓일 수 있다는 것인데, 그건 청소로 해결되고 데이터 삭제는

되돌릴 수 없다.

LocalPV 의 한계, 실제로 겪은 사례

온프레미스에서 kubecost 의 PV 3 종이 worker-a2 에 결박돼 있었다(pv.spec.nodeAffinity).
노드를 정비하려고 cordon 하자 그 파드는 어디에도 뜰 수 없었다, LocalPV 는 정의상
노드를 못 벗어난다.

이 제약이 EBS 로 옮기는 실질적 근거 중 하나였다. EBS 는 같은 AZ 안이면 다른 노드에
재부착된다.

다만 AZ 는 못 넘는다, 그래서 PVC 를 쓰는 워크로드는 AZ 고정이라는 제약이 남는다.

4.3.8 데이터 티어

PostgreSQL 과 Elasticsearch 는 매니지드로 가지 않고 오퍼레이터로 클러스터
안에서 운영한다. Redis 만 ElastiCache 로 옮겼다.

구성요소	운영 방식	형상	비고
PostgreSQL	CloudNativePG (오퍼레이터)	pg-1 (primary) + pg-2 (standby) + pgBouncer Pooler	RDS 미채택
Elasticsearch	ECK (오퍼레이터)	3 노드 · replicas 1 커스텀 이미지 mp- elasticsearch-nori	OpenSearch Service 미채택
Redis	Amazon ElastiCache for Valkey	cache.t4g.micro × 2 (Multi-AZ)	온프레미스는 오퍼레이터 + Sentinel 이었다
메시지 큐	없음 (클러스터 내)	S3 이벤트 알림 SQS 로 대체	MSK · Strimzi 모두 미채택
CDC	PGSync	Deployment replicas 1 고정	복제 슬롯이 단일 소비자

CloudNativePG Cluster (요약)

```
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: pg
  namespace: data
spec:
  instances: 2
  primaryUpdateStrategy: unsupervised
  storage: { storageClass: gp3-retain, size: 40Gi }
  postgresql:
    parameters:
      archive_timeout: "302s"    # RPO 5 분의 근거
  plugins:
    - name: barman-cloud.cloudnative-pg.io
      parameters: { barmanObjectName: mp-pg-backup }
```

검증

```
$ kubectl -n data get cluster pg
NAME AGE INSTANCES READY STATUS PRIMARY
pg ... 2 2 Cluster in healthy state pg-1
$ kubectl -n data get pooler
$ kubectl -n data get elasticsearch
$ aws elasticache describe-replication-groups --replication-group-id mp-cache
```

```
mp@mealbong:~$ kubectl -n data get cluster pg ; kubectl -n data get pooler
NAME INSTANCES READY PRIMARY STATUS
pg 2 2 pg-2 Cluster in healthy state

NAME AGE CLUSTER TYPE STATUS
pg-pooler 10d pg rw active
mp@mealbong:~$
```

[그림 4.3.8-1] CloudNativePG 클러스터 상태 · Pooler

선택 근거, RDS · OpenSearch Service 를 쓰지 않은 이유

매니지드 DB 에서 실제로 사는 것은 (1) 백업 (2) 페일오버 (3) 패치 세 가지다.

그런데 (1)(2)는 CloudNativePG 가 이미 제공한다, barman-cloud 로 S3 연속 백업, 자동 페일오버.

(3)은 얻지만, 대신 잃는 것이 크다 :

Elasticsearch 는 한국어 nori 형태소 분석기를 넣은 커스텀 이미지를 쓴다. 매니지드에는 넣을 수 없다.

PostgreSQL 은 스키마-퍼-서비스 + role 격리 구조인데, 이를 CR 로 선언적으로 관리하고

있다.

두 사이트(온프레미스 · AWS)에 같은 매니페스트를 쓸 수 있는 이식성이 사라진다.

반대로 Redis 는 ElastiCache 로 옮겼다. 판단 축이 달랐기 때문이다

비용이 아니라 Sentinel 운영 부담이었다. 오퍼레이터 결함을 우회하는 코드가 프로덕션에 있었고

(master Service 가 노드 상실 국면에 갇히지 않아 클라이언트가 Sentinel 을 직접 봐야 했다),

그 코드를 지울 수 있다는 것이 관리형으로 옮기는 실익이었다. 파드 5 개가 0 개가 된 것은 덤이다.

4.3.9 배포, ArgoCD GitOps · Blue-Green

CI 는 클러스터 밖(GitLab EC2), CD 는 클러스터 안(ArgoCD)에 둔다. 빌드가 죽어도 배포 상태는 유지되며, 클러스터의 desired state 는 항상 config 레포에 있다.

레포를 2 개로 분리, 앱 레포(소스 · Dockerfile · IaC)와 config 레포(K8s 매니페스트만)

CI 가 이미지를 굽고 config 레포의 이미지 태그를 :sha 로 커밋 여기가 CD 인계 지점

ArgoCD 가 config 레포를 watch 하여 클러스터에 반영 (웹훅 · push 후 약 2 초)

Application 44 개 · 뿌리를 2 개로 분리(mealplanning-root = 앱 / platform-root = 플랫폼)

config 레포는 Kustomize 오버레이를 overlays/onprem · overlays/eks 로 나눠 두 사이트를 함께 관리

account · recipe 는 Argo Rollouts Blue-Green 으로 배포

Argo Rollouts, Blue-Green

```

apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: mp-account
  namespace: app
spec:
  replicas: 2
  strategy:
    blueGreen:
      activeService: account
      previewService: account-preview
      autoPromotionEnabled: false      # 사람이 승격한다
      scaleDownDelaySeconds: 300      # 구 버전을 5 분 유지 — 즉시 롤백 대비
      prePromotionAnalysis:
        templates: [{ templateName: mp-success-rate }]

```

검증 · 운영 명령

```

$ kubectl argo rollouts get rollout mp-account -n app
$ kubectl argo rollouts promote mp-account -n app # 승격
$ kubectl argo rollouts undo mp-account -n app # 롤백
$ kubectl -n argocd get applications | grep -v Synced # 어긋난 것만

```

```

mp@mealbong:~$ kubectl -n app get rollouts
NAME           DESIRED  CURRENT  UP-TO-DATE  AVAILABLE  AGE
mp-account     2        2        2           2          10d
mp-recipe      2        2        2           2          10d
mp@mealbong:~$

```

[그림 4.3.9-1] Argo Rollouts — Blue-Green 대상 워크로드

실측, 무중단 전환

승격 소요 1 초 · 롤백 소요 2 초 · 전환 중 1,409 요청에서 5xx 0 건.

다만 '전 서비스 Blue-Green' 은 아니다. 실제 적용은 account · recipe 2 종이고,
나머지는 롤링 업데이트다. 전 서비스 전환은 확정된 방향이지 현재 형상이 아니다.

실측 사고, 승격 게이트가 배포를 죽였다

Blue-Green 은 승격 전에 AnalysisRun 으로 성공률을 검사한다. 그 지표원이 Prometheus 였다.

그런데 Blue-Green 은 순간적으로 파드가 2 배가 되고, 그 버스트가 Prometheus 를 자원 경합으로 밀어냈다.

결과적으로 지표를 못 읽어 분석이 실패하고, 정상 배포가 롤백됐다.

교훈 : 승격 게이트는 승격에 흔들리는 것에 의존하면 안 된다.

운영 함정, ArgoCD sync 는 hard refresh 가 먼저다
 config 레포에 push 한 직후 sync 를 걸면 ArgoCD 는 캐시된 옛 HEAD 를 본다.
 명령은 성공으로 끝나는데 실제로는 아무 일도 일어나지 않고, 실측 5 분 뒤에야 반영됐다.
 해결 : kubectl annotate application <앱> argocd.argoproj.io/refresh=hard 를 먼저 준 뒤 sync 20 초.

4.4 GitLab CI/CD

CI 는 클러스터 밖의 전용 VPC(mp-vpc-ci)에 EC2 1 대를 두고 GitLab · GitLab Runner · SonarQube 를 구성해 수행한다. CD 는 클러스터 안의 ArgoCD 가 담당하며, 두 역할은 config 레포 커밋을 경계로 나뉜다.

(1) CI 서버 EC2 생성

항목	값
AMI	Ubuntu 24.04 LTS (arm64)
인스턴스 유형	t4g.xlarge (Graviton · 4 vCPU / 16 GiB)
네트워크	mp-vpc-ci / mp-subnet-ci-public (10.11.0.0/24) · 공인 IP 할당
보안 그룹	mp-sg-ci, SSH · HTTPS 인바운드
스토리지	140 GiB gp3
VPC 피어링	없음, 빌드 서버가 서비스 VPC 를 볼 수 없다
S3 Endpoint	mp-vpce-ci-s3, NAT 가 없으므로 ECR 레이어가 이 경로로 나간다(빌드마다 수 GB)

(2) GitLab 설치

```
sudo apt-get update
sudo apt-get install -y curl openssh-server ca-certificates tzdata perl postfix
# GitLab CE 저장소 등록 및 설치
curl https://packages.gitlab.com/install/repositories/gitlab/gitlab-ce/script.deb.sh | sudo bash
sudo apt-get install -y gitlab-ce
# /etc/gitlab/gitlab.rb, 외부 URL 설정 (TLS 는 Cloudflare Tunnel 이 중단)
external_url "https://gitlab.mealbong.cloud"
```

```
nginx['listen_https'] = false
nginx['listen_port'] = 80
sudo gitlab-ctl reconfigure
sudo cat /etc/gitlab/initial_root_password | grep Password:
```

구축은 Ansible 로 자동화했다. 역할은 다음과 같이 분리했다.

Ansible 룰	담당
gitlab_base	OS 베이스라인 · 패키지 · 디스크
gitlab_server	GitLab CE 설치 · gitlab.rb · reconfigure
gitlab_runner	Runner 설치 · 등록 · concurrent 설정
gitlab_cloudflared	Cloudflare Tunnel, UI 노출 경로
gitlab_hardening	보안 설정 (API 경유 · 가입 차단 · 보호 브랜치)
gitlab_sonarqube	SonarQube 컨테이너
gitlab_ci_vars	CI/CD 변수 주입 (MP_CI_ROLE_ARN 등)
gitlab_repo_import / _sync	GitHub GitLab 초기 임포트 및 동기화

Ansible 실행

```
ansible-playbook -i inventory_aws.aws_ec2.yml gitlab.yml
```

(3) 파이프라인 구조

stages: [test, build, deploy], 3 단계

잡은 서비스별로 나뉘며(test:account · build:account ...), rules:changes 로
변경된 서비스만 돈다

SERVICES 변수로 수동 지정도 가능 (SERVICES="all" 이면 전체)

빌드는 arm64 네이티브, 멀티아치 manifest 를 만들지 않는다

Trivy 게이트(CRITICAL · --ignore-unfixed)를 통과해야 ECR push 가
일어난다

.gitlab-ci.yml, 파이프라인 생성 조건과 공통 변수

stages: [test, build, deploy]

workflow:

rules:

- if: \$CI_COMMIT_BRANCH == "main"
- if: \$CI_PIPELINE_SOURCE == "merge_request_event"
- if: \$CI_PIPELINE_SOURCE == "web"
- if: \$CI_PIPELINE_SOURCE == "api"
- when: never # 그 외에는 파이프라인을 만들지 않는다

variables:

```
MP_AWS_REGION: "ap-northeast-2"
HARBOR_IMAGE_PREFIX: "192.168.0.10/mealplanning"
PLATFORM: "linux/arm64"
TRIVY_IMAGE: "mirror.gcr.io/aquasec/trivy:0.72.0"
KUSTOMIZE_VERSION: "v5.4.3"
```

.build, 빌드 · 스캔 · push 템플릿 (요약)

```
.build:
  stage: build
  id_tokens:
    AWS_TOKEN:      # 정적 AWS 키를 CI 변수에 넣지 않는다 — OIDC 로 신원을 증명한다
    aud: sts.amazonaws.com
  script:
    - |
      PUSH=0
      case "${MP_PUSH_REFS:-main}" in *) ${CI_COMMIT_BRANCH:-} *) PUSH=1 ;; esac
      [ "${CI_PIPELINE_SOURCE:-}" = "merge_request_event" ] && PUSH=0
      # 계정 ID 는 롤 ARN 에서 꺼낸다 — 레포가 공개라 계정 ID 를 커밋하지 않는다
      ACCOUNT="$(echo "${MP_CI_ROLE_ARN}" | cut -d: -f5)"
      export ECR_REGISTRY="${ACCOUNT}.dkr.ecr.${MP_AWS_REGION}.amazonaws.com"
      if [ "${PUSH}" = "1" ]; then
        echo "${AWS_TOKEN}" > /tmp/mp-oidc-token
        export AWS_WEB_IDENTITY_TOKEN_FILE=/tmp/mp-oidc-token
        export AWS_ROLE_ARN="${MP_CI_ROLE_ARN}"
        aws ecr get-login-password | docker login --username AWS --password-stdin "${ECR_REGISTRY}"
      fi
    - docker build --platform "$PLATFORM" -f "$DOCKERFILE" -t "$IMG:$CI_COMMIT_SHA" "$CONTEXT"
    - docker run --rm $TRIVY_IMAGE image --severity CRITICAL --ignore-unfixed \
      --exit-code 1 "$IMG:$CI_COMMIT_SHA"      # 여기서 막히면 push 하지 않는다
```

```
- '[ "$PUSH" = "1" ] && docker push "$IMG:$CI_COMMIT_SHA"
```

(4) CD 인계, config 레포 커밋

config 레포 이미지 태그 편

```
kustomize edit set image \
  ${HARBOR_IMAGE_PREFIX}/${IMAGE}=${IMAGE_PREFIX}/${IMAGE}:${CI_COMMIT_SHA}
git commit -am "cd(${IMAGE}): ${CI_COMMIT_SHA}"
git push origin main
# ArgoCD 가 감지하여 클러스터에 반영 (약 2 초)
```

설계 근거 (1), 왜 정적 키가 아니라 OIDC 인가

CI 변수에 AWS 액세스 키를 넣으면 그 키는 만료되지 않고, 유출돼도 회수할 때까지 유효하다.

GitLab OIDC 는 잡마다 수명이 짧은 토큰을 발급하고, IAM 신뢰정책이 sub(브랜치)와 aud 를 검증한다.

신뢰정책의 sub 가 브랜치를 못박고 있어서(기본 main), 허용 밖 브랜치는 AssumeRole 이 거부된다.

그래서 잡 안에서 PUSH 판정을 먼저 하고, 허용 밖이면 build + scan 까지만 하고 끝낸다.

이 두 곳(IAM 의 ci_oidc_allowed_refs 와 스크립트의 MP_PUSH_REFS)은 반드시 같이 움직여야 한다.

설계 근거 (2), MR 파이프라인은 ECR 에 올리지 않는다

merge_request_event 이면 PUSH=0 으로 강제한다.

MR 파이프라인은 테스트 · 빌드 · Trivy 까지만 하고 레지스트리를 건드리지 않는다.

검문은 하되 아직 배포 가능한 산출물을 만들지는 않는다는 뜻이다.

실측 사고 (1), 동기화가 파이프라인을 26 잡씩 만들었다

초기에 GitLab 은 GitHub 의 하류 사본이었고, 브랜치가 동기화로 밀려 들어왔다.

새 브랜치의 첫 파이프라인은 rules:changes 의 비교 기준이 없어 모든 잡이 매칭된다.

실측(2026-08-14) : PR 브랜치 2 개가 들어오면서 파이프라인 2 개가 main 파이프라인을

30 분 밀어냈다.

러너가 1 대(concurrent 2)라 큐가 곧 지연이었다.

해결 : workflow.rules 로 main · MR · 수동 트리거만 파이프라인을 만들게 제한했다.

실측 사고 (2), Docker Hub 가 빌드의 임계 경로에 있었다

DinD 는 잡마다 새 데몬이라 이미지 캐시가 없어, 전체 런에서 Trivy 이미지를 16 번 pull 한다.

실측 : 502 Bad Gateway 로 build:video 가 실패했다. 빌드는 성공했는데 스캐너를 못 받아 죽은 것이라

원인이 보이지 않는 형태였다.

해결 : TRIVY_IMAGE 를 mirror.gcr.io(Google 이 운영하는 Docker Hub 풀스루 캐시)로 바꿨다.

실측 사고 (3), MR 파이프라인이 아예 안 생기는 경우

모든 잡이 rules:changes 게이트를 갖고 있어서, docs · infra · CI 파일만 바꾸는 MR 은 매칭되는 잡이 하나도 없고 파이프라인 자체가 생성되지 않는다.

그러면 '필수 파이프라인' 으로 걸 대상이 없어 머지 게이트가 사실상 사라진다.

게다가 MR 위젯은 옛 파이프라인을 붙들고 표시해서 생성 실패를 숨긴다.

판정하려면 head_pipeline.sha 와 MR 의 sha 를 비교해야 한다.

해결 : 아무 잡도 안 걸릴 때 도는 폴백 잡 mr-guard 를 추가했다.

4.5 Observability

개요

본 프로젝트는 Docker 에서 시작한 모니터링 구조를 On-premise Kubernetes 와 AWS EKS 로 이전하면서 Metric·Log·Trace 수집 방식을 유지·확장하고, 최종적으로 별도 Operations EC2 에서 최근 Metric 측정값 기반 이상징후 탐지, 관련 Alert 를 하나의 장애 건으로 묶는 Incident

상관분석, Metric·Log·Trace 등 장애 근거 데이터(Evidence)를 활용한 Amazon Bedrock 기반 근본 원인 분석(RCA), 운영 자동화를 수행하도록 구축하였다.

배경 및 필요성

MSA 환경에서 장애가 발생하면 Metric·Log·Trace 를 각각 확인해야 해 원인 파악 시간이 길어지는 문제를 줄이기 위해 통합 Observability 환경을 구성하였다.

크롤러 수집량처럼 요일·딜마다 정상 범위가 달라지는 지표는 고정 임계값만으로 오탐·미탐이 반복될 수 있어 최근 Metric 측정값의 변화 패턴을 기준으로 이상징후 탐지를 추가하였다.

하나의 장애가 CPU·지연·오류율 등 여러 Alert 로 확산되는 경우, 발생 시간과 서비스 정보를 기준으로 관련 Alert 를 하나의 Incident(장애 조사 단위)로 묶고 Metric·Log·Trace 등 근거 데이터를 자동 수집하여 조사 우선순위를 명확히 하도록 하였다.

환경 정보

단계	배포 방식	구성	목적
Docker	Ansible + Docker Compose	Prometheus · Alertmanager · Grafana · Loki · Tempo / node-exporter · cAdvisor · Alloy	VM·Container 관측
Kubernetes	Ansible + Helm/ArgoCD	kube-prometheus-stack · Alloy · Loki · Tempo · Grafana · Istio	Cluster 내부 관측
AWS EKS	EKS + S3 / ServiceAccount-IAM 연계(IRSA)	동일 관측 Stack + Loki/Tempo S3 저장	Cloud 환경으로 이전

Operations	Terraform + EC2	운영 분석 API(Operations API) · Dashboard · PostgreSQL 연동 · Bedrock · Slack	이상징후 탐지·장애 그룹화·원인 분석·운영 자동화
------------	-----------------	---	-----------------------------------

기능

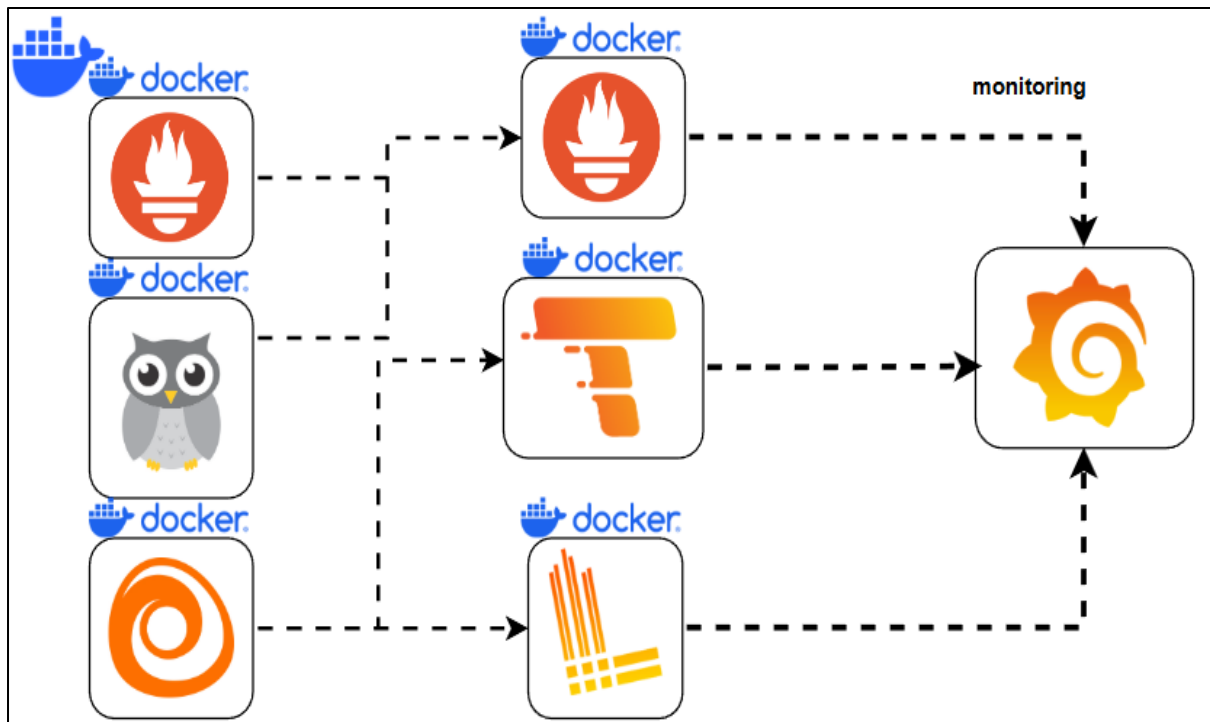
구분	기능
통합 관측	Node/Pod/Container Metric, Container Log, Application/Mesh Trace 를 Grafana 에서 동일 시간대로 조회
Alert	PrometheusRule Alertmanager Slack 및 Operations API Webhook 전달
이상징후	최근 기준선 대비 Z-score·MAD·변화율·연속 이탈 조건으로 조사 대상 우선순위화
장애 그룹화 / 원인 분석	관련 Alert 를 Incident(장애 조사 단위)로 묶고 Metric·Log·Trace·Kubernetes Event·배포 변경을 장애 근거 데이터(Evidence)로 구성하여 Bedrock 기반 RCA 초안을 생성
운영 자동화	장애 발생(FIRING)·해결(RESOLVED) Slack 알림, 매일 09:05 일일 리포트, 장애 건(Incident) 기반 Operations 헬프데스크 챗봇

구현 절차

4.5.1 Docker 기반 모니터링 구축

초기 단계에서는 각 애플리케이션을 Docker Compose 로 실행하고, 별도 모니터링 서버에 Prometheus·Alertmanager·Grafana·Loki·Tempo 를 두었다.

각 서비스 VM 에는 Ansible 템플릿으로 node-exporter·cAdvisor·Alloy 를 배포하여 인프라와 컨테이너 상태를 수집하였다.



a. VM 관측 Agent 배포

Docker Compose 기반 Agent 구성

적용 파일: `/infra/ansible/roles/monitoring_agents/templates/compose.yml.j2`

services:

node-exporter:

image: quay.io/prometheus/node-exporter:latest

network_mode: host

volumes:

- '/:/host:ro,rslave'

cadvisor:

image: gcr.io/cadvisor/cadvisor:latest

privileged: true

ports: ["8080:8080"]

volumes:

- /var/run:/var/run:ro

- /sys:/sys:ro

- /var/lib/docker:/var/lib/docker:ro

alloy:

image: grafana/alloy:latest

volumes:

- ./config.alloy:/etc/alloy/config.alloy:ro

- /var/run/docker.sock:/var/run/docker.sock:ro

- **node-exporter** 는 Host Metric, **cAdvisor** 는 Container Metric 을
노출하고, **Alloy** 는 Docker socket 을 이용해 실행 중인 Container
Log 를 수집하도록 배포하였다.

b. Log 수집 및 Trace 계측**Alloy - Docker Log Loki**

적용 파일: /infra/ansible/roles/monitoring_agents/templates/config.alloy.j2

```

discovery.docker "containers" {
  host = "unix:///var/run/docker.sock"
}
discovery.relabel "containers" {
  targets = discovery.docker.containers.targets
  rule {
    source_labels = ["__meta_docker_container_name"]
    target_label = "container"
  }
  rule {
    source_labels = ["__meta_docker_container_label_com_docker_compose_service"]
    target_label = "service_name"
  }
}
loki.source.docker "default" {
  host = "unix:///var/run/docker.sock"
  targets = discovery.relabel.containers.output
  forward_to = [loki.write.default.receiver]
}

```

- Alloy 가 Container 를 자동 발견하고 container/service_name Label 을 붙여 Loki 로 전송하여 서비스 단위 로그 검색이 가능하도록 하였다.

FastAPI OpenTelemetry Tempo

적용 파일: /deploy/app/docker-compose.yml

```

OTEL_TRACES_EXPORTER=otlp_proto_grpc
OTEL_EXPORTER_OTLP_ENDPOINT=192.168.0.11:4317
OTEL_EXPORTER_OTLP_INSECURE=true
OTEL_TRACES_SAMPLER=parentbased_traceidratio
OTEL_TRACES_SAMPLER_ARG=0.1

```

- FastAPI 는 opentelemetry-instrument 자동 계측으로 Span 을 생성하고 Tempo 의 OTLP gRPC 포트(4317)로 직접 전송하였다.

4.5.2 Kubernetes 기반 모니터링 구축

Kubernetes 전환 후에는 모니터링 서버에 분리되어 있던 기능을 Cluster 내부로 이전하였다. Ansible 을 배포 진입점으로 사용하여 Prometheus 계열은

Helm 으로 설치하고, Loki·Tempo·Alloy 는 ArgoCD Application 을 등록해 자동 동기화되도록 구성하였다.

a. Prometheus · Alertmanager · Grafana 배포

Ansible 에서 kube-prometheus-stack 설치

적용 파일: /infra/ansible/roles/k8s_observability/tasks/main.yml

```
- name: kube-prometheus-stack 설치
  kubernetes.core.helm:
    name: kube-prometheus-stack
    chart_ref: prometheus-community/kube-prometheus-stack
    chart_version: "{{ kube_prom_stack_version }}"
    release_namespace: observability
    values_files:
      - /etc/kubernetes/kube-prometheus-stack-values.yaml
    wait: true
    wait_timeout: 15m

- name: node-exporter 설치 (kube-system)
  kubernetes.core.helm:
    name: node-exporter
    chart_ref: prometheus-community/prometheus-node-exporter
    release_namespace: kube-system
    values:
      prometheus:
        monitor:
          enabled: true
```

- kube-prometheus-stack 으로

Prometheus·Alertmanager·Grafana·kube-state-metrics 를 구성하고,
Host 접근이 필요한 **node-exporter** 는 **kube-system** 에 별도 배포하여
ServiceMonitor 가 자동 생성되도록 하였다.

Prometheus 수집 대상 설정

적용 파일: /infra/ansible/roles/k8s_observability/templates/kube-prometheus-stack-values.yaml.j2

prometheus:

```

prometheusSpec:
  retention: {{ prometheus_retention_in_cluster }}
  storageSpec:
  volumeClaimTemplate:
    spec:
      storageClassName: openebs-lvm
      resources:
        requests:
          storage: {{ prometheus_pvc_size }}
      serviceMonitorSelectorNilUsesHelmValues: false
      podMonitorSelectorNilUsesHelmValues: false
      ruleSelectorNilUsesHelmValues: false

```

- **ServiceMonitor·PodMonitor** 로 애플리케이션과 Kubernetes 대상 **Metric** 을 수집하고, **PrometheusRule** 을 통해 **Alert** 규칙을 Cluster 내부에서 관리하도록 구성하였다.

b. Pod Log 수집

Alloy DaemonSet - Node 별 Pod Log 수집

적용 파일: /infra/ansible/roles/k8s_platform_apps/templates/lgtm-apps.yaml.j2

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: alloy
spec:
  source:
    chart: alloy
    helm:
      valuesObject:
        alloy:
          configMap:
            content: |
              discovery.kubernetes "pods" {
                role = "pod"
                selectors {
                  field = "spec.nodeName=" + sys.env("NODE_NAME")
                }
              }
          loki.source.file "pod_logs" {
            targets = local.file_match.pod_logs.targets
            forward_to = [loki.write.default.receiver]
          }
        controller:
          type: daemonset

```

- Alloy 를 DaemonSet 으로 실행하여 모든 Node 의 /var/log/pods 를 수집하고 Namespace·Pod·Container Label 을 유지한 상태로 Loki 에 저장하도록 구성하였다.

c. Application Trace + Istio Mesh Trace

Istio OpenTelemetry Provider Tempo

적용 파일: /infra/ansible/roles/k8s_istio/templates/istiod-values.yaml.j2,
telemetry-tracing.yaml.j2

```
meshConfig:
  extensionProviders:
    - name: mp-tempo-otlp
  opentelemetry:
    service: tempo.observability.svc.cluster.local
    port: 4317
    apiVersion: telemetry.istio.io/v1
    kind: Telemetry
    metadata:
      name: mp-mesh-tracing
      namespace: istio-system
    spec:
      tracing:
        - providers:
            - name: mp-tempo-otlp
        randomSamplingPercentage: 100
```

- **Application Span** 은 FastAPI OpenTelemetry 자동 계측이 Tempo 로 직접 전송하고, 서비스 간 네트워크 요청은 Istio Envoy 가 OpenTelemetry Provider 를 통해 Tempo 로 전송하였다.

d. Alertmanager Slack / Operations

Alert 그룹화 및 Slack 전송

적용 파일: /infra/ansible/roles/k8s_observability/templates/kube-prometheus-stack-values.yaml.j2

alertmanager:

```

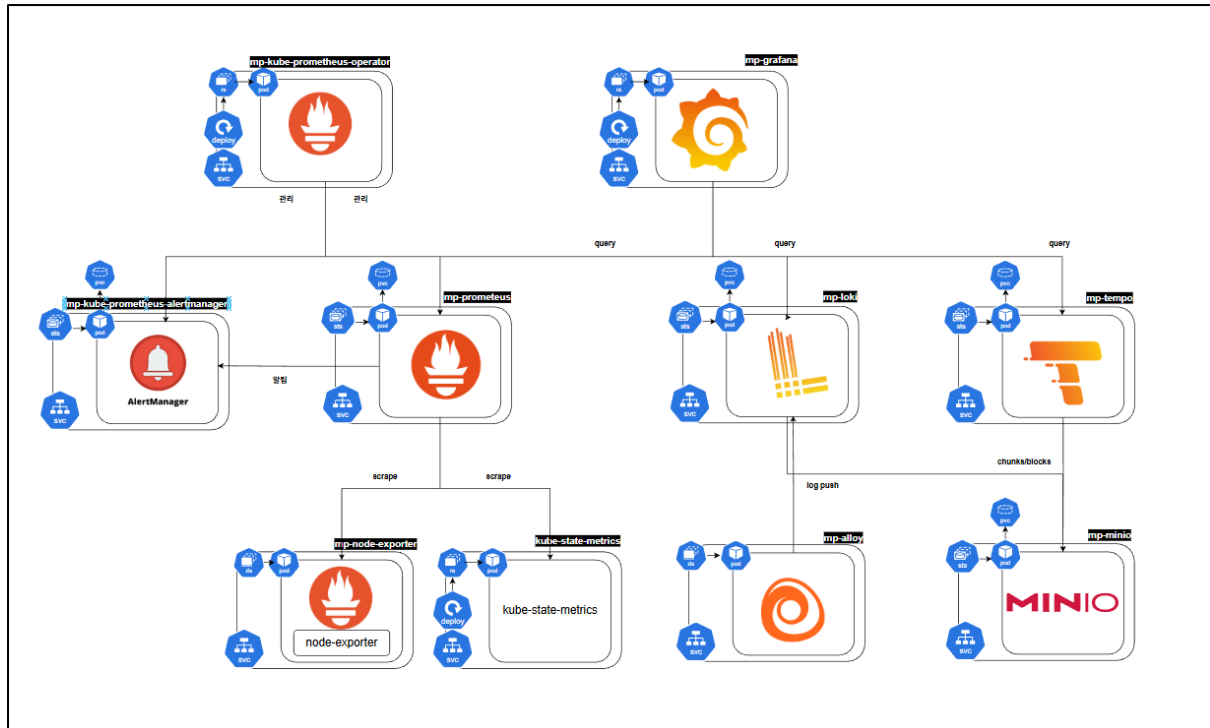
config:
route:
receiver: slack-default
group_by: ['alertname', 'service', 'namespace']
group_wait: 2m
group_interval: 30m
repeat_interval: 24h
receivers:
- name: slack-default
slack_configs:
- channel: '#monitoring-alerts'
send_resolved: true
webhook_configs:
- url: http://operations.app.svc:8011/internal/alerts/alertmanager
send_resolved: true

```

- 동일 원인의 Alert 를 서비스/Namespace 기준으로 묶고, 장애 발생(FIRING)·해결(RESOLVED) 상태를 Slack 에 전송하였다. Kubernetes 단계에서는 동일 Alert JSON 을 Operations API Webhook 에도 전달하여 관련 Alert 그룹화(Incident 상관분석)의 입력으로 사용하였다.

4.5.3 AWS EKS 기반 Observability 이전

AWS 이전에서는 Kubernetes 단계의 Prometheus·Alertmanager·Alloy·Loki·Tempo·Grafana 구조를 EKS 에 유지하였다. 변경점은 Loki 와 Tempo 의 장기 저장 계층을 온프레미스 저장소에서 S3 로 전환하고 이상징후 분석/운영 UI 를 별도 EC2 로 분리한 것이다.



a. Loki · Tempo 저장소를 S3 로 전환

Loki·Tempo 의 S3 접근 권한 구성(IRSA)

적용 파일: /infra/terraform/aws-platform/s3_observability.tf

```
resource "aws_s3_bucket" "observability" {
  bucket = "mp-observability-eks"
}

resource "aws_s3_bucket_server_side_encryption_configuration" "observability" {
  bucket = aws_s3_bucket.observability.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_iam_role" "loki_s3" {
  name = "mp-loki-s3"
  assume_role_policy = data.aws_iam_policy_document.irs_trust["loki_s3"].json
}

resource "aws_iam_role" "tempo_s3" {
  name = "mp-tempo-s3"
  assume_role_policy = data.aws_iam_policy_document.irs_trust["tempo_s3"].json
}
```

- Loki 와 Tempo Pod 가 AWS Access Key 를 직접 저장하지 않고, 각 Kubernetes ServiceAccount 에 연결된 IAM Role 을 통해 S3 에 접근하도록 구성하였다. Loki 와 Tempo 의 IAM Role 을 각각 분리해

필요한 S3 권한만 부여하였다. 애플리케이션 보존 정책은 7 일이며 S3 30 일 Lifecycle 은 compactor 가 남긴 orphan 객체를 정리하는 백스톱으로 사용하였다.

b. Operations EC2 구축 및 EKS 관측 데이터 연결

Operations EC2 와 Alertmanager Webhook 허용

적용 파일: /infra/terraform/mp-dashboard/dashboard_ec2.tf

```
resource "aws_vpc_security_group_ingress_rule" "dashboard_alertmanager_webhook" {
  security_group_id      = data.aws_security_group.dashboard.id
  referenced_security_group_id = data.aws_security_group.eks_node.id
  ip_protocol = "tcp"
  from_port   = 8011
  to_port     = 8011
}
resource "aws_instance" "dashboard" {
  ami          = data.aws_ami.al2023_x86.id
  instance_type = "t3.medium"
  subnet_id    = data.aws_subnet.dashboard.id
  iam_instance_profile = aws_iam_instance_profile.dashboard.name
  root_block_device {
    volume_type = "gp3"
    volume_size = 30
    encrypted   = true
  }
}
```

- Operations Dashboard/API 는 EKS 와 장애 도메인을 분리하기 위해 별도 t3.medium EC2 에 배치하였다. Alertmanager Webhook 8011 포트는 EKS Node Security Group 에서 오는 트래픽만 허용하고, Operations API 는 같은 VPC 의 private 경로로 EKS Prometheus·Loki·Tempo 를 조회하도록 구성하였다.

c. Operations API 에서 Prometheus Metric 수집기 실행

FastAPI 시작 시 Prometheus Metric 수집기(Collector) 상시 실행

적용 파일: /operations-backend-ec2/app/main.py

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    settings = Settings()
    pool = make_pg_pool(settings)
    await pool.open()
    collector = PrometheusCollector(settings=settings, analyzer=_analyzer)
    collector_task = asyncio.create_task(
        collector.run_forever(pool.connection),
        name="operations-prometheus-collector",
    )
    try:
        yield
    finally:
        collector_task.cancel()
        await pool.close()
```

- 운영 분석 API(Operations API)가 시작되면 PrometheusCollector 를 background task 로 실행하여 EKS Prometheus 의 Metric 측정값을 주기적으로 수집·평가하고 이상징후 후보를 PostgreSQL 에 저장하도록 구성하였다.

4.5.4 이상징후 Dashboard 구축

a. 이상징후 탐지 정책

고정 임계값을 완전히 대체하는 구조가 아니라 정상 기준선에서 벗어난 변화를 먼저 조사 대상으로 올리고 운영상 실제 영향 기준을 함께 사용하였다. 이를 통해 작은 값의 통계적 변화가 과도한 Alert 로 이어지는 문제를 줄였다.

Metric 측정값 기반 이상징후 분석 기준(Analyzer 정책)

적용 파일: /operations-backend-ec2/app/models.py

```
class AnalyzerConfig(BaseModel):
```

```
    baseline_window: int = 60
```

```
    min_samples: int = 30
```

```
    z_threshold: float = 3.0
```

```
    mad_threshold: float = 3.5
```

```
    change_rate_threshold: float = 0.5
```

```
    consecutive_windows: int = 3
```

```
    severe_breach_multiplier: float = 2.0
```

```
    rebaseline_after_windows: int = 30
```

- 최근 60 개의 Metric 측정값을 기준선으로 사용하고 최소 30 개의 측정값이 확보된 이후 분석한다. Z-score·MAD·변화율을 함께 계산하고 기본 3 개 구간 연속 이탈 시 이상징후(anomaly)로 확정한다. 기준 대비 매우 큰 이탈은 심각 이탈(severe breach)로 즉시 승격하며, 장시간 새로운 수준이 유지되면 기준선 재설정(rebaseline)을 통해 기준선이 영구적으로 낮지 않도록 하였다.

Metric 별 오탐 방지 기준

적용 파일: /operations-backend-ec2/app/metric_catalog.py

CatalogMetric(

```
metric_id="service_5xx_rate",
minimum_current_value=0.1,
minimum_absolute_delta=0.1,
p95_request_rate_guard=True,
impact_threshold=5.0,
)
```

CatalogMetric(

```
metric_id="pod_cpu_utilization",
minimum_current_value=1.0,
impact_threshold=85.0,
)
```

- 통계 점수가 높더라도 실제 값이 너무 작으면 조사 대상에서 제외하도록 Metric 별 최소 현재값/최소 변화량을 두었다. 반대로 5xx 5%, CPU limit 85% 등 명확한 영향 기준은 이상징후(anomaly) 승격 기준으로 사용하였다.

b. 관련 Alert 를 하나의 장애 건으로 그룹화**시간 · Service · Pod · 의존성 기준 Alert 그룹화**

적용 파일: /operations-backend-ec2/app/incident_correlator.py

```
window = timedelta(minutes=config.time_window_minutes)
if alert.starts_at - group.last_seen_at > window:
    return set()
reasons = {"time_window"}
if alert.service in group_services:
    reasons.add("same_service")
elif alert.pod and alert.pod in group_pods:
    reasons.add("same_pod")
elif any(self._services_connected(alert.service, s, config)
         for s in group_services):
    reasons.add("service_dependency")
```

- 하나의 장애로 동시에 발생한 여러 Alert 를 시간대, 동일 Service, 동일 Pod, Service Dependency 기준으로 하나의 Incident(장애 조사 단위)에

연결하였다. 최초 Alert 는 조사 시작점이며 코드가 이를 근본 원인으로
단정하지 않는다.

4.5.5 장애 근거 데이터 기반 Bedrock 원인 분석 및 운영 자동화

a. 장애 원인 분석용 근거 데이터(Evidence) 구성

장애 건별 Metric·Log·Trace 등 근거 데이터 통합

적용 파일: /operations-backend-ec2/app/evidence_builder.py

```
return EvidencePackage(  
    incident=incident,  
    selection_window_start=window_start,  
    selection_window_end=window_end,  
    anomalies=selected,  
    alerts=incident.alerts,  
    logs=logs or [],  
    traces=traces or [],  
    kubernetes_events=kubernetes_events or [],  
    deployments=deployments or [],  
    unavailable_sources=...  
)
```

- Incident(장애 조사 단위)의 시간 범위를 기준으로 관련
이상징후·Alert·Log·Trace·Kubernetes Event·배포 변경을 하나의
Evidence Package(장애 분석 근거 묶음)로 구성하여 Bedrock RCA 에
전달되는 입력 근거를 동일하게 유지하였다.

b. Amazon Bedrock 기반 근본 원인 분석(RCA)

수집한 장애 근거 데이터 Bedrock RCA 초안 생성 + Guardrails

적용 파일: /operations-backend-ec2/app/rca_contract.py

```

client = boto3.client("bedrock-runtime", region_name=region_name)
converse_kwargs = {
    "modelId": model_id,
    "messages": [{
        "role": "user",
        "content": [{"text": format_evidence_for_prompt(evidence)}],
    }],
    "toolConfig": {
        "tools": [RCA_TOOL_SCHEMA],
        "toolChoice": {"tool": {"name": RCA_TOOL_NAME}},
    },
    "inferenceConfig": {"maxTokens": 2200, "temperature": 0.2},
}
if guardrail_id and guardrail_version:
    converse_kwargs["guardrailConfig"] = {
        "guardrailIdentifier": guardrail_id,
        "guardrailVersion": guardrail_version,
        "trace": "enabled",
    }
response = client.converse(**converse_kwargs)

```

- 이상징후와 Incident(장애 그룹화) 판정은 애플리케이션 코드가 수행하고, Bedrock 은 이미 선택된 장애 근거 데이터(Evidence)만 받아 원인 후보·확인 명령·권장 조치를 포함한 RCA(근본 원인 분석) 초안을 생성하도록 역할을 제한하였다. Guardrails 도 동일 호출에 적용하였다.

c. Slack 알림 및 일일 리포트

systemd timer - 매일 09:05 KST 실행

적용 파일: /infra/systemd/mp-daily-report.timer, mp-daily-report.service

```
[Timer]
OnCalendar=*-*-* 09:05:00 Asia/Seoul
Persistent=true
Unit=mp-daily-report.service
[Service]
Type=oneshot
WorkingDirectory=/opt/mealbong-app
ExecStart=/usr/bin/docker compose exec -T operations-api \
python -m app.daily_report
```

- Alertmanager 는 장애 발생(FIRING)·해결(RESOLVED) 상태를 실시간 Slack 으로 전송한다. 일일 리포트는 scrape/Webhook 지연을 흡수하기 위해 09:05 에 실행하되 집계 구간은 전일 09:00 부터 당일 09:00 까지로 고정하였다.

구현 결과 요약

단계	최종 결과
Docker	VM/Container Metric·Log·Trace 수집 구조를 구축하고 Grafana 에서 운영 상태를 조회
Kubernetes	Prometheus Operator 기반 Metric 수집, Alloy DaemonSet Log 수집, FastAPI OTel + Istio Mesh Trace 를 Tempo 에 통합
AWS EKS	동일 수집 구조를 EKS 로 이전하고 Loki/Tempo 저장소를 S3 로 전환, ServiceAccount 연계 IAM Role(IRSA) 적용
Operations EC2	Prometheus Metric 수집기와 이상징후 분석기(Analyzer)를 상시 실행하고 관련 Alert 그룹화, 장애 근거 데이터 수집, Bedrock RCA, Slack 자동화를 별도 운영 영역으로 분리

4.6 Serverless 구성도



[그림 4.6-1] Serverless 구성도

Serverless 구성도 설명

- 서울 리전에 AWS Lambda 로 AI 기능 10 종을 구성
- Lambda 는 전부 arm64(Graviton)로 빌드하여 동일 성능 대비 비용을 낮춤
- 컨테이너 이미지 대신 zip 번들로 배포하여 빌드와 레지스트리 단계를 없앴
- ALB 에 별도 도메인을 연결하고 ACM 인증서로 TLS 통신 구성
- 처리에 오래 걸리는 영상과 영수증은 Amazon SQS 를 사이에 두어 접수와 워커를 분리
- 실패한 메시지는 DLQ 로 보내 유실을 막고, DLQ 적체를 알람으로 감시
- 영수증 이미지는 Amazon S3 에 올리고 수명주기 규칙으로 자동 삭제
- 정기 실행이 필요한 배치는 Amazon EventBridge Scheduler 로 구성
- Amazon CloudWatch 를 이용하여 모니터링과 알람 구성
- Terraform 으로 서버리스 전용 스택을 별도 구성하고 상태 파일을 분리

- 기존 EKS 파드를 대체하지 않고 같은 데이터를 보는 별도 경로로 나란히 운영

4.7 Serverless 사용 기술

Lambda 함수 구성

AI 기능을 역할에 따라 접수, 워커, 동기, 배치 네 가지로 나누어 총 10 개의 함수로 구성했다.

구분	함수	트리거	역할
접수	mp-ai-video-api	ALB	영상 URL 을 접수하고 큐에 넣은 뒤 즉시 반환
접수	mp-ai-ocr-api	ALB	영수증 이미지를 접수하고 S3 와 큐에 전달
워커	mp-ai-video-worker	SQS	Gemini 로 영상에서 레시피를 추출
워커	mp-ai-ocr-worker	SQS	Vision 과 Vertex AI 로 영수증을 인식
동기	mp-ai-chat-api	ALB	챗 어시스턴트 응답을 즉시 반환
배치	mp-ai-price-detect	Scheduler	가격 이상을 탐지하고 알림을 발송
배치	mp-ai-sentiment-batch	Scheduler	Bedrock 으로 리뷰 감성을 분류
배치	mp-ai-summarize-batch	Scheduler	Bedrock 으로 리뷰를 요약
배치	mp-ai-shelflife-draft	수동	소비기한 초안을 생성
배치	mp-ai-ner-backfill	수동	재료 구조화 결과를 채워 넣음

함수 이름	런타임	아키텍처	메모리(MB)	제한시간(초)	패키지 크기
mp-ai-chat-api	python3.12	arm64	1024	30	12.6 MB
mp-ai-ner-backfill	python3.12	arm64	1024	900	6.4 MB
mp-ai-ocr-api	python3.12	arm64	512	10	0.3 MB
mp-ai-ocr-worker	python3.12	arm64	1024	120	21.8 MB
mp-ai-price-detect	python3.12	arm64	1024	900	5.1 MB
mp-ai-sentiment-batch	python3.12	arm64	512	900	5.1 MB
mp-ai-shelflife-draft	python3.12	arm64	512	900	5.1 MB
mp-ai-summarize-batch	python3.12	arm64	512	900	5.1 MB
mp-ai-video-api	python3.12	arm64	256	10	2.8 MB
mp-ai-video-worker	python3.12	arm64	1024	150	10.4 MB

기존 애플리케이션 재사용

챗 어시스턴트는 이미 FastAPI 로 작성되어 있어 ASGI 어댑터인 Mangum 으로 감싸 그대로 Lambda 에서 실행했다. 다만 ALB 는 경로 접두사를 떼어 주지 않으므로, 별도 경로로 붙이는 경우에는 함수 쪽에서 접두사를 제거해야 한다. 이것을 환경변수로 두어 접두사가 있는 구성과 없는 구성이 같은 코드로 동작하도록 했다.

ai_chat_api/handler.py (FastAPI 앱 재사용)

```

from __future__ import annotations
import os
import sys
from pathlib import Path

# 번들 루트 = import 루트. build.sh 가 `services/chat/app` 을 `app/` 로 통째로 넣는다.
# 레포에서 직접 돌릴 때(테스트)는 `services/chat` 도 봐야 `app.main` 이 잡힌다.
_HERE = Path(__file__).resolve()
for _p in (_HERE.parents[1], _HERE.parents[2] / "services" / "chat"):
    if _p.exists() and str(_p) not in sys.path:
        sys.path.insert(0, str(_p))
from mangum import Mangum # noqa: E402
from common.runtime import log_start, logger # noqa: E402
from common.secrets import inject # noqa: E402
FUNCTION = "mp-ai-chat-api"
log = logger(FUNCTION)

# `app.main` 보다 먼저 불러야 한다. 그 모듈이 import 시점에 Settings 를 읽고 PG 접속
# 정보를 굳히기 때문이다 — 뒤에 채우면 이미 늦다.
#
# 이게 빠져 있었다(2026-08-18 실배포에서 발견). 증상이 특히 나뉘었다 — `/health` 가
# 200 `degraded` 를 돌려줘서 스모크가 지나갔다. *`떴는데 자격증명이 없다`* 가
# *`떴다`* 로 읽힌 것이다. 배치 5 종만 이 줄을 갖고 있었고, 그래서 배치만 정직하게 죽었다.
# ⇒ 초록불이 «동작한다» 를 뜻하려면 실패할 수 있는 경로를 실제로 지나야 한다.
inject()
# import 시점에 앱을 만든다 — 의도다. Lambda 는 이 모듈 import 까지가 INIT 이고,
# 거기서 끝낼 수 있는 준비(라우트 등록·계측 부착)는 끝내야 warm 호출이 빨라진다.
from app.main import app # noqa: E402
_asgi = Mangum(app, lifespan="off")
# — ALB 는 경로를 «잘라주지 않는다» —————
#
# 리스너 규칙이 `/ai/api/mealplan/assistant/chat` 이면 Lambda 가 받는 `event["path"]` 도
# 그대로 `/ai/...` 다. 그런데 FastAPI 라우트는 `services/chat/app/main.py:581` 의
# `@app.post("/api/mealplan/assistant/chat")` 이다. ⇒ 404.
#
# 그리고 이걸 «규칙을 붙이는 순간» 처음 드러난다. 지금 스모크는 접두사 없는 경로로 부르므로
# 통과한다 — 즉 초록불이 또 거짓말을 한다. 오늘날 세 번째 같은 모양이다.
#
# Mangum 의 `api_gateway_base_path` 로는 안 된다. 실측(2026-08-18):
# `mangum/handlers/api_gateway.py` → 그 옵션을 쓴다
# `mangum/handlers/alb.py` → 언급 0 회. `event["path"]` 를 그대로 scope 에 넣는다
# ⇒ 옵션을 쥐도 조용히 무시된다. 어댑터에 맡길 수 없으니 여기서 직접 벗긴다.
#
# 접두사를 코드에 박지 않는다 — Terraform `alb_path_prefix` 와 같은 값이어야 하고,
# 두 군데 적으면 언젠가 갈린다. 환경변수로 받고, 비면 아무것도 안 한다(로컬·직접호출 경로).
_PREFIX = os.environ.get("ALB_PATH_PREFIX", "").rstrip("/")
def _strip_prefix(event: dict) -> dict:
    """`/ai/...` → `/.` . 접두사가 없거나 안 맞으면 손대지 않는다."""
    if not _PREFIX:
        return event
    path = event.get("path") or ""
    if not path.startswith(_PREFIX + "/") and path != _PREFIX:
        return event # 접두사 없는 직접 호출 — 그대로 통과
    # `/ai` 만 온 경우 빈 문자열이 되면 ASGI 가 깨진다. 루트로 만든다.
    return {**event, "path": path[len(_PREFIX):] or "/"}
def handler(event, context):
    """ALB 가 부른다. 요청 번역·응답 직렬화는 어댑터가 한다.
    ALB 이벤트인자의 판정은 Mangum 이 `requestContext.elb` 로 한다. 그 키가 없으면
    «핸들러를 못 고르겠다» 로 죽는다 — 콘솔 Test 버튼의 기본 payload 로는 재현이 안 된다.
    로컬 확인은 `serverless/tests/` 의 ALB 이벤트 픽처를 쓸 것.
    """
    event = _strip_prefix(event)
    log_start(log, FUNCTION,
        {"path": event.get("path"), "method": event.get("httpMethod")}, context)
    return _asgi(event, context)

```

SQS 큐 구성

aws-ai/sqs.tf (작업 큐와 DLQ)

```
# 접수 → 워커 큐. 형상은 `./aws/sqs.tf` (크롤 큐)를 그대로 따른다 — 이미 검증된 값이다.
#
# 이미지는 큐에 안 실린다. SQS 본문 상한 256KB < 영수증 사진(1600px JPEG, 200~500KB).
# 큐에는 좌표(`bucket`/`key`)만 흐르고 실물은 S3에 있다 — `docs/serverless/07_...$2`.
resource "aws_sqs_queue" "dlq" {
  for_each = toset(local.queue_names)
  name      = "mp-ai-${each.key}-jobs-dlq"
  message_retention_seconds = 1209600 # 14 일 — 사람이 월요일에 봐도 남아 있어야 한다
}
resource "aws_sqs_queue" "jobs" {
  for_each = toset(local.queue_names)
  name     = "mp-ai-${each.key}-jobs"
  # 가시성 타임아웃 ≥ 함수 타임아웃. 짧으면 워커가 아직 도는 중에 메시지가 다시 보이고
  # 같은 잡을 두 개가 동시에 처리한다(유료 모델 호출이 두 배). 여유를 두고 함수의 2 배.
  visibility_timeout_seconds = each.key == "video" ? 300 : 240
  message_retention_seconds = 345600 # 4 일
  receive_wait_time_seconds = 20    # long polling — 빈 폴링 요금을 줄인다
  redrive_policy = jsonencode({
    deadLetterTargetArn = aws_sqs_queue.dlq[each.key].arn
    # 이 값은 코드의 `MAX_RECEIVE_COUNT`와 같아야 한다(`common/jobs.py`).
    # 어긋나면 워커가 "마지막 시도"를 잘못 판단해서, 아직 재시도가 남았는데 FAILED로
    # 마감하거나(유저가 성공할 수 있었던 잡을 잃는다) 끝났는데 안 마감한다(영원히 PENDING).
    maxReceiveCount = 3
  })
}
```

S3 버킷 구성

aws-ai/s3.tf (영수증 버킷과 수명주기 규칙)

```

# 영수증 업로드 버킷. 개인정보다.
#
# 흐름 = 브라우저 → (인라인 또는 presigned) → 이 버킷 → 워커가 읽고 성공 즉시 삭제.
# 그래도 실패 잔여물이 남으므로 수명주기로 한 번 더 받는다. 결정 근거 = `docs/serverless/07_...`.
resource "aws_s3_bucket" "uploads" {
  bucket = var.upload_bucket_name
}
resource "aws_s3_bucket_public_access_block" "uploads" {
  bucket = aws_s3_bucket.uploads.id
  block_public_acls = true
  block_public_policy = true
  ignore_public_acls = true
  restrict_public_buckets = true
}
resource "aws_s3_bucket_server_side_encryption_configuration" "uploads" {
  bucket = aws_s3_bucket.uploads.id
  rule {
    apply_server_side_encryption_by_default { sse_algorithm = "AES256" }
  }
}
resource "aws_s3_bucket_lifecycle_configuration" "uploads" {
  bucket = aws_s3_bucket.uploads.id
  rule {
    id = "expire-receipts"
    status = "Enabled"
    filter { prefix = "receipts/" }
    # 1 일. 워커가 성공하면 즉시 지우므로 여기 남는 것은 실패했거나 접수만 되고 버려진 것들이다.
    # 보관할 이유가 없고, 영수증은 이름·품목·금액이 다 들어 있다.
    expiration { days = 1 }
    # 멀티파트 찌꺼기도 같이 — 안 지우면 «보이지 않는 채로» 과금된다.
    abort_incomplete_multipart_upload { days_after_initiation = 1 }
  }
}

```

IAM 권한 설계

Lambda 실행 역할에 **Permissions Boundary** 를 붙여, 정해진 이름 범위 밖으로는 어떤 정책을 붙여도 효력이 없도록 했다. 경계는 권한의 천장이지 허가가 아니므로, 역할에 인라인 정책이 없으면 여전히 아무것도 하지 못한다. 실제로 배치 역할에 S3 권한이 빠져 있어 모델 파일을 읽지 못하고 403 으로 실패한 적이 있다.

aws-ai/iam.tf (실행 역할과 Permissions Boundary)

```

# 실행 역할 — AI 파트가 직접 만든다. 우회가 아니라 설계된 경로다.
#
# 권한요청서를 쓸 때는 "인프라가 역할을 만들고 AI 는 PassRole 만 받는다"(A 안)를 전제했는데,
# 실측해 보니 이미 그럴 필요가 없다(2026-08-17 `simulate-principal-policy`):
#
# iam:CreateRole (iam:PermissionsBoundary = mp-ai-boundary 조건) allowed
# iam:PutRolePolicy · iam:AttachRolePolicy · iam:TagRole allowed
# iam:PassRole (iam:PassedToService = lambda.amazonaws.com) allowed
#
# 그리고 경계(`infra/iam/mp-ai/mp-ai-boundary.json`)에 `ec2:CreateNetworkInterface` 가
# 이미 들어 있다. 이걸 VPC Lambda 를 전제하고 설계했다**는 뜻이다. 즉 우리가 역할을
# 만드는 것은 «권한을 비집고 들어가는 것» 이 아니라 그 설계가 의도한 사용법이다.
# (`docs/mp_aws_team_access.md §4` — "경계가 붙으면 실효 권한 = 경계 n 정책 이라 천장을
# 못 넘고, 그 안에서는 자유롭다")
#
# 경계 밖은 적어도 무효다. 아래 인라인 정책은 전부 경계 안에서만 쓴다:
# RuntimeCommon logs · bedrock · ecr · ec2 네트워크인터페이스 (Resource *)
# RuntimeOwnResources s3:* · sqs:* · lambda:InvokeFunction · secretsmanager:GetSecretValue
# → `mp-ai` / `mp-ai` 안에서만**
# 그래서 `mp/prod/*` 시크릿은 이 역할들이 못 읽는다. 배치 5 종이 PG 비밀번호를 쓰려면
# `mp-ai/*` 로 따로 넣어야 한다(관리자 몫이 아니라 우리 몫이다 — dev 정책이
# `secretsmanager:*` 를 `secret:mp-ai/*` 에 준다).
locals {
  create_roles = length(var.exec_role_arns) == 0
  # 만들면 이 이름, 받으면 넘겨받은 ARN. 아래 lambda.tf 는 이것만 본다.
  role_arns = local.create_roles ? {
    for k, r in aws_iam_role.exec : k => r.arn
  } : var.exec_role_arns
  boundary_arn = "arn:aws:iam::data.aws_caller_identity.me.account_id:policy/{var.boundary_policy_name}"
  role_names = ["batch", "api", "worker", "scheduler"]
}
data "aws_caller_identity" "me" {}
data "aws_iam_policy_document" "assume" {
  for_each = local.create_roles ? toset(local.role_names) : toset([])
  statement {
    actions = ["sts:AssumeRole"]
    principals {
      type = "Service"
      # 스케줄러 역할만 주체가 다르다 — Lambda 가 맡는 역할이 아니라
      # 스케줄러가 맡아서 Lambda 를 부르는 역할이다.
      identifiers = [each.key == "scheduler" ? "scheduler.amazonaws.com" : "lambda.amazonaws.com"]
    }
  }
}
resource "aws_iam_role" "exec" {
  for_each = local.create_roles ? toset(local.role_names) : toset([])
  name = "mp-ai-${each.key}"
  assume_role_policy = data.aws_iam_policy_document.assume[each.key].json
  # 이게 빠지면 `iam:CreateRole` 자체가 거부된다. dev 정책이 경계 조건부로 허용한다 —
  # 없으면 `AdministratorAccess` 붙인 역할을 만들어 Lambda 에 넘기는** 권한 상승이 되기 때문이다.
  permissions_boundary = local.boundary_arn
  tags = { Project = "mp-ai" }
}
# — 공통: 로그 + VPC ENI —————
# VPC Lambda 는 ENI 3 종이 없으면 함수가 아예 안 뜬다(INIT 전에 실패한다).
# 관리형 정책 `AWSLambdaVPCLambdaAccessExecutionRole` 과 같은 내용을 인라인으로 둔다 —
# 경계 안에 있는 것만 쓴다는 것을 눈으로 확인할 수 있게.
data "aws_iam_policy_document" "common" {
  statement {
    actions = ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"]
    resources = ["arn:aws:logs:*:${data.aws_caller_identity.me.account_id}:log-group:/aws/lambda/mp-ai-*"]
  }
  statement {
    actions = ["ec2:CreateNetworkInterface", "ec2:DescribeNetworkInterfaces",
      "ec2:DeleteNetworkInterface"]
    # ENI 는 리소스 수준 제한이 성립하지 않는다(만들기 전엔 ARN 이 없다). 경계가 천장이다.
    resources = ["*"]
  }
}

```

데이터 티어 접근

Lambda 가 EKS 안의 PostgreSQL 과 Elasticsearch 에 접근해야 하는데, 그 주소는 클러스터 내부 DNS 라 VPC 쪽에서는 이름으로 찾을 수 없다. 내부 로드밸런서를 두는 방안을 검토했으나 비용 대비 이득이 없어 NodePort 로 노출하고 노드 사설 IP 로 접근하도록 했다.

방안	월 비용	채택 여부
내부 NLB 2 개	약 32 달러	미채택
NodePort 와 노드 사설 IP	0 달러	채택

`externalTrafficPolicy` 를 Local 로 두면 출발지 IP 가 보존되므로, `NetworkPolicy` 로 Lambda 가 쓰는 대역만 허용하도록 좁힐 수 있다. 다만 주소가 특정 노드에 매이므로 노드가 교체되면 값을 다시 설정해야 한다. 되돌리는 경로로 로드밸런서 승격을 열어 두었고, 포트를 변수로 빼 두어 그때에도 애플리케이션 코드는 바뀌지 않는다.

`netpol-lambda.yaml` (Lambda 대역만 허용하는 `NetworkPolicy`)

AI Lambda → PG-ES NetworkPolicy — 아직 `resources` 에 넣지 않았다. 의도다.

#

== 왜 지금 안 켜나 ==

#

허용 대상이 노드 서브넷 `/20` 두 개다. Cilium 이 ENI 모드라 **클러스터의 모든 파드가
그 안에** 있으므로, 이걸 켜는 순간 `pipeline`·`observability`·`cost`·`kube-system` 파드까지
전부 pooler 5432 에 L3 로 닿는다. tier-3 잠금(`netpol-pg.yaml` 이 `app` ns `tier: backend`
로 좁혀 둔 것)이 그만큼 넓어진다.

#

쓰는 함수가 없는 동안 그 넓힘이 존재할 이유가 없다.

⇒ `kustomization.yaml` 의 `resources` 에 이 파일을 함수 배포와 같은 시점에 한 줄 넣는다.

되돌리기도 그 한 줄이다.

#

== 왜 `/20` 밖에 못 쓰나 ==

#

Lambda ENI 는 우리가 지정한 서브넷에서 IP 를 자동 할당받는다 — 고정할 수단이 없다.

그리고 노드 서브넷을 써야만 NAT-VPC 엔드포인트 경로가 산다(테이터 서브넷은 «밖으로 나가는

경로 없음» 이 산출물이다). ⇒ 지금 표현 가능한 가장 좁은 범위가 이 두 `/20` 이다.

#

좁히는 길 = 전용 Lambda 서브넷(예: `/24`). 그러면 ipBlock 이 그 하나로 줄고 파드는

빠진다. 다만 `ec2:CreateSubnet` 은 guardrails 의 `DenyTouchingMigrationInfra` 대상이라

인프라 담당 몫이고, 라우트테이블·NAT 연결까지 따라온다. 노드 SG 인그레스를 부탁드릴 때

같이 검토하면 좋겠다.

#

그리고 이게 유일한 방어선은 아니다 — PG 는 `svc\_\*` 를 비밀번호를 요구하고(pgbouncer

`auth\_query`), ES 도 인증이 켜져 있다. 네트워크는 한 겹이지 전부가 아니다.

#

== 출발지가 Lambda ENI IP 인 근거 ==

#

`lambda-access.yaml` 의 두 Service 가 `externalTrafficPolicy: Local` 이다. 그 모드는

출발지 IP 보존을 보장한다(SNAT 없음). 그래서 파드가 보는 주소가 노드 IP 가 아니라

Lambda ENI IP 이고, 아래 CIDR 이 그것을 덮는다.

`Cluster` 로 되돌리면 이 전제가 깨진다 — 그때는 노드 IP `/32` 2 개로 바꿔야 한다.

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: mp-pg-pooler-lambda

namespace: data

labels: { app.kubernetes.io/part-of: mp-ai-lambda-access }

spec:

pooler 만이다. PG 인스턴스(`cnpg.io/podRole: instance`)는 열지 않는다 —

Lambda 는 pooler 를 지나야 하고, 그래야 커넥션 산수(overlays/eks 의 풀 계산)가 성립한다.

podSelector:

matchLabels:

cnpg.io/poolerName: pg-pooler

policyTypes: [Ingress]

ingress:

- from:

- ipBlock: { cidr: 10.10.64.0/20 } # mp-subnet-node-ap-northeast-2a

- ipBlock: { cidr: 10.10.80.0/20 } # mp-subnet-node-ap-northeast-2b

ports:

- { protocol: TCP, port: 5432 } # 포트를 반드시 못박는다 — 열린 건 이 하나뿐이다

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: mp-es-lambda

namespace: data

labels: { app.kubernetes.io/part-of: mp-ai-lambda-access }

spec:

podSelector:

matchLabels:

elasticsearch.k8s.elastic.co/cluster-name: es

policyTypes: [Ingress]

ingress:

- from:

- ipBlock: { cidr: 10.10.64.0/20 }

모니터링과 알람

파드는 재시작을 반복하면 상태가 눈에 띄지만, Lambda 는 실패해도 겉으로는 아무 일도 일어나지 않는다. 그래서 알람 기준을 한 건이라도 실패하면 울리도록 잡았다. 배치는 하루에 한 번만 돌기 때문에 평균을 기준으로 하면 한 건의 실패가 통계에 묻힌다.

알람 종류	개수	기준
함수 오류	10	함수별로 오류가 1 건이라도 발생하면 알림
DLQ 적체	2	처리하지 못한 메시지가 쌓이면 알림
큐 정체	2	메시지가 오래 처리되지 않으면 알림

alarms.py (CloudWatch 알람 생성)

```
#!/usr/bin/env python3
"""mp-ai 알람 — 생성/갱신 (떡등). `serverless/alarms.sh` 의 대체물이다.
## 왜 셸이 아니라 파이썬인가
`alarms.sh` 는 이 환경에서 한 번도 돌지 못한다. aws CLI 2.31.35 + Python 3.14 조합에서
`cloudwatch put-metric-alarm` 이 인자 파싱 전에 죽는다:
  ValueError: unsupported format character '.' (0x3a) at index 696
  ValueError: badly formed help string
CLI 가 도움말 문자열을 `%` 포맷으로 만들다 터지는 클라이언트 버그다. 권한도 인자도 문제가
아니라서, 에러 메시지가 원인을 전혀 안 가리킨다.
그리고 이걸 몰라서 알람이 도는 줄 알았다. 기존 알람 7 개는 Terraform 시절 산물이었고,
스크립트로 옮긴 뒤로는 실행이 성공한 적이 없다. 2026-08-18 에 함수가 3→10 으로 늘었는데
새 7 중에 알람이 안 붙는 것을 보고서야 드러났다.
⇒ `*스크립트가 있다*` 와 `*스크립트가 돈다*` 는 다르다. 오늘 같은 부류를 여러 번 봤다.
## 왜 Terraform 이 아닌가
`cloudwatch:ListTagsForResource` 가 없어서 프로바이더 refresh 가 죽고, 스택 전체가
못 쓰게 된다. 권한요청 ②가 오면 Terraform 으로 되돌린다.
사용: ALERT_EMAILS="a@b.com" serverless/scripts/alarms.py
serverless/scripts/alarms.py --dry-run
"""

from __future__ import annotations
import os
import sys
import boto3
REGION = os.environ.get("AWS_REGION", "ap-northeast-2")
DRY = "--dry-run" in sys.argv
sts = boto3.client("sts", region_name=REGION)
ACCOUNT = sts.get_caller_identity()["Account"]
TOPIC = f"arn:aws:sns:{REGION}:{ACCOUNT}:mp-ai-alerts"
cw = boto3.client("cloudwatch", region_name=REGION)
lam = boto3.client("lambda", region_name=REGION)
sns = boto3.client("sns", region_name=REGION)
def put(**kw):
    kw.setdefault("ActionsEnabled", True)
    kw["AlarmActions"] = [TOPIC]
    kw["OKActions"] = [TOPIC]
    # 호출이 없는 구간은 정상이다. 안 그러면 배치가 안 도는 23 시간 내내 INSUFFICIENT_DATA
    # 로 울려서 진짜 실패가 왔을 때 아무도 안 본다(알람 피로).
    kw["TreatMissingData"] = "notBreaching"
    if DRY:
        print(f" [dry-run] {kw['AlarmName']}")
        return
    cw.put_metric_alarm(**kw)
    print(f" {kw['AlarmName']}")
if os.environ.get("ALERT_EMAILS"):
    print("구독 등록")
    for m in [x.strip() for x in os.environ["ALERT_EMAILS"].split(",") if x.strip()]:
        if not DRY:
            sns.subscribe(TopicArn=TOPIC, Protocol="email", Endpoint=m)
            print(f" {m} 확인 메일을 눌러야 활성화된다(그전엔 PendingConfirmation)")
# — ① 함수가 죽는다 —————
# 임계값이 1 이다 — «몇 건 이상» 이 아니라 «한 건이라도». 배치는 하루 1 회라 평균을 내면
# 1 건 실패가 0.0x 로 묻힌다.
print("① 함수 오류")
names = []
for page in lam.get_paginator("list_functions").paginate():
    names += [f["FunctionName"] for f in page["Functions"]
               if f["FunctionName"].startswith("mp-ai")]
for fn in sorted(names):
    put(AlarmName=f"{fn}-errors",
        AlarmDescription=f"{fn} raised errors. Check CloudWatch Logs for the stack trace.",
        Namespace="AWS/Lambda", MetricName="Errors",
        Dimensions=[{"Name": "FunctionName", "Value": fn}],
        Statistic="Sum", Period=300, EvaluationPeriods=1,
        Threshold=1, ComparisonOperator="GreaterThanOrEqualToThreshold")
for q in ("video", "ocr"):
    # — ② 잡이 죽어서 쌓인다 —————
    # 워커가 실패해도 집수는 계속 202 를 준다. 유저는 «접수됐다» 를 보고 기다리는데
    # 아무도 처리하지 않는 상태가 된다 — DLQ 가 그 유일한 신호다.
    :('②', 'DLQ')
```

CloudWatch, 경보, 모든 경보

경보 (14) 정상 14 · 경보 0

경보 이름	상태	지표	임계값	평가 기간
mp-ai-chat-api-errors	● OK	Errors	1	1
mp-ai-ner-backfill-errors	● OK	Errors	1	1
mp-ai-ocr-api-errors	● OK	Errors	1	1
mp-ai-ocr-dlq-not-empty	● OK	ApproximateNumberOfMessagesVisible	0	1
mp-ai-ocr-queue-stalled	● OK	ApproximateAgeOfOldestMessage	900	2
mp-ai-ocr-worker-errors	● OK	Errors	1	1
mp-ai-price-detect-errors	● OK	Errors	1	1
mp-ai-sentiment-batch-errors	● OK	Errors	1	1
mp-ai-shelflife-draft-errors	● OK	Errors	1	1
mp-ai-summarize-batch-errors	● OK	Errors	1	1
mp-ai-video-api-errors	● OK	Errors	1	1
mp-ai-video-dlq-not-empty	● OK	ApproximateNumberOfMessagesVisible	0	1
mp-ai-video-queue-stalled	● OK	ApproximateAgeOfOldestMessage	900	2
mp-ai-video-worker-errors	● OK	Errors	1	1

배포 검증

Lambda 의 실패는 대부분 첫 호출에서만 드러난다. 배포는 성공하고 콘솔에도 함수가 정상으로 보이는데 부르면 죽는다. 그래서 배포 후 모든 함수를 실제로 한 번씩 호출하는 검증 스크립트를 작성했다.

검증 항목	그냥은 보이지 않는 이유
함수 내부 오류	함수가 죽어도 호출 자체는 200 으로 끝난다
응답 형식 오류	ALB 는 502 를 주는데 함수는 성공으로 기록된다
아키텍처 불일치	번들이 arm64 인데 함수가 x86 이면 첫 호출에서 실패한다
미배포 함수	실패와 구분해서 세지 않으면 진단이 흐려진다

처음 배포한 함수들이 이 검증을 통과했는데, 통과한 이유가 비밀 값 주입을 시도조차 하지 않아서였다. 설정이 빠져 있어 어댑터가 조용히 아무것도 하지 않았고, 비밀을 부르지 않으니 실패할 일도 없었다. 검증이 통과를 뜻하려면 실패할 수 있는 경로를 실제로 지나야 한다.

smoke.py (함수 오류와 응답 형식을 판정하는 검증 코드)

```

        "path": "/api/mealplan/assistant/없는경로", "expect": 404,
        "prefix": True},
    "video-worker": {"kind": "sqs"},
    "ocr-worker": {"kind": "sqs"},
}

def alb_event(method: str, path: str) -> dict:
    """`requestContext.elb`가 반드시 있어야 한다 — Mangum 과 우리 `common/alb.py`가
    둘 다 그 키로 ALB를 판정한다. 콘솔 Test 버튼의 기본 payload로는 재현이 안 된다."""
    return {
        "requestContext": {"elb": {"targetGroupArn": "arn:aws:elasticloadbalancing:smoke"}},
        "httpMethod": method, "path": path, "queryStringParameters": {},
        "headers": {"host": "smoke.local", "user-agent": "mp-smoke/1"},
        "body": "", "isBase64Encoded": False,
    }

_prefix_cache: dict[str, str] = {}
def _alb_prefix(client, name: str) -> str:
    """그 함수에 실제로 걸린 `ALB_PATH_PREFIX`. 스모크에 상수로 적지 않는다 —
    Terraform 값과 같으면 «검사는 통과하는데 실패는 404»가 된다(그게 이 검사의 목적이다)."""
    if name not in _prefix_cache:
        try:
            cfg = client.get_function_configuration(FunctionName=PREFIX + name)
            _prefix_cache[name] = (cfg.get("Environment") or {}).get(
                "Variables", {}).get("ALB_PATH_PREFIX", "").rstrip("/")
        except Exception:
            # noqa: BLE001
            _prefix_cache[name] = ""
    return _prefix_cache[name]

def invoke(client, name: str, payload: dict) -> tuple[bool, str]:
    try:
        r = client.invoke(FunctionName=PREFIX + name, InvocationType="RequestResponse",
            Payload=json.dumps(payload).encode())
    except Exception as exc:
        # noqa: BLE001
        return False, f"{type(exc).__name__}: {str(exc)[:120]}"
    body = r["Payload"].read().decode("utf-8", "replace")
    # `FunctionError`를 안 보면 예외를 성공으로 읽는다. Lambda는 함수가 죽어도
    # HTTP 200을 준다 — 실패는 이 헤더에만 있다. 우리가 오늘 쫓은 «조용한 실패»와 같은 모양이다.
    if r.get("FunctionError"):
        try:
            err = json.loads(body)
            return False, f'{err.get("errorType")}: {str(err.get("errorMessage"))[:140]}'
        except json.JSONDecodeError:
            return False, body[:160]
    return True, body

def check(client, name: str, spec: dict, write: bool) -> tuple[str, str]:
    kind = spec["kind"]
    # `chat-api#404`처럼 한 함수를 두 번 두드리는 항목을 위해, 표시 이름과 함수 이름을 가른다.
    name = spec.get("fn", name)
    if kind == "sqs":
        # 직접 부르지 않는다(머리말 참조). 배선만 확인한다.
        try:
            cfg = client.get_function(FunctionName=PREFIX + name)
            arch = ", ".join(cfg["Configuration"].get("Architectures", []))
            # arm64가 아니면 첫 호출에서 invalid ELF header로 죽는다(C-29·번들이 aarch64월).
            ok = arch == "arm64"
            return (" " if ok else ""), f'배선만 확인 · arch={arch or "?"}'
        except Exception as exc:
            # noqa: BLE001
            return "", f'{type(exc).__name__}: {str(exc)[:120]}'
    if kind == "batch":
        payload = dict(spec["payload"])
        if write:
            payload["apply"] = True
        ok, body = invoke(client, name, payload)
        if not ok:
            return "", body
        # «미리보기였나 실제였나»를 결과에 남긴다 — CloudTrail은 payload를 기록하지 않는다.
        return "", f'apply={payload["apply"]} · {body[:110]}'
    # alb
    # 접두사를 스모크가 직접 안다. ALB가 보내는 경로를 그대로 재현해야 하고,
    # 그 값은 Terraform `alb_path_prefix`와 같아야 한다 — 그래서 함수 환경변수에서 읽는다.
    # (여기 상수로 적으면 Terraform과 같리고, 걸린 순간 이 검사가 무의미해진다.)

```

```
mp@mealbong:~/food-budget-app$ python serverless/smoke.py --region ap-northeast-2
```

```
▶ 스모크 · region=ap-northeast-2 · write=False
```

✓ shelve-life-draft	1800ms	apply=False · {"made": 1, "skipped": 0, "processed": 1, "targets": 1}
✓ ner-backfill	1330ms	apply=False · {"ingredients": 0, "matched": 0, "match_rate": 0.0}
✓ price-detect	1617ms	apply=False · {"series": 534, "scanned": 14100, "matched": 9}
✓ sentiment-batch	1430ms	apply=False · {"scored": 0, "failed": 0, "pending": 0, "calls": 0}
✓ summarize-batch	1157ms	apply=False · {"summarized": 0, "skipped": 0, "processed": 0}
✓ video-api	2587ms	statusCode=404 · {"detail": "결과를 찾을 수 없어요(만료되었거나 잘못된 요청)."}
✓ ocr-api	1533ms	statusCode=404 · {"detail": "알 수 없는 job_id"}
✓ chat-api	3485ms	statusCode=200 · {"status": "degraded"}
✓ chat-api#404	34ms	statusCode=404 · {"detail": "Not Found"}
✓ video-worker	115ms	배선만 확인 · arch=arm64
✓ ocr-worker	97ms	배선만 확인 · arch=arm64

```
✓ 배포된 11종 중 실패 0
```

```
mp@mealbong:~/food-budget-app$
```

4.8 Backup

백업 설계의 출발점은 '무엇을 잃으면 되살릴 수 없는가' 였다. 데이터 티어를 전부 같은 비중으로 백업하지 않고, 재생성 가능 여부로 나눴다.

(1) 백업 대상 판정

대상	재생성 가능?	백업 여부	근거
PostgreSQL	불가, 유일한 원본	연속 WAL + 정기 base	사용자 데이터 · 가격 이력의 원본
Elasticsearch	가능	백업하지 않음	PostgreSQL 에서 재색인, 실측 1.0 초
Redis	가능	백업하지 않음	캐시 · 비영속. 비운 채로 시작해도 된다
Kafka (온프레임)	가능	백업하지 않음	드레인 후 재수집
etcd (온프레임)	불가	스냅샷 매일 S3	클러스터 상태 · Secret 포함
소스코드	가능(원격)	GitLab S3	정본이 GitLab 이므로 그 자체를 백업
릴리스 이미지	재빌드 가능	ECR 수명주기	다만 재빌드에 시간이 든다

이 표가 백업 전략의 전부다

되살릴 수 없는 데이터는 PostgreSQL 하나뿐이다.

그래서 백업 자원과 복구 훈련을 여기에 집중하고, 나머지는 재생성 절차만 문서화했다.

파생 데이터를 같이 백업하면 비용과 복구 시간이 늘지만 얻는 것은 없다.

(2) PostgreSQL 백업, barman-cloud S3

연속 WAL 아카이빙, 매 변경을 계속 S3 로 보낸다. RPO 를 결정한다
 정기 base backup, 매일 03:00 KST 전체 스냅샷. 복구 앵커로 RTO 를
 결정한다

**둘 다 있어야 성립한다, base 가 오래되면 복구 시 WAL 재생량이 늘어
 RTO 가 다시 나빠진다**

ScheduledBackup, 정기 base

```
apiVersion: postgresql.cnpg.io/v1
kind: ScheduledBackup
metadata:
  name: mp-pg-daily
  namespace: data
spec:
  schedule: "0 0 3 * * *" # 매일 03:00 KST
  backupOwnerReference: self
  cluster: { name: pg }
  method: plugin
  pluginConfiguration: { name: barman-cloud.cloudnative-pg.io }
```

검증

```
$ kubectl -n data get backup
$ kubectl -n data get cluster pg -o jsonpath='{.status.lastSuccessfulBackup}'
$ aws s3 ls s3://mp-backup-ap2/pg/ --recursive | tail
```

(3) RPO / RTO 목표

대상	RPO	RTO	근거
PostgreSQL (파드-노드 장애)	약 0	초 ~ 1 분	CloudNativePG 자동 페일오버
PostgreSQL (전체 손실)	약 5 분	10 분 미만	archive_timeout 302 초 · S3 복원 실측 promote 4 초
Elasticsearch	해당없음	분 단위	재색인
Redis	해당없음	즉시	재생성
etcd (온프레임)	24 시간	약 30 분	스냅샷 간격 · 단일 멤버
사이트 전환 (DR)	5 분	분 단위	Warm Standby · S3 WAL 경유

			복제
--	--	--	----

(4) DR 구성, 온프레미스를 Warm Standby 로

AWS = Active / 온프레미스 = Warm Standby (DR 4 등급 중 3 등급, 전체 스택이 돌고 트래픽만 안 받는다)

복제 경로는 S3 WAL 아카이브 경유, 두 사이트 사이에 상시 터널을 두지 않는다

전환은 2 단으로 나눈다, (1) 읽기 재개(가역) (2) 쓰기 개방(PG 승격 · 편도)
온프레미스는 DR 전용이 아니다, 크롤 CronJob 7 종의 상시 프로덕션으로 계속 돈다(이중역할)

설계 근거, 왜 터널 직결이 아니라 S3 경유인가

터널로 직접 스트리밍 복제를 하면 프로덕션 primary 에 원격용 복제 슬롯이 생긴다.

그러면 DR 사이트가 끊겼을 때 primary 가 WAL 을 보관하며 기다리고, 결국 프로덕션 디스크를 채운다.

즉 DR 사이트의 장애가 프로덕션 장애로 번진다.

S3 경유는 그 결합이 없다, 아카이브는 일방향이고, 받는 쪽이 죽어도 보내는 쪽은 영향이 없다.

대가는 RPO 가 초 단위가 아니라 분 단위라는 것이고, 5 분은 우리 서비스에서 감당할 수 있다.

백업 감사에서 드러난 것, '설정됨' 과 '동작함' 은 다르다

전 트랙을 감사한 결과 다음이 실제로는 돌지 않고 있었다 :

릴리스 이미지 백업, 전량 실패 (credential username 자리에 안내 문구가 들어가 있었다)

비밀 묶음 백업, 정지 상태

kubeadm PKI, 백업 자체가 없었다

세 건 모두 '설정은 되어 있었다'. 알람이 없어서 실패를 아무도 몰랐다.

조치 : 백업 신선도 감시 롤(backup_freshness)을 신설해 마지막 성공 시각을 지표로

내보내고,
임계를 넘으면 Slack 으로 알리게 했다.

4.9 Security

보안은 한 겹으로 막지 않고 계층마다 통제를 둔다(심층방어). 네트워크 · 신원 · 파드 · 비밀 · 탐지 다섯 축으로 구성했다.

계층	통제 수단	막는 것
네트워크 (인프라)	Security Group 7 종 · 서브넷 라우팅	VPC 안팎의 도달 자체
네트워크 (파드)	NetworkPolicy 26 · CiliumNetworkPolicy 29	파드 간 · 외부로 나가는 통신
전송	Istio mTLS STRICT · ACM(TLS 종단)	도청 · 신원 위조
신원 (사람)	IAM 그룹 EKS Access Entry RBAC	권한 초과 · 회수 불가
신원 (워크로드)	IRSA (OIDC)	파드가 남의 AWS 자원을 만지는 것
파드	Pod Security Admission · SecurityContext	특권 상승
비밀	Secrets Manager + ESO	평문 노출 · 커밋 유출
탐지	GuardDuty · CloudTrail · WAF	이상 행위 · 웹 공격

4.9.1 IAM · IRSA, 신원과 권한

사람과 워크로드의 신원을 분리한다. 사람은 IAM 그룹에서 EKS Access Entry 로 파생되고, 워크로드는 IRSA 로 서비스 어카운트에 롤을 붙인다.

IAM 그룹	멤버	용도	부여 권한
mealplanning-dev	4	EKS 접근 · SSM 세션 ·	eks:DescribeCluster ·

		조회	eks:ListClusters 등 (커스텀 mp-team-dev)
mealplanning-ai	1	AI 독립 트랙 (서버리스)	mp-ai-dev + 전용 정책 3 종, mp-ai-* 리소스 밖은 거부
mealplanning- dashboard	2	FinOps · 운영 대시보드	대시보드 EC2 와 조회 대상에 한정

사람이 클러스터에 붙으려면 서로 다른 층 세 개가 전부 있어야 한다

(1) IAM 자격증명 aws sts get-caller-identity 로 확인

(2) IAM 권한 eks:DescribeCluster aws eks update-kubeconfig 로 확인

(3) EKS Access Entry kubectl get pods 로 확인

진단이 어려운 이유 : 세 층의 실패가 전부 '키가 잘못됐다' 로 읽힌다.

특히 (2)가 없으면 kubeconfig 파일 자체가 생성되지 않고(AccessDeniedException),

(3)만 없으면 파일은 생기는데 Unauthorized 가 난다.

권한의 실체는 IAM 이 아니라 Kubernetes RBAC 다, mp:admin 이 무엇을 할 수 있는지는

Ansible eks_rbac 룰이 만드는 ClusterRole 이 정한다. Access Entry 만 있고 RBAC 이 없으면 권한은 0 이다.

eks_access.tf, Access Entry

```
resource "aws_eks_access_entry" "team" {
  for_each = toset(var.team_users)
  cluster_name = aws_eks_cluster.mp.name
  principal_arn = "arn:aws:iam::${local.account_id}:user/${each.key}"
  kubernetes_groups = ["mp:admin"] # 실체는 우리가 정의한 ClusterRole
  type = "STANDARD"
}
```

iam_irsa.tf, 워크로드 신원 (13 종)

```
resource "aws_iam_role" "irsa" {
  for_each = local.irsa_roles # 13 종
  assume_role_policy = jsonencode({
    Statement = [{
      Effect = "Allow"
      Principal = { Federated = aws_iam_openid_connect_provider.eks.arn }
      Action = "sts:AssumeRoleWithWebIdentity"
      Condition = { StringEquals = {
        "${local.oidc}:sub" = "system:serviceaccount:${each.value.ns}:${each.value.sa}"
        "${local.oidc}:aud" = "sts.amazonaws.com"
      }}
    }]
  })
}
```

EKS Pod Identity 를 쓰지 않은 이유

Pod Identity 는 배선이 간단하지만, 제한 조건을 신뢰정책 안에 직접 쓸 수 없다.

IRSA 는 sub 에 네임스페이스와 서비스 어카운트를 정확히 못박을 수 있어,

'이 파드만 이 롤을 쓸 수 있다' 가 IAM 쪽에 남는다.

4.9.2 Security Group

SG 7 종으로 계층별 도달 범위를 제한한다. NACL 은 채택하지 않았다.

Security Group	적용 대상	인바운드
mp-sg-alb-public	ALB ENI	443 (인터넷) · 80 443 리다이렉트
mp-sg-eks-node	EKS 노드 · 파드 ENI	ALB SG 로부터 · 노드 간 · 컨트롤플레인
mp-sg-elasticache	ElastiCache	노드 SG 로부터 6379 만
mp-sg-vpce	Interface Endpoint ENI	노드 SG 로부터 443
mp-sg-ci	CI 서버 EC2	SSH · HTTPS (제한된 소스)
mp-sg-dashboard	대시보드 EC2	443

NACL 미채택 근거

Security Group 은 상태 유지(stateful)라 반환 트래픽이 자동으로 허용된다.

NACL 은 무상태(stateless)라 반환 트래픽 규칙을 사람이 직접 관리해야 한다.

우리 규모에서 NACL 을 더하면 통제가 늘어나는 게 아니라 실수할 표면이 늘어난다.

이미 SG 로 도달을 제한하고, 그 안쪽은 NetworkPolicy 가 다시 제한한다.

4.9.3 NetworkPolicy · mTLS

클러스터 내부는 default-deny 를 기본으로 하고 필요한 경로만 연다. 표준 NetworkPolicy 로 파드 간 통신을, CiliumNetworkPolicy 로 외부 도메인 egress 를 제어한다.

app 네임스페이스 default-deny

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: mp-default-deny
  namespace: app
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress] # 양방향 — Egress 만 빼면 나가는 건 다 열린다
```

CiliumNetworkPolicy, FQDN 기반 egress

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: mp-allow-s3-fqdn
  namespace: observability
spec:
  endpointSelector: { matchLabels: {app: loki} }
  egress:
    - toFQDNs:
        - matchName: "s3.ap-northeast-2.amazonaws.com"
        - matchPattern: "*.s3.ap-northeast-2.amazonaws.com" # 가상호스트 형식
    toPorts: [{ ports: [{ port: "443", protocol: TCP }] }]
```

검증

```
$ kubectl -n app get networkpolicy
$ kubectl -n app run t --rm -it --image=curlimages/curl --restart=Never -- \
curl -m 3 http://recipe.app:8002/health # 차단 확인
```

실측 사고, S3 가상호스트 이름 누락으로 Loki · Tempo 가 죽었다

toFQDNs 에 s3.ap-northeast-2.amazonaws.com 만 넣고 <버킷>.s3.... 형식을 빠뜨렸다.

SDK 기본 주소 지정 방식이 가상호스트라 실제 요청이 막혔다.

증상이 16 시간 늦게 나왔다, 이미 떠 있는 파드는 연결을 유지하고 있어서 멀쩡했고,

재시작하는 순간부터 크래시루프에 빠졌다. '적용 직후 멀쩡했다' 가 무죄의 증거가 아니다.

함정, 잠그기 전 실측이 트래픽을 다 보지 못했다

온프레미스에서 default-deny 를 적용하기 전에 흐름을 실측했는데, :80 리다이렉트 경로가 빠져 있었다.

관측된 트래픽만 허용했더니 그 경로가 끊겼다.

교훈 : 잠그기 전에는 관측을 기다리지 말고 트래픽을 직접 만들어 보고 다시 측정한다.

4.9.4 Secret Manager · External Secrets

비밀은 AWS Secrets Manager 에 저장하고, External Secrets Operator 가 클러스터 안의 Secret 으로 동기화한다.

스토어 mp-aws-secrets · prefix mp/prod/

인증은 IRSA, ESO 서비스 어카운트에 롤을 붙인다

**비밀은 Terraform 이 만들지 않는다, tfstate 에 평문으로 남기 때문이다.
사람이 넣는다**

온프레미스에서도 ESO 를 썼고 백엔드만 Kubernetes provider 였다, 앱 매니페스트를 한 줄도 안 고쳤다

SecretStore, AWS Secrets Manager

```
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata: { name: mp-aws-secrets, namespace: app }
spec:
  provider:
    aws:
      service: SecretsManager
      region: ap-northeast-2
      auth: { jwt: { serviceAccountRef: { name: mp-eso } } }
```

검증

```
$ kubectl -n app get externalsecret
NAME STORE REFRESH STATUS READY
mp-account-env mp-aws-secrets 1h SecretSynced True
```

IAM 정책 작성 시 주의

Secrets Manager 는 시크릿 이름 뒤에 무작위 6 자 접미사를 붙인다.

그래서 리소스 ARN 에 와일드카드가 반드시 필요하다 :

```
arn:aws:secretsmanager:....:secret:mp/prod/*
```

기본 KMS 키를 쓰면 kms:Decrypt 권한은 따로 필요 없다.

함정 : 시크릿 값이 비어 있으면 ESO 가 SecretSyncedError 를 내는데,

증상은 '배포 실패' 로 보이고 원인은 '시크릿 부재' 라 연결이 잘 안 된다.

4.9.5 WAF · GuardDuty · CloudTrail

서비스	구성	용도
AWS WAF	Web ACL 1 개 + 규칙 3 개 · ALB 에 부착	SQL 인젝션 · XSS · 악성 IP 차단 내부 도구는 IP allowlist 로 문지기 역할
Amazon GuardDuty	계정 전역 활성화	이상 API 호출 · 알려진 악성 IP 통신 탐지
AWS CloudTrail	전 리전 · S3 저장 (SSE-S3) · Object Lock	API 호출 감사 기록
EKS 감사로그	audit CloudWatch Logs · 보존 30 일	클러스터 API 호출 감사

보안 알림 경로

GuardDuty / CloudTrail

- ↳ Amazon EventBridge (규칙 매칭)
- ↳ AWS Lambda (포맷)
- ↳ Slack #mp-security

설계 근거 (1), 보안 알림이 Alertmanager 를 거치지 않는 이유

관측 스택(Prometheus · Alertmanager)이 죽으면 보안 알림도 함께 죽는 순환 의존이

생긴다.

그래서 보안 경로는 EventBridge Lambda Slack 으로 완전히 분리했다.

다만 GuardDuty 에서 EventBridge 까지 약 8 분 지연이 있다, 즉시 알림이 아니다.

설계 근거 (2), CloudTrail 을 KMS 가 아니라 SSE-S3 로 암호화한 이유

CloudTrail 로그에 Object Lock 을 걸어 삭제를 막았는데, KMS 로 암호화하면 우회가 가능하다

객체를 지우지 않고 KMS 키를 지우면 로그를 영구히 읽을 수 없게 된다.

즉 '지울 수 없게 만든' 효과가 사라진다. 그래서 의도적으로 SSE-S3 를 선택했다.

레포 비밀 전수 스캔 결과

공개 레포이므로 이력 전체를 대상으로 자격증명을 스캔했다. 실제 키는 0 건이었다.

AKIA 패턴으로 걸린 13 개 커밋은 전부 '커밋하지 말 것' 이라고 적은 문장이었다(오탐).

판정은 단순 문자열이 아니라 정규식(AKIA[0-9A-Z]{16})으로 해야 한다.

4.10 Terraform 을 활용한 IaC

인프라는 전부 코드로 관리한다. 도구는 두 개로 나누고 경계를 명확히 했다, Terraform 은 클라우드 자원의 생성·소멸을, Ansible 은 그 위의 구성을 담당한다.

도구	담당 범위	대상
Terraform	클라우드 자원의 생명주기	AWS (VPC · EKS · IAM · S3 · ECR · ALB · WAF ...) Proxmox VM (온프레미스)
Ansible	OS · 플랫폼 구성	노드 베이스라인 · Cilium · Istio · ArgoCD · Karpenter GitLab EC2 · 온프레미스

		호스트
Kustomize	애플리케이션 매니페스트	config 레포, overlays/onprem · overlays/eks

(1) 디렉터리 구조

```

infra/terraform/
├── backend.tf S3 원격 backend (잠금 = S3 네이티브 락파일)
├── provider.tf
├── vms.tf vms_k8s.tf 온프레미스 Proxmox VM
├── aws-platform/ ★ AWS 플랫폼 (주 작업 대상)
│ ├── vpc_service.tf VPC · 서브넷 7 · IGW · NAT · 라우팅
│ ├── vpc_ci.tf CI VPC (피어링 없음)
│ ├── endpoints.tf S3 Gateway · Interface Endpoint 3 종
│ ├── security_groups.tf SG 7 종
│ ├── eks_cluster.tf EKS 클러스터 · OIDC provider
│ ├── eks_nodegroup.tf 관리형 노드그룹
│ ├── eks_access.tf Access Entry
│ ├── karpenter.tf Karpenter 롤 · 중단 알림 SQS
│ ├── iam_team.tf 사람 IAM 그룹 3 종
│ ├── iam_irsa.tf 워크로드 IRSA 롤 13 종
│ ├── iam_ci_oidc.tf GitLab OIDC provider · mp-ci-job 롤
│ ├── ecr.tf 리포지토리 18 개 · 태그 불변성 · 수명주기
│ ├── elasticache.tf Valkey Multi-AZ
│ ├── alb.tf acm.tf waf.tf
│ ├── s3_observability.tf s3_pg_dump.tf
│ ├── security_audit.tf CloudTrail · GuardDuty
│ ├── security_notify.tf EventBridge Lambda Slack
│ └── ec2_ci.tf CI 서버
├── aws-ai/ AI 서버리스 트랙 (별도 state)
└── mp-dashboard/ 운영 대시보드 EC2
infra/ansible/
├── site.yml 온프레미스 호스트
├── k8s.yml 온프레미스 Kubernetes
├── eks.yml ★ EKS 플랫폼 (eks_cilium · eks_istio · eks_argocd ...)
├── gitlab.yml CI 서버
└── roles/ 55 개 롤

```

(2) 원격 state

backend.tf

```
terraform {
  backend "s3" {
    bucket    = "mp-backup-ap2"
    key       = "aws-platform/terraform.tfstate"
    region    = "ap-northeast-2"
    use_lockfile = true    # S3 네이티브 잠금 — DynamoDB 불필요
    encrypt   = true
  }
}
```

state 백엔드를 PostgreSQL 에서 S3 로 옮긴 이유

온프레미스 초기에는 Terraform state 를 fb-data VM 의 PostgreSQL 에 두었다.
그런데 그 PostgreSQL 이 Terraform 이 관리하는 클러스터 위로 이사하면서,
'인프라를 만드는 도구의 상태가 그 인프라 안에 있는' 순환 의존이 됐다.
클러스터를 잃으면 state 도 함께 잃어 복구 수단이 사라진다. 그래서 S3 로 분리했다.

(3) 실행 절차

Terraform 실행

```
cd infra/terraform/aws-platform
terraform init -backend-config=backend.conf
# -auto-approve 를 쓰지 않는다, 아래 근거 참조
terraform plan -out=tfplan
# 출력에서 반드시 눈으로 확인 : "X to add, Y to change, Z to destroy"
terraform apply tfplan
# 드리프트 점검
terraform plan -detailed-exitcode # 0 = 일치 · 2 = 드리프트
```

Ansible 실행, EKS 플랫폼

```
ansible-playbook -i inventory_aws.aws_ec2.yml eks.yml
ansible-playbook -i inventory_aws.aws_ec2.yml eks.yml --tags eks_cilium # 특정 롤만
```

(4) 관리 범위 (실측)

구분	수량	비고
Terraform 관리 리소스	118	aws-platform 기준 · 드리프트 0

Ansible 롤	55	온프레미스 + EKS + CI
ECR 리포지토리	18	arm64 16 + amd64 2
IRSA 롤	13	워크로드별
Security Group	7	
서브넷	7	서비스 VPC 6 + CI VPC 1
S3 버킷	11	

실측 사고, 공유 state 에서 남의 리소스가 사라졌다

부하 시험을 위해 낡은 체크아웃에서 apply 를 실행했다. apply · 검증 · 재 plan 이 전부 통과했는데,

5 분 뒤 다른 사람이 만든 리소스가 사라져 있었다.

원인 : 오래된 로컬 코드가 최신 state 를 덮어써, 그 사이 추가된 리소스를 '삭제 대상' 으로 계산한 것이다.

진단의 결정타는 CloudTrail 의 Authorize/Revoke 호출 쌍이었다, '빈 state 였을 것' 이라는 가설을 즉시 부정했다.

그리고 -auto-approve 를 쓰지 않은 것이 더 큰 사고를 막았다

plan -out 출력에 진행 중이던 다른 작업의 WAF 변경이 섞여 있는 것이 보였다.

그대로 갔으면 시험 중이던 트래픽이 차단됐을 것이다.

규칙으로 정착 : (1) apply 전 반드시 main 리베이스 (2) plan 출력의 'to destroy' 를 눈으로 확인

(3) '0 to change' 가 아니면 남의 작업이 섞인 것으로 간주하고 멈춘다.

IaC 경계에서 벗어난 것, 정직하게 기록한다

다음은 코드로 관리되지 않는다. 의도적인 예외이며, 각각 이유가 있다.

IAM 사용자 · 그룹 멤버십 · 액세스 키, 키가 tfstate 에 평문으로 남기 때문에 손으로 만든다.

Secrets Manager 의 비밀 값, 같은 이유.

온프레미스 호스트 C 의 VM 생성, VirtualBox 라 Terraform 프로바이더를 쓰지 않는다(Ansible 은 적용된다).

Cloudflare DNS 레코드 · WAF IP allowlist, 토큰 관리 부담 대비 항목 수가 적다.

예외를 '없는 것' 으로 두지 않고 문서에 남기는 것이 중요하다, 재구축 시 여기서 막힌다.

4 장 요약

인프라는 Docker(VM) 온프레미스 Kubernetes AWS EKS 3 단계를 거쳤고, 각 전환은 앞 단계에서 실제로 겪은 한계를 근거로 삼았다.

AWS 로 이전하면서도 매니지드를 무조건 채택하지 않았다, RDS ·

OpenSearch Service · MSK 는 미채택이고, Redis 만 ElastiCache 로 옮겼다. 판단 기준은 '운영 부담이 실제로 줄어드는가' 였다.

같은 Kubernetes 버전 · 같은 CNI · 같은 메시를 유지해 매니페스트와 정책 자산을 그대로 이식했다. 이것이 이전 비용을 가장 크게 줄인 결정이다.

확장은 파드 두 층(HPA · KEDA)과 노드 한 층(Karpenter)으로 구성했고, 무중단 배포는 Blue-Green 으로 1,409 요청 5xx 0 건을 실측했다.

온프레미스는 폐기하지 않고 DR 대기 사이트 겸 크롤 상시 프로덕션으로 재활용해, 두 사이트 구성을 추가 비용 없이 확보했다.

5. 프로젝트 수행 결과

5.1 개발 환경



개발환경

서비스 URL : <https://aws.mealbong.cloud/>

Docker 환경에서 온프레미스 Kubernetes 로 오케스트레이션 구조를 전환한 뒤, 최종적으로 AWS EKS 기반 클라우드 환경으로 서비스를 이전하였다.

구분	개발·운영 환경
Application	React 19.2.7 · TypeScript 6.0.2 · Vite 8.1.1 · FastAPI · Python 3.12
Data	PostgreSQL · Redis · Kafka · Elasticsearch
Container / Orchestration	Docker · Kubernetes · Amazon EKS · Istio
AWS	EC2 · EKS · IAM · Security Group · Amazon Bedrock
Observability	Prometheus · Alertmanager · kube-state-metrics · node-exporter · Grafana Alloy · Loki · Tempo · Grafana

AI / LLM	Gemini Vision · 자체 식재료 인식 모델 · RAG 챗봇 · Amazon Bedrock · Bedrock Guardrails · Titan Embeddings
IaC / Deploy	Ansible · Terraform · GitLab Repository
Load Test	k6 · Karpenter 기반 확장 검증

5.2 구현 내용

Meal Planning 은 냉장고 속 재료, 레시피, 식비 예산, 식자재 가격처럼 따로 관리되던 정보를 하나의 서비스로 연결하여 사용자가 식단 계획부터 장보기·재고·지출까지 함께 관리할 수 있도록 구현하였다. 아래 표는 실제 서비스 전체 기능을 페이지 기준으로 정리한 내용이며, 서비스는 아래 URL 에서 확인할 수 있다.

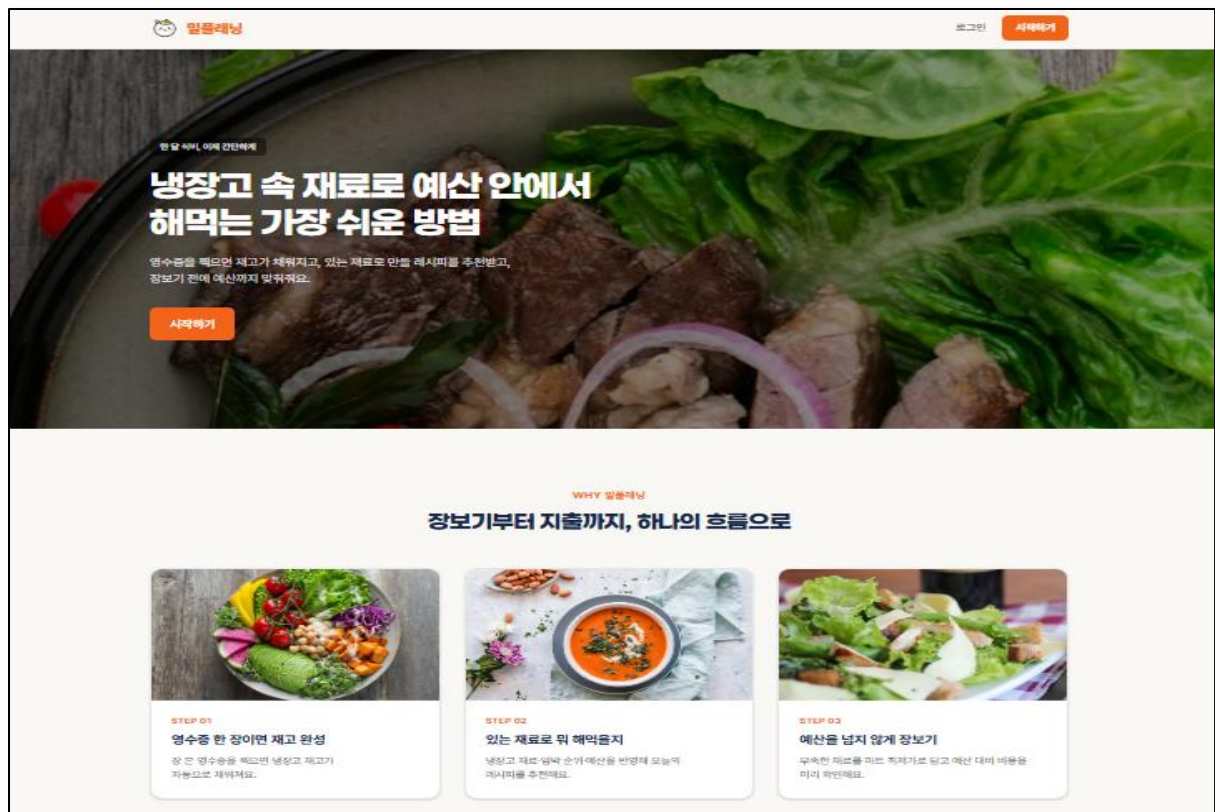
페이지	기능	내용
Landing	서비스 소개	서비스 목적과 이용 흐름을 소개하고 로그인·회원가입 화면으로 이동하는 기능
로그인 / 회원가입	소셜 로그인	카카오·Google OAuth 를 이용해 사용자 인증 후 서비스에 로그인하는 기능
	이메일 인증·예산 설정	이메일 로그인·회원가입을 지원하고 최초 가입 시 월 식비 예산을 설정하는 기능
Home	서비스 요약	냉장고 재고·남은 예산·재료 기반 추천 레시피·가격 하락 식재료를 한 화면에서 확인하는 기능

내 냉장고	재고 관리	식재료를 조회·직접 추가하고 먹음·버림·오입력 삭제로 재고 상태를 관리하는 기능
	영수증 OCR 재료 추가	영수증/이커머스 결제 이미지를 분석해 사용자가 확인한 품목을 냉장고와 식비에 반영하는 기능
레시피	검색·상세 조회	레시피명·재료 검색과 조리시간·난이도 필터를 제공하고 재료·조리순서를 확인하는 기능
	가격·영양·장바구니	컬리·오아시스 가격 비교와 영양정보를 제공하고 레시피북 저장·부족 재료 장바구니 담기를 지원하는 기능
뭐 해먹지?	냉장고 기반 추천	보유 재료 기준 추천 레시피와 재료 보유율·예상 부족분 비용을 제공하고 룰렛 선택을 지원하는 기능
장바구니	구매·예산 계산	재료별 100g 기준 최저가와 구매 후 예상 잔액을 계산하고 구매 완료 시 지출·냉장고 반영을 지원하는 기능
식비관리	예산·지출 관리	월 예산·지출액·잔액과 일별 지출을 조회하고 지출을 직접 기록하는 기능
성과지표	식비·냉장고 성과	예산 잔여율·월말 예상 절약/초과·소비 실천율·집밥 비중 등 소비 성과를 확인하는 기능

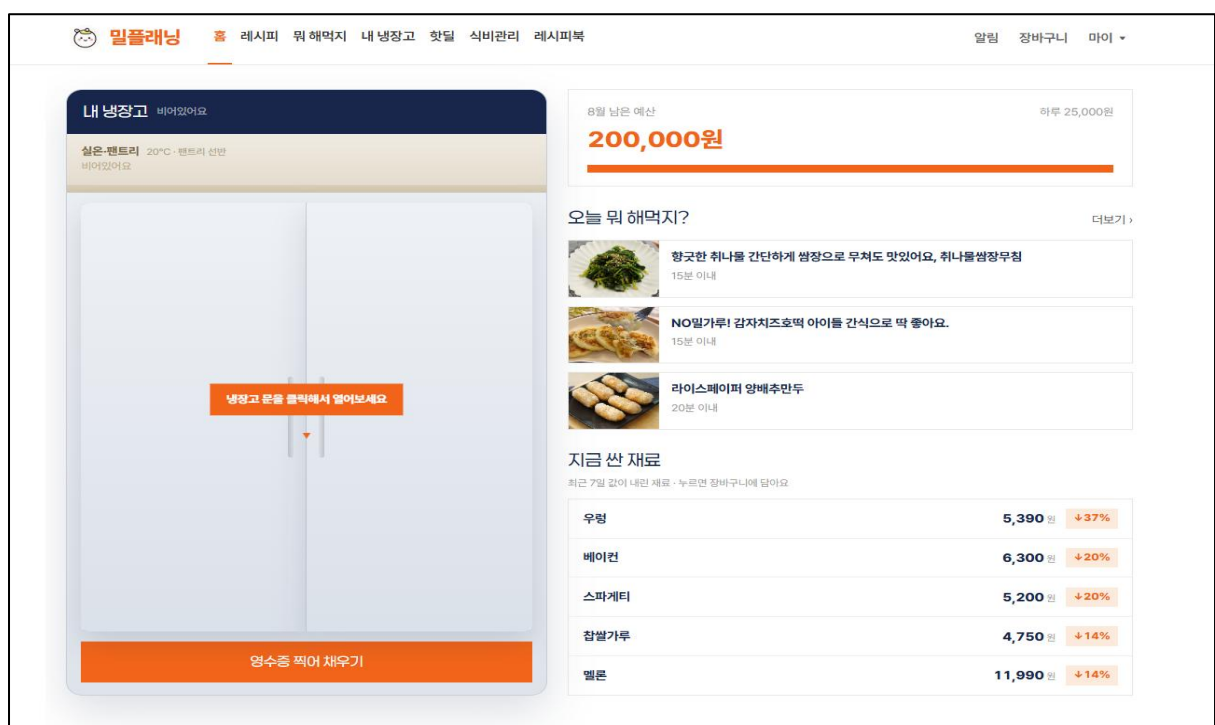
핫딜	가격 확인·관심 품목	마감특가 상품을 조회하고 원본 상품으로 이동하며 관심 품목의 가격 알림 대상을 관리하는 기능
내 레시피북	저장·작성·공유	저장한 레시피와 직접 작성한 레시피를 관리하고 사용자 작성 레시피를 공개 링크로 공유하는 기능
	YouTube 레시피 추출	YouTube URL 에서 재료·조리순서를 추출하고 결과를 확인·수정한 뒤 레시피북에 저장하는 기능
밥풀이 · 식비 어시스턴트	RAG 대화형 지원	레시피·식재료 가격·영양 데이터를 근거로 질문에 답하고 관련 레시피와 후속 질문을 지원하는 기능
알림센터	서비스 알림 관리	최저가·가격 변동·소비기한·예산 알림을 조회하고 읽음 상태와 관련 화면 이동을 관리하는 기능
마이페이지	사용자 설정	로그인 방식·월 예산·제외 재료·레시피북·알림 및 계정 정보를 확인·관리하는 기능

결과 화면

Landing 페이지



메인 페이지



레시피 페이지

[홈](#)
[레시피](#)
[튀 해먹지](#)
[내 냉장고](#)
[핫딜](#)
[식비관리](#)
[레시피북](#)

[알림](#)
[장바구니](#)
[마이](#)

레시피

조리시간

전체
10분 이내
15분 이내
20분 이내
30분 이내
60분 이내

난이도

전체
아무나
초급
중급

총 6,399개 · 24개 표시 · 유저 공유 14

유저 공유
공격수세프의 초간단 투움바 ...
유저 공유 레시피

유저 공유
백종원의 마늘버터볶음밥
유저 공유 레시피

유저 공유
백종원의 참치밥전
유저 공유 레시피

유저 공유
돼지김치찜
유저 공유 레시피

유저 공유
마라탕
유저 공유 레시피

유저 공유
과일 탕후루
유저 공유 레시피

유저 공유
백종원의 누룽지맥숙
유저 공유 레시피

유저 공유
샤브샤브
유저 공유 레시피

유저 공유
테스트레시피
유저 공유 레시피

유저 공유
건우국수
유저 공유 레시피

냉장고 페이지

[홈](#)
[레시피](#)
[튀 해먹지](#)
[내 냉장고](#)
[핫딜](#)
[식비관리](#)
[레시피북](#)

[알림](#)
[장바구니](#)
[마이](#)

내 냉장고

영수증 스캔
재료 추가

냉장고 문을 열고 재료 열 ...로 먹음 버림을 정리하면 소비 실적에 반영돼요. 소비기한 임박은 빨간색 표시돼요. 소금·설탕처럼 기한을 따로 두지 않는 재료는 기한이 비어 있어요 — 포장 표기를 확인해 주세요.

아직 등록된 재료가 없어요. 재료 추가로 시작해요.

내 냉장고 비어있어요

문 닫기

실온·냉동실 20°C · 팬트리 선반
비어있어요

냉장실 3°C

비어있어요

냉동실 -18°C

비어있어요

<재료등록>

1. 영수증으로 등록

영수증으로 재고 등록

촬영하거나 갤러리에서 고르기
종이 영수증 · 이커머스 결제 캡처 지원

- 지원 형식: JPG, PNG (최대 8MB) · 갤러리 사진도 됩니다
- 분석 결과는 저장 전에 직접 수정할 수 있어요
- 확인을 눌러야 재고·식비에 반영돼요 (자동 확정 X)

2. 직접입력

재료 직접 등록

재료명
예: 대파 — 검색해서 표준 재료를 고르면 추천에 반영돼요

수량
1

단위
개

소비기한 (비우면 추정)
연도-월-일

보관 위치
냉장

취소 저장

뭐 해먹지 페이지 (냉장고 기반 레시피 추천)

일플래닝

홈 레시피 **취해역식** 내 냉장고 핫딜 식비관리 레시피북

알림 장바구니 마이 >

취해역식?

냉장과 재료를 만들 수 있는 추천이에요. 큰 집사일수록 재료를 많이 갖고 있어요. 누르면 옆에서 레시피가 열려요.

입막 재료 우선
추천 10개

> 플랫폼으로 정하기

29%

김 없는 삼각김밥 보았나? 난 보았지~! 베이컨삼각주먹밥 ★

재료 29% 보유 · 부족분 2,307원

22%

어묵으로 놀기하고 간단하게! 어묵볶음밥

재료 22% 보유 · 부족분 2,691원

20%

향긋한 취나를 간단하게 심장으로 무쳐도 맛있어요, 취나를쌈장무침

재료 20% 보유 · 부족분 3,996원

20%

도까스 김치 나베

직접한 찹쌀 매뉴 1위!! 도까스 김치나베

재료 20% 보유 · 부족분 3,740원

18%

카레주먹밥 ♥ 콩글루텐 귀여워 Curry Rice balls

재료 18% 보유 · 부족분 1,763원

18%

진밥백선생 신시킴치콘남늘찜

진밥백선생에서 나온 <신시킴치콩나물찜>

재료 18% 보유 · 부족분 2,439원

18%

스팸찌글이

년액독설기만해 ♥ 스팸찌글이

재료 18% 보유 · 부족분 4,698원

18%

베이피데세 뽕통원의 참치통고명면 & 참치떡볶이 & 참치면게랑반까지!

재료 18% 보유 · 부족분 5,940원

17%

밀가루 없이 두부도우로 파자만들기, 두부파자

재료 17% 보유 · 부족분 7,863원

17%

이거 된장으로 만든거 맞을? 조맛 뉴 뉴 된장삼겹살

재료 17% 보유 · 부족분 2,807원

하트 딜 페이지

밀플래닝

홈

레시피

취해먹지

내냉장고

맛달

식비관리

레시피북

알림

잡바구니

마이

맛달

오후 5시부터 자정까지, 하루 한 번 열리는 마감특가예요.

마감특가 미리보기 (50)

ONLY
1
hour

오후 5시부터 자정까지 만날 수 있는!
마감특가 오픈까지 남은시간
00 : 15 : 23

관심 품목 최저가 알림

등록해 두면 가격이 크게 떨어졌을 때 알림을 보내드려요. 같은 품목은 7일에 한 번만 알려요.

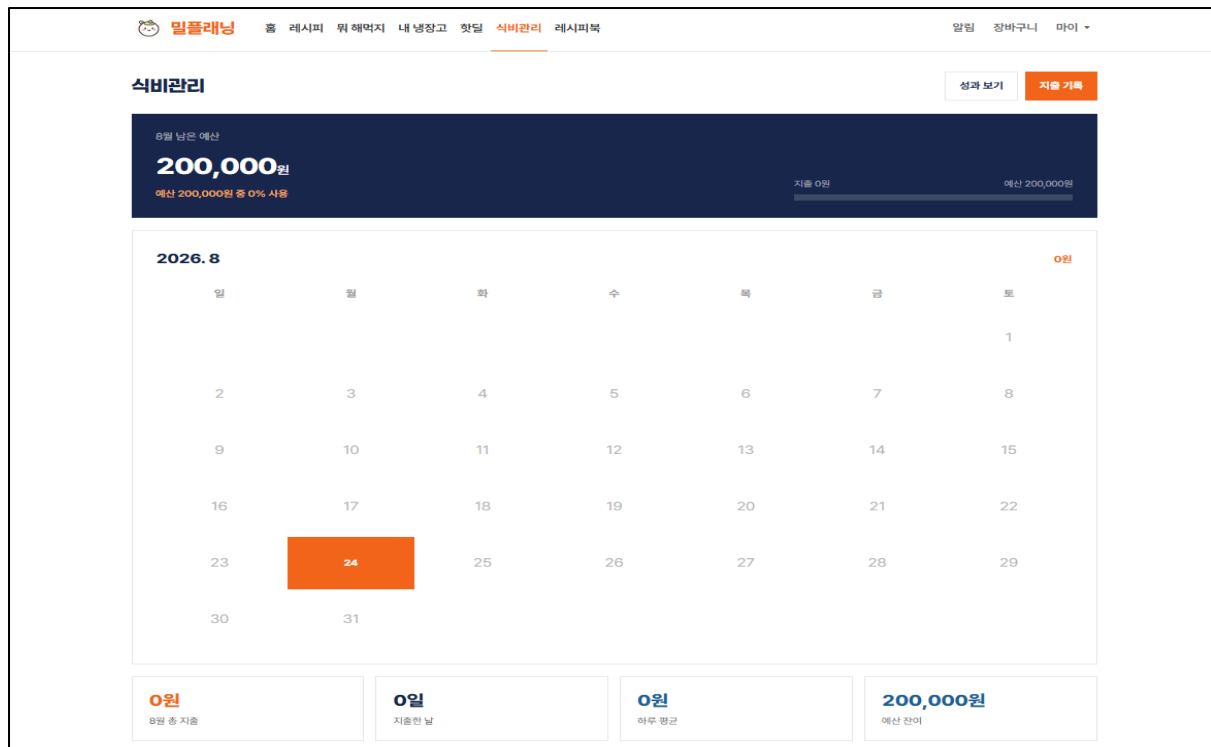
품목 검색 (예: 삼겹살)

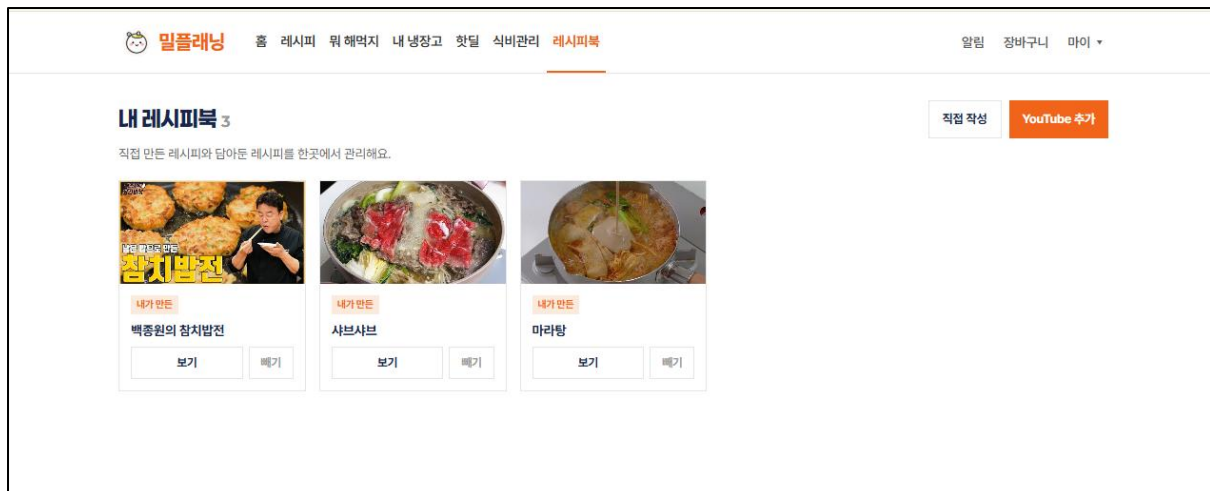
등록한 품목 1개

돼지고기

해제

식비관리 페이지





<레시피 등록>

1.YouTube 레시피추출



2.직접등록

김치찌개

가장 인기 있는
김치찌개

다양한 맛의 김치찌개

보기

김치찌개

다양한 맛의 김치찌개

보기

레시피 직접 작성

레시피 이름

대표 사진 (선택)

업로드 시 자동으로 축소됩니다.

조리 정보 (선택 - 만개 레시피작성 함으로 표시돼요)

재료 (한 줄에 하나씩 - "재료 수량")

김치 100g	
밥 1공기	
대파 1대	

조리 순서 (한 줄에 한 단계)

팬에 기름을 두르고 김치를 볶는다

밥을 넣고 함께 볶는다

저장 후 목록에서 공유를 누르면 링크가 만들어져요.

장바구니 페이지

 **밀플래닝**

홈 레시피 뭐 해먹지 내 냉장고 핫딜 식바구니 레시피북

알림장바구니마이

장바구니

다음은 재료

장바구니가 비어 있어요.
레시피 상세에서 **장바구니 담기**로 재료를 추가해보세요.

이번 달 예산 잔여

200,000원 / 200,000원

이번 달 지출을 반영한 실 잔여예요.

재료 합계 (가격 0/0)

0원

예산 결제

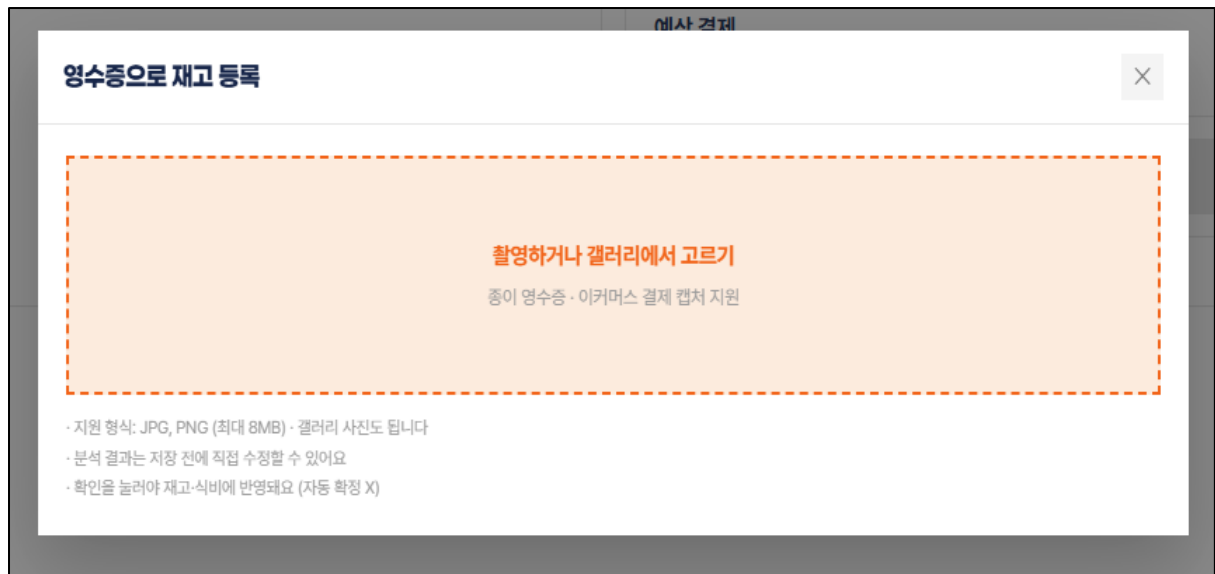
0원

* 100g 최저 단가 × 수량 추정치예요.

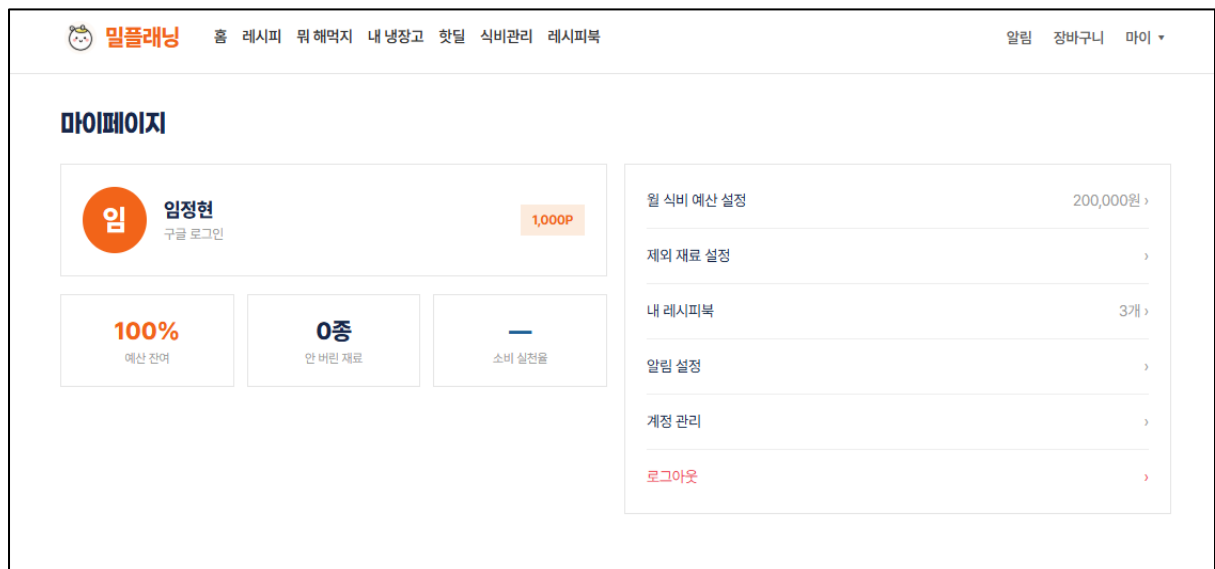
구매 완료 · 지출로 기록

영수증으로 재고 등록

<영수증으로 재고 등록>



마이 페이지



챗봇


밥풀이 · 식비 어시스턴트
 온라인

↻ 새 대화
 ✕

김치찌개 레시피

'콩치 김치찌개', '묵은지 김치찌개', '돼지고기 김치찌개 황금레시피 묵은지 김치찌개 끓이는법' 같은 요리는 어때요?


콩치 김치찌개
 ⌚20분 이내 · 아무나 · 3인분

보기


묵은지 김치찌개
 ⌚20분 이내 · 조금 · 6인분 이상

보기


돼지고기 김치찌개 황금레시피 묵은지 김치찌개...
 ⌚30분 이내 · 아무나 · 6인분 이상

보기

양파 얼마야
 두부로 뭐 해먹지
 김치찌개 레시피

무엇이든 물어보세요...
 ↑